Runtime Evaluation and Mitigation (REM): A Cost-Sensitive and Behavior-Aware Security Framework for Financial AI Agents

Master's thesis — DRAFT FOR SUPERVISOR REVIEW (provisional title; TITLE decision pending)

Al-Balqa Applied University — Faculty of Artificial Intelligence — Department of Intelligent Systems (Cyberspace Security)

Draft of 27 October 2026

Status of This Draft
CHAPTER ONE
INTRODUCTION
1.1 Background
1.2 Problem Statement
1.3 Motivation
1.4 Research Gap
1.5 Research Aim
1.6 Research Objectives
1.7 Research Questions
1.8 Proposed Approach
1.9 Research Methodology
1.10 Scope
1.11 Significance and Expected Contribution
1.12 Current Status of the Research
1.13 Thesis Organization
References
CHAPTER TWO
LITERATURE REVIEW: A SYSTEMATIC REVIEW OF RUNTIME SECURITY FOR TOOL-USING AI AGENTS
2.1 Introduction
2.2 Systematic Review Methodology
2.3 AI Agents and Emerging Security Risks
2.4 Threats Against AI Agents
2.5 Benchmarks and Threat Models
2.6 Defensive Approaches
2.7 Runtime Monitoring and Behavioral Analysis
2.8 Statistical and Algorithmic Foundations Relevant to REM
2.9 Runtime Decision and Mitigation
2.10 Financial AI-Agent Security
2.11 Comparative Evidence Synthesis
2.12 Critical Synthesis
2.13 Research Gap and REM Positioning
2.14 Chapter Summary
References
CHAPTER THREE
RESEARCH METHODOLOGY AND DESIGN OF THE REM FRAMEWORK
3.1 Research Methodology and System Scope
3.2 REM Architecture
3.3 Threat Model and Financial Action Boundary
3.4 Input & Context Layer
3.5 Detection Layer

3.6 Behavioral Analysis Layer
3.7 Decision Engine Layer
3.8 Mitigation Layer
3.9 Offline Fitting and Recalibration
3.10 Algorithm 1: REM Runtime Evaluation and Mitigation
3.11 Experimental Methodology
3.12 Evaluation Metrics
3.13 Contribution and Novelty Boundaries
3.14 Threats to Validity
3.15 Equation Provenance Summary
3.16 Chapter Summary
References
CHAPTER FOUR
IMPLEMENTATION OF THE REM FRAMEWORK
4.1 Purpose and Scope of the Chapter
4.2 Software Environment
4.3 Code Organization
4.4 Layer 1 — Input and Context
4.5 Layer 2 — Detection
4.6 Layer 3 — Behavioral and Action Evidence
4.7 Layer 4 — Decision Engine
4.8 Layer 5 — Mitigation
4.9 Offline Fitting (Procedure 3.1)
4.10 Integration with AgentDojo (HARNESS)
4.11 Optional Sequential Analysis (A2)
4.12 Verification
4.13 Alignment Between Chapter 3 and the Implementation
4.14 Chapter Summary
References
CHAPTER FIVE
EXPLORATORY PILOT EVALUATION
5.1 Purpose
5.2 Data and Protocol
5.3 Pilot E0: Feasibility and Feature Freezing
5.4 RQ1 — Discrimination and Calibration (E1)
5.5 RQ2 — REM versus the Consequence-Independent Baseline, Observe-Only (E2)
5.6 RQ2 and RQ3 — Enforcement **[REPLAY — SC-7]** (E4)
5.7 RQ4 — Ablations (E5)
5.8 Optional Sequential Analysis (E3, A2, observe-only)
5.9 Validity Findings That Qualify the Results
5.10 Realized Loss across the Declared Grid (E6)
5.11 Summary of Evidence by Research Question ([PD])
References
CHAPTER SIX
DISCUSSION, CONTRIBUTIONS, LIMITATIONS, AND CONCLUSION
6.1 Interpretation of the Findings
6.2 Relationship to the Objectives
6.3 Comparison with Prior Work
6.4 Contributions
6.5 Limitations
6.6 Future Work
6.7 Conclusion
References

Status of This Draft

- **Supervisor:** Dr. Husam Fakhouri. **Reviewer:** Prof. Omar.
- **Formatting is provisional.** The university thesis template was not available.
- **Chapter 3 is reproduced unchanged** from its 22 September 2026 version, and **Chapter 2** from its 25 September 2026 systematic-review version.
- **Chapter 1** is the 24 September draft with Sections 1.12–1.13 revised.
- **Chapters 4–6 are new.**
- **Chapter 5 reports an EXPLORATORY pilot, not confirmatory results.** Results that depend on trajectory replay are out of scope under Chapter 3 Section 3.1.4 (SC-7).
- **Open decisions** are listed in `SUPERVISOR_DECISIONS_REQUIRED.md`. The five-chapter alternative is preserved in `REM_STAGE2_OUTPUT/THESIS_COMPLETION/` (SD-STRUCT).

> **Revision note (27 Oct 2026).** Copied from `REM_STAGE2_OUTPUT/THESIS_COMPLETION/CHAPTER1_INTRODUCTION.md` (24 Sep 2026, unchanged on disk). Only Sections 1.12 and 1.13 were rewritten to reflect the implementation, the exploratory pilot and the six-chapter organization. RQ1–RQ4 remain [PD] pending supervisor confirmation. The Platt (1999) entry is under citation review (`SOURCE_VERIFICATION_REPORT.md`).

CHAPTER ONE

INTRODUCTION

1.1 Background

Large language model (LLM) agents use a language model as the controller of a system that perceives, reasons, plans and acts in pursuit of a task (L. Wang et al., 2024). In the execution pattern made explicit by ReAct (Yao et al., 2023), the model interleaves reasoning with actions, and the observations returned by those actions re-enter its context and shape the next step. Tool-using agents act through calls to external tools with arguments that the model generates.

When such an agent operates in a financial setting, a single proposed tool call can have direct external effects:

- initiate a payment;
- schedule or alter a recurring payment;
- change account credentials;
- send customer data to an external recipient.

The security question therefore shifts from what a model says to what an agent does. It concerns the moment at which a proposed action crosses from the agent into its environment (Chapter 2, Section 2.1).

1.2 Problem Statement

An agent receives input through channels of different trustworthiness:

- the system prompt;
- the user’s request;
- tool outputs;
- retrieved content;
- stored memory.

Indirect prompt injection exploits the difference. Content from an external source, processed by the agent, alters its behavior without any access to the user channel (OWASP Gen AI Security Project, 2025; Greshake et al., 2023). Benchmarks designed for tool-using agents establish that this threat is practical, including in environments with financial tools (Debenedetti et al., 2024; Zhan et al., 2024; H. Zhang et al., 2025). The rates they report depend strongly on benchmark, model and attack template, and they cannot be pooled across studies (Chapter 2, Section 2.3).

For a financial agent, the effect of a successful injection becomes visible, and still preventable, at one point: when the agent proposes an action that serves the attacker’s objective. This is the last point at which the action can be prevented rather than repaired (Chapter 2, Section 2.6). A defense placed at this point must decide, for each proposed tool call, whether to release it, modify it, refer it for human review or block it.

That decision raises two questions that the reviewed literature leaves open (Chapter 2, Section 2.10):

- how an estimated probability that an action was adversarially induced should be combined with the consequence of that action when choosing among responses;
- how effective each response actually is.

The problem addressed in this thesis is the runtime compromise of tool-using financial agents through indirect prompt injection, and the design and evaluation of a decision layer at the action boundary that addresses these two questions (Chapter 3, Table 3.1).

1.3 Motivation

Three considerations motivate the work.

First, the consequences in the financial domain are concrete. Tools that transfer value, modify payments or change credentials expose money, account control and personal data to manipulation (Chapter 2, Section 2.7). Conventional evaluation of financial LLM agents, based on accuracy and return metrics, has been argued to give an illusion of reliability while overlooking vulnerabilities such as adversarial prompt manipulation (Z. Chen, J. Chen, et al., 2025).

Second, regulation adds requirements for record-keeping and human control. Regulation (EU) 2024/1689 requires high-risk AI systems to allow the automatic recording of events and effective human oversight, including the ability to intervene (European Parliament & Council of the European Union, 2024). Whether a given financial agent falls within a high-risk category is not assessed in this thesis. The provisions motivate two design properties: an audit record for every decision and an explicit route to human review.

Third, the evaluation of learned security mechanisms is known to be fragile. Pitfalls in the design and evaluation of learning-based security systems lead to unrealistic performance estimates (Arp et al., 2022). Audits of agent-safety benchmarks report metric flaws, inconsistent rankings and contradictory conclusions across benchmarks (Y. Wang et al., 2026; M. Q. Li et al., 2026). A defense for financial agents must therefore be evaluated so that attack success is reported together with benign utility, dependence between benchmark tasks is respected, and leakage between fitting and test data is controlled.

1.4 Research Gap

The gap is not the absence of runtime guardrails, probability calibration, graduated interventions, human review, provenance analysis or consequence-aware control. Each of these is present in the reviewed literature (Chapter 2, Section 2.14). Several recent systems are close to the design studied here:

- **NEXUS** combines rules and argument inspection with a logistic-regression risk score calibrated by Platt scaling. It defines an expected-loss objective with fixed costs for its interventions, while its deployed policy is a rule cascade in which the calibrated score gates certain cases (Hossain et al., 2026).
- **AgentTrust** intercepts tool use and returns one of four verdicts through rule-based severity aggregation (C. Yang, 2026).
- **H.-H. Chen (2026)** prices the consequence of agent actions before execution.
- **FinHarness** addresses runtime safety of financial agents through monitors and a judge cascade (Jia et al., 2026).
- **C. Zhang et al. (2026)** show that calibrating a risk score need not improve control.

Among the studies reviewed in Chapter 2, none reports an empirical evaluation of a non-invasive runtime layer for a financial tool-using agent with all of the following features (Chapter 2, Section 2.14):

- provenance-based evidence about proposed financial actions;
- a calibrated per-step probability that the proposed action is adversarially induced;
- a verdict that minimizes expected loss over Allow, Modify, Escalate and Block, using losses declared per consequence tier;
- deterministic mitigation;
- separate measurement of attack success, benign utility, intervention behavior and latency;
- comparison against a consequence-independent baseline that uses the same estimator.

This gap is empirical and integrative. It concerns the combination and evaluation of established mechanisms, not a new learning algorithm. It is bounded by the sources reviewed and by the September 2026 literature update (Chapter 2, Section 2.14).

1.5 Research Aim

The approved research proposal states objectives but no separate aim statement. The aim below is derived from the approved objectives of Section 1.6 and adds nothing to them.

The aim of this research is to design a runtime evaluation and mitigation framework for the security of financial AI agents, to evaluate it against adversarial attacks, and to compare it with existing security approaches.

1.6 Research Objectives

The approved objectives are (Chapter 3, Section 3.1.2):

- **O1:** Design a REM framework for financial AI agents' and task automation agents' security.
- **O2:** Evaluate the framework against adversarial attacks.
- **O3:** Compare performance with existing security approaches.

The evaluation designed in this thesis uses a financial (banking) environment. Task-automation environments are outside the evaluation scope unless approved as an extension (Chapter 3, Section 3.1.2).

1.7 Research Questions

The approved proposal does not state research questions. The following questions are derived from the objectives in Chapter 3 (Section 3.1.3) and are presented for supervisor confirmation. **Until confirmed, they remain proposed ([PD]).**

- **RQ1 (estimation).** How well does a ridge logistic regression estimator, using context, behavioral and action evidence available at the tool-call boundary, discriminate adversarially induced proposed actions in the AgentDojo banking environment, and how well calibrated are its probabilities under grouped cross-fitting?
- **RQ2 (decision).** Compared with a consequence-independent threshold policy that uses the same estimator, how does consequence-tiered expected-loss verdict selection change attack success, benign utility, utility under attack and intervention rates, across a

declared range of loss structures?

- **RQ3 (mitigation and cost).** What residual attack success remains after the Modify mechanism, how often is escalation selected, and what per-step latency does REM add?
- **RQ4 (component contribution).** What is the marginal contribution of each evidence group, of calibration, and of consequence tiers, as determined by ablation?

1.8 Proposed Approach

The Runtime Evaluation and Mitigation (REM) framework is a runtime mediation layer placed between one agent and its environment. It treats the agent as opaque: it observes inputs, context, proposed actions and tool results, but not model parameters or internal reasoning (Chapter 3, Section 3.3.1). REM comprises exactly five layers (Chapter 3, Section 3.2.1):

1. **Input & Context Layer.** Captures the information the agent handles and the action it proposes, and labels each context segment with its provenance.
2. **Detection Layer.** Computes evidence about untrusted context.
3. **Behavioral Analysis Layer.** Computes evidence about the episode history and the proposed action.
4. **Decision Engine Layer.** Performs the following steps:
    - estimates the probability that the proposed action is adversarially induced, using ridge logistic regression (Cox, 1958; le Cessie & van Houwelingen, 1992);
    - calibrates that probability on the logit (Platt, 1999; Guo et al., 2017);
    - assigns a declared consequence tier and applies policy predicates;
    - selects the verdict among Allow, Modify, Escalate and Block by minimizing conditional risk under tier-indexed losses (Elkan, 2001; Chow, 1970);
    - writes an audit record.
5. **Mitigation Layer.** Executes the mechanism associated with the verdict.

Offline fitting and recalibration are cross-cutting activities performed between deployments. They are not a layer. A linear attribution of the logit (Lundberg & Lee, 2017) is written to the audit record and never influences the verdict.

Declared costs play the role that hand-chosen thresholds play in threshold policies. The threshold between two verdicts becomes a consequence of the declared cost ratio for the action's consequence tier (Chapter 3, Section 3.7.5).

1.9 Research Methodology

The research follows design science research. In design science, knowledge is produced by building an artifact that addresses an identified problem and evaluating it (Hevner et al., 2004), following the six activities of Peffers et al. (2007) (Chapter 3, Section 3.1.1). The artifact is REM. The planned evaluation is quantitative and experimental, in the AgentDojo banking environment (Debenedetti et al., 2024). It compares REM with a baseline that uses the same estimator but a consequence-independent decision rule, so that differences in outcomes can be attributed to the decision rule. Estimation, calibration and threshold selection use nested grouped cross-fitting, because the benchmark does not support independent partitions (Chapter 3, Sections 3.11.4–3.11.5).

1.10 Scope

The scope is set in Chapter 3 (Section 3.1.4).

In scope:

- a single LLM-based agent that invokes tools with generated arguments;
- a financial (banking) task environment with executable tools and state-based checks;
- indirect prompt injection through tool outputs as the evaluated attack class;
- runtime mediation of each proposed tool call before it is released.

Out of scope:

- training-time defenses and modification of the agent's model or prompts;
- multi-agent systems;
- adaptive adversaries who tune attacks to REM;
- production deployment in a financial institution;
- counterfactual estimation of intervention value by replaying agent trajectories.

The design rests on stated assumptions (Chapter 3, Table 3.3). Among them:

- REM can intercept tool calls and results without modifying the agent (AS-1);

- REM's own code, parameters and logs are protected from the adversary (AS-3);
- adversaries do not adapt to REM in the primary evaluation (AS-6).

1.11 Significance and Expected Contribution

The expected contribution is empirical and integrative (Chapter 3, Section 3.13). It consists of the design and evaluation, on an open and executable banking benchmark under indirect prompt injection, of a non-invasive five-layer runtime layer that combines:

- provenance-based action evidence;
- a calibrated per-step probability of adversarial inducement;
- verdict selection by minimizing conditional risk with losses declared per consequence tier;
- deterministic mitigation whose effectiveness is measured.

The evaluation compares this combination with a consequence-independent baseline on the same estimator, and measures security, utility, intervention behavior and latency separately.

The thesis does not claim:

- a new algorithm;
- the first runtime guardrail, calibrated agent risk score or consequence-aware controller;
- that calibration improves control;
- robustness against adaptive adversaries;
- any prevention guarantee;
- validity in production financial deployment.

Each constituent mechanism is established and attributed to its source (Chapter 3, Table 3.16). The contribution holds only to the extent that the specified experiments support it.

1.12 Current Status of the Research

The design is specified in full in Chapter 3. Chapter 4 reports the implementation as built:

- the mathematical core of the Decision Engine (estimation, calibration on the logit, conditional risk and verdict selection);
- the Chapter 3 components, implemented as explicitly labelled exploratory modules:
  - provenance labelling;
  - the evidence features of Tables 3.8–3.9, except the injection classifier, which is blocked;
  - consequence tiers;
  - the loss structure with its dominance check;
  - predicates D1 and D2;
  - the feasible verdict set;
  - the baseline threshold policy;
  - the optional observe-only CUSUM;
  - nested grouped cross-fitting;
- an integration with the AgentDojo banking environment.

The software is verified by 192 passing tests.

Chapter 5 reports an **exploratory pilot**. Its design was declared and committed before any held-out prediction was computed. It uses recorded traces of 21 real LLM agents in AgentDojo banking: 3,334 episodes and 6,762 proposed tool calls. Its results are labelled EXPLORATORY for two reasons:

- the Chapter 3 design values (costs, tiers, predicates, the feature set) await supervisor approval;
- live enforcing-mode runs could not be executed, because no LLM access is available.

Instead, enforcement was estimated by replaying the recorded trajectories, a method that Chapter 3 places outside its scope (Section 3.1.4). This conflict is reported as a supervisor decision; the replay-based results are therefore not presented as fulfilling the Chapter 3 evaluation.

Specification conflicts and pending design decisions await supervisor decision (`SUPERVISOR_DECISIONS_REQUIRED.md`). They include:

- the mitigation semantics, tie-breaking, the calibration ablation, the calibration fitting procedure, the Brier score and the component-to-layer mapping (SC-1 to SC-6);
- the replay conflict (SC-7).

1.13 Thesis Organization

The thesis is organized in six chapters, consistent with the design science activities of Chapter 3 (Table 3.1). The alternative five-chapter organization of the 24 September drafts is preserved and remains a pending decision (SD-STRUCT).

- **Chapter 1** introduces the problem, gap, aim, objectives, research questions, approach and scope.
- **Chapter 2** reports a systematic review of the literature on runtime protection of LLM agents and states the research gap.
- **Chapter 3** specifies the research methodology and the design of REM: its architecture, threat model, algorithms and equations, runtime procedure, experimental methodology and metrics.
- **Chapter 4** reports the implementation as built, the integration with AgentDojo, the verification of the software, and the alignment between Chapter 3 and the code.
- **Chapter 5** reports the exploratory pilot evaluation, including negative results and threats to validity.
- **Chapter 6** discusses the findings against the objectives, research questions and prior work, states the limitations and contributions, and concludes with future work.

References

Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., & Rieck, K. (2022). Dos and don'ts of machine learning in computer security. In *31st USENIX Security Symposium*. USENIX Association. https://arxiv.org/abs/2010.09470

Chen, H.-H. (2026). *Insuring every action: An authority frontier framework for runtime actuarial control of autonomous AI agents* (arXiv:2605.25632) [Preprint]. arXiv. https://arxiv.org/abs/2605.25632

Chen, Z., Chen, J., Chen, J., & Sra, M. (2025). *Standard benchmarks fail — Auditing LLM agents in finance must prioritize risk* (arXiv:2502.15865) [Preprint]. arXiv. https://arxiv.org/abs/2502.15865

Chow, C. K. (1970). On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory, 16*(1), 41–46. https://doi.org/10.1109/TIT.1970.1054406

Cox, D. R. (1958). The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B (Methodological), 20*(2), 215–232. https://doi.org/10.1111/j.2517-6161.1958.tb00292.x

Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., & Tramèr, F. (2024). *AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents* (arXiv:2406.13352). arXiv. https://arxiv.org/abs/2406.13352

Elkan, C. (2001). The foundations of cost-sensitive learning. In *Proceedings of the Seventeenth International Joint Conference on Artificial Intelligence (IJCAI-01)*. https://cseweb.ucsd.edu/~elkan/rescale.pdf

European Parliament & Council of the European Union. (2024). *Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence*. Official Journal of the European Union. https://eur-lex.europa.eu/eli/reg/2024/1689/oj

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security* (pp. 79–90). ACM. https://doi.org/10.1145/3605764.3623985

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning* (Vol. 70, pp. 1321–1330). PMLR. https://proceedings.mlr.press/v70/guo17a.html

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75–106. https://doi.org/10.2307/25148625

Hossain, E., Nipu, M. M. H., Ornee, T. N., Rana, R., & Yousefi, N. (2026). *NEXUS: Structured runtime safety for tool-using LLM agents* (arXiv:2607.19356) [Preprint]. arXiv. https://arxiv.org/abs/2607.19356

Jia, H., Liu, Y., Chong, B., Yang, Y., Chen, Y., Liang, J., Li, Q., Lu, H., Xu, K., Zheng, H., Zhang, C., Peng, H., & Yu, P. S. (2026). *FinHarness: An inline lifecycle safety harness for finance LLM agents* (arXiv:2605.27333) [Preprint]. arXiv. https://arxiv.org/abs/2605.27333

le Cessie, S., & van Houwelingen, J. C. (1992). Ridge estimators in logistic regression. *Applied Statistics, 41*(1), 191–201. https://doi.org/10.2307/2347628

Li, M. Q., Fung, B. C. M., Li, B., Ismail, H., & Iqbal, F. (2026). *Taxonomy and consistency analysis of safety benchmarks for AI agents* (arXiv:2605.16282) [Preprint]. arXiv. https://arxiv.org/abs/2605.16282

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30*. https://arxiv.org/abs/1705.07874

OWASP Gen AI Security Project. (2025). *LLM01:2025 Prompt injection*. OWASP Foundation. https://genai.owasp.org/llmrisk/llm01-prompt-injection/

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems, 24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In A. J. Smola, P. Bartlett, B. Schölkopf, & D. Schuurmans (Eds.), *Advances in large margin classifiers* (pp. 61–74). MIT Press.

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science, 18*(6), Article 186345. https://doi.org/10.1007/s11704-024-40231-1

Wang, Y., Han, X., Shang, D., Tang, Y., & Liu, B. (2026). *Safety, or just capability? A validity audit of agent-safety benchmarks* (arXiv:2607.28685) [Preprint]. arXiv. https://arxiv.org/abs/2607.28685

Yang, C. (2026). *AgentTrust: Runtime safety evaluation and interception for AI agent tool use* (arXiv:2605.04785) [Preprint]. arXiv. https://arxiv.org/abs/2605.04785

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*. https://arxiv.org/abs/2210.03629

Zhan, Q., Liang, Z., Ying, Z., & Kang, D. (2024). InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. https://arxiv.org/abs/2403.02691

Zhang, C., Wan, Z., Yu, X., Wu, J., Wen, Q., Zhou, P., Zhao, W., & Tsang, I. (2026). *Calibration is not control: Why LLM-agent oversight needs intervention* (arXiv:2606.21399) [Preprint]. arXiv. https://arxiv.org/abs/2606.21399

Zhang, H., Huang, J., Mei, K., Yao, Y., Wang, Z., Zhan, C., Wang, H., & Zhang, Y. (2025). Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In *International Conference on Learning Representations*. https://arxiv.org/abs/2410.02644

CHAPTER TWO

LITERATURE REVIEW: A SYSTEMATIC REVIEW OF RUNTIME SECURITY FOR TOOL-USING AI AGENTS

2.1 Introduction

Large language model (LLM) agents no longer only produce text for a human reader. They break tasks into steps, call external tools with arguments they generate themselves, read the results of those calls, and act again on what they read. When such an agent operates on financial tools, a single proposed tool call can:

- initiate a transfer;
- change a payee;
- modify a scheduled payment;
- send customer data to an external recipient.

The security question therefore shifts from what a model *says* to what an agent *does*. More precisely, it shifts to the moment at which a proposed action leaves the agent and takes effect in its environment.

This chapter reviews the literature on protecting such an agent at runtime. Earlier versions of the chapter were structured narrative reviews. This version is a **systematic literature review with qualitative, structured evidence synthesis**. It follows PRISMA 2020 for reporting and PRISMA-S for the search. The methodology in Section 2.2 records what was actually done: which sources were searched and which could not be accessed, the exact search strings, the screening decisions, the quality appraisal and the extraction. Where a step of the systematic-review process could not be carried out in full, the limitation is stated rather than concealed. No meta-analysis is attempted. The included studies differ in architecture, threat model, benchmark, metric and intervention, so their quantitative results cannot be pooled (Section 2.2.8).

The review is used to build an argument in eight steps:

1. agents create security risks that differ from those of conventional LLM use;
2. prior research has characterized the main threats and proposed many defenses;
3. those defenses differ in *where* they intervene;
4. they also differ in *what* they observe;
5. some capabilities are now well addressed;
6. others remain fragmented or weakly evidenced;
7. these patterns locate the Runtime Evaluation and Mitigation (REM) framework studied in this thesis;
8. the financial setting raises the stakes of each of these questions.

The research gap at the end of the chapter was derived only after screening, extraction and synthesis were complete. Section 2.13.1 assesses five candidate gaps against the evidence and retains, narrows or rejects each.

Chapter structure.

- Section 2.2 reports the review methodology.
- Section 2.3 explains why agent architecture changes the security problem.
- Sections 2.4 and 2.5 synthesize the evidence on threats, benchmarks and threat models.
- Section 2.6 organizes defenses by where they intervene.
- Section 2.7 examines runtime monitoring, behavioral evidence and provenance.
- Section 2.8 reviews the statistical and algorithmic foundations on which a runtime decision layer depends. The detailed REM specification is left to Chapter 3.
- Section 2.9 examines how runtime systems decide and mitigate.
- Section 2.10 separates financial-agent evidence from general agent-security evidence.
- Sections 2.11 and 2.12 compare the evidence across studies and synthesize it by theme.
- Section 2.13 states the research gap, positions REM, and records the limitations of the review.
- Section 2.14 summarizes the chapter.

2.2 Systematic Review Methodology

2.2.1 Review objectives and research questions

Derivation. The review questions are derived from the approved thesis direction and introduce no new thesis research questions. The approved proposal states three objectives:

- **O1:** design a REM framework for financial AI agents' and task automation agents' security;
- **O2:** evaluate the framework against adversarial attacks;
- **O3:** compare performance with existing security approaches (Chapter 3, Section 3.1.2).

The proposal does not state research questions. Chapter 3 (Section 3.1.3) derives four questions, RQ1–RQ4, from these objectives. They concern estimation, decision, mitigation and cost, and component contribution, and they remain marked as proposed ([PD]) pending supervisor confirmation. This inconsistency between the proposal and the chapters is recorded here as found. It is not resolved.

Review questions. The review is designed to supply the literature basis that O1–O3, and RQ1–RQ4 once confirmed, require. It asks five review questions (SRQ). These define the scope of this chapter only; they are not thesis research questions.

Review question	Wording	Serves
SRQ1	What security threats affect tool-using AI agents, particularly those operating through external tools and multi-step execution?	O1 (threat model); O2 (attack scenarios)
SRQ2	What defensive approaches have been proposed to detect, monitor, evaluate and mitigate these threats at runtime?	O1 (design space); O3 (comparators)
SRQ3	What behavioral, provenance, probabilistic, sequential and decision-theoretic techniques are used?	O1; RQ1, RQ2 and RQ4 [PD]
SRQ4	What evidence exists for these approaches in financial or financially consequential agent settings?	O1 (financial scope); O2
SRQ5	What limitations and evidence gaps remain that motivate the REM framework?	Research gap (Section 2.13)

2.2.2 Information sources

The review protocol named eleven preferred sources:

- IEEE Xplore;
- ACM Digital Library;
- Scopus or Web of Science;
- ScienceDirect;
- SpringerLink;
- ACL Anthology;
- NeurIPS and PMLR proceedings;
- ICLR proceedings;
- USENIX;
- arXiv, as a supplementary source.

These sources were not directly searchable from the review environment. Direct access was attempted on 25 September 2026, and the network policy refused every connection. The connection log is retained with the search record. An attempt to use Elicit, a second scholarly index, also failed: the account's plan does not include API access. Each of these sources is therefore recorded as **DATABASE ACCESS UNAVAILABLE** (`CHAPTER2_SYSTEMATIC_SEARCH_STRATEGY.md`, §1).

Source actually searched. The one bibliographic database searched was **OpenAlex**, through a literature-search tool that queries its index of about 250 million records. OpenAlex indexes the proceedings and journals of the publishers listed above, as well as arXiv. Studies from those venues could therefore be retrieved, but only through OpenAlex's index and relevance ranking, not through each library's own search interface. The review accordingly does **not** claim to have searched the named digital libraries.

Other methods. A second, non-database stream was used, as PRISMA 2020 allows. It consists of the 70 sources in the reference list of the previous integrated version of this chapter. That list was assembled by known-item searching of the thesis corpus and backward citation checking, and it was verified between 16 and 25 September 2026. Crossref was used only to verify metadata, never to identify records.

2.2.3 Search strategy

Concept blocks. The protocol defines three concept blocks and a financial block:

- **A — Agents:** "LLM agent", "AI agent", "language model agent", "tool-using agent", "autonomous agent";
- **B — Security:** "security", "prompt injection", "indirect prompt injection", "tool misuse", "agent safety", "goal hijacking", "memory poisoning", "action manipulation";
- **C — Runtime, behavior and mitigation:** "runtime monitoring", "runtime interception", "behavioral monitoring", "trajectory", "tool use", "provenance", "anomaly detection", "mitigation", "guardrail", "policy enforcement";

- **Financial:** “financial agent”, “banking agent”, “financial AI agent”, “financial tool”, “financial transaction”, combined with security, prompt-injection, tool-misuse, runtime, safety or mitigation terms.

Database variant. The OpenAlex interface available to the review is lexical and relevance-ranked, and it does not accept Boolean operators. The blocks were therefore translated into eleven free-text queries, each combining at least one term from blocks A, B and C. Two queries (Q7, Q8) carried the financial block. The exact strings are reproduced in Table 2.1. Each query was restricted to publication years 2019–2026. Page 2 was also retrieved for the two queries with the highest page-1 yield of eligible records (Q1, Q2), giving 13 result pages and 320 records in total.

The final search date was **25 September 2026**, and every counted search ran on that date. The raw output of every query is stored unchanged, and a script recomputes all counts from it (`chapter2/slr/`).

Table 2.1. Database searches (OpenAlex; 25 September 2026; years 2019–2026)

Query	Exact search string	Blocks	Records screened	Index estimate
Q1 (pages 1–2)	LLM agent indirect prompt injection runtime defense tool use	A+B+C	50	1,366
Q2 (pages 1–2)	LLM agent security runtime monitoring guardrail tool call	A+B+C	49	1,123
Q3	tool-using language model agent behavioral trajectory monitoring anomaly detection safety	A+B+C	24	5,572
Q4	LLM agent provenance goal alignment tool call misalignment detection	A+B+C	25	484
Q5	prompt injection benchmark tool-integrated LLM agents attacks and defenses evaluation	A+B+C	24	2,950
Q6	LLM agent memory poisoning goal hijacking multi-step attack	A+B	24	579
Q7	financial LLM agent security prompt injection tool misuse	Fin+A+B+C	24	1,368
Q8	banking agent safety benchmark financial transaction LLM agent	Fin+A+B	25	536
Q9	calibrated risk score intervention policy LLM agent runtime safety	A+B+C	25	880
Q10	privilege control policy enforcement LLM agents tool calls prompt injection	A+B+C	25	1,380
Q11	pre-execution action validation guardrail autonomous LLM agent unsafe actions	A+B+C	25	744
Total			**320**	—

Retrieval cap. Only the relevance-ranked top of each query was screened: 24–25 records per page, the maximum the tool returned. The index estimates show that far more records matched the query terms. This cap is a deliberate feasibility decision, consistent with the protocol’s instruction not to screen thousands of records manually. It is also the review’s largest search limitation (Section 2.13.4). The stopping rule, the tool’s handling of each string, and a one-off pilot probe that is not counted are all documented in the search record.

2.2.4 Eligibility criteria

A study was included when it satisfied all six criteria.

- **I1 — Population.** The study addresses AI or LLM agents, tool-using agents or agentic systems. *Operational clarification, fixed before eligibility assessment:* LLM-integrated applications that ingest untrusted third-party content were counted as satisfying I1 when the attack or defense applies to the agent’s input channel, because indirect injection against agents originates in that setting.

Studies whose threat is specific to inter-agent communication in multi-agent systems were treated as outside the review population, which is single tool-using agents.

- **I2 — Problem.** The study addresses a security, safety, robustness, monitoring or misuse problem.
- **I3 — Contribution.** The study contains a substantive technical method, benchmark, empirical evaluation or analytical contribution. Position papers qualified only if they supplied a formal framework, a system design, or an empirical analysis of primary data. Argument-only position papers did not qualify.
- **I4 — Relevance.** The study directly informs at least one REM-relevant dimension: input or context security, runtime monitoring, behavioral analysis, action or tool monitoring, provenance, decision, mitigation, or financial execution security.
- **I5 — Extractability.** The study provides enough information to extract its relevant contribution.
- **I6 — Period.** The study was published between January 2019 and 25 September 2026, or it is a necessary **FOUNDATIONAL SOURCE**.

Exclusion labels. Records were excluded with a standard label:

- wrong population/system;
- wrong threat;
- wrong outcome;
- insufficient technical evidence;
- duplicate;
- outside scope;
- inaccessible full text or insufficient information;
- secondary study (survey, review, SoK or tutorial);
- metadata conflict;
- withdrawn.

Secondary studies and methodological sources. Secondary studies were not synthesized as primary evidence. Three of them, identified by the database search, are cited only as background: two definitional surveys and a finance SoK.

Methodological sources that do not concern agents also fail I1. These are the statistical, calibration, change-detection, decision-theoretic and anomaly-detection sources, and they were admitted under the second clause of I6 as **FOUNDATIONAL SOURCES**. Those published within the review period are labelled methodological sources. They support Section 2.8 and do not count as included studies.

The prior-art studies named in the protocol were given no special route. Each of AgentTrust, NEXUS, ProvenanceGuard, FinHarness, AgentDojo, AgentHarm, Progent, AgentSpec, ShieldAgent, Task Shield, MELON, CaMeL and LlamaFirewall passed through the same criteria.

2.2.5 Study selection

Selection had two stages, both carried out by a single reviewer.

1. **Title and abstract screening** of the 219 records that remained after automated deduplication. Records counted as duplicates when they shared a DOI or their normalized titles matched; this removed 101 of the 320 hits.
2. **Eligibility assessment** of the records that passed screening.

Full texts were not retrieved. The network policy that blocked the databases also blocked publisher and preprint servers. Every report sought for retrieval was therefore assessed for eligibility on **record-level evidence**: title, abstract, indexed metadata, Crossref metadata and, for two records without an indexed abstract (VIGIL and R-Judge), the abstract text returned by a web search. This departure from standard practice is shown explicitly in the PRISMA flow diagram (Figure 2.1).

Other-methods stream. The 70 other-methods records were assessed in the same way against the prior verification record. Eleven were the same study as a database record and were counted once.

Screening log. Every decision, with its reason, is in `CHAPTER2_SCREENING_LOG.csv`.

Table 2.2. Study selection (counts computed from the screening log)

Stage	Databases (OpenAlex)	Other methods
Records identified	320	70
Duplicates removed before screening	101	11 (same study as a database record)
Records screened (title/abstract)	219	—
Records excluded at screening	174	—
Reports sought / full text not retrieved	45 / 45	59 / 59
Reports assessed for eligibility (record level)	45	59

Stage	Databases (OpenAlex)	Other methods
Reports excluded at eligibility	5	28 (6 background, 21 foundational/methodological, 1 withdrawn)
Studies included	**40**	**31**
Total included studies	**71**	

Exclusions at screening. The 174 records excluded at screening break down as follows:

- 81 outside scope;
- 71 secondary studies;
- 7 multi-agent-channel studies;
- 7 wrong threat;
- 4 wrong population;
- 2 official reports;
- 1 wrong outcome;
- 1 duplicate.

Exclusions at eligibility. Five records passed screening but were excluded at eligibility:

- a retrieval-augmented question-answering system without tool actions (wrong population/system);
- a record whose index metadata attached AgentDojo's DOI and authors to an unrelated abstract (metadata conflict);
- an argument-only position paper (insufficient technical evidence);
- an observability taxonomy without a security evaluation (wrong outcome);
- a record without an abstract (insufficient information).

Recall of the database search. Of the thirteen named prior-art studies, the database search retrieved three: Task Shield, LlamaFirewall and, with corrupted metadata, AgentDojo. The other ten entered through the other-methods stream. This low recall of known relevant studies is itself a finding about the search, discussed in Section 2.13.4.

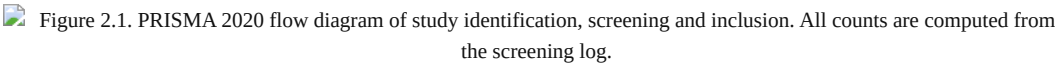


Figure 2.1. PRISMA 2020 flow diagram of study identification, screening and inclusion. All counts are computed from the screening log.

Figure 2.1. PRISMA 2020 flow diagram. Counts are computed by script from the stored search output and the screening log.

2.2.6 Quality appraisal

Every included study was appraised against the protocol's ten-item checklist. The items ask whether the study reports:

- Q1 — its objective;
- Q2 — its threat model;
- Q3 — its method;
- Q4 — reproducibility;
- Q5 — its dataset or benchmark;
- Q6 — its metrics;
- Q7 — its comparators;
- Q8 — its limitations;
- Q9 — evidence for its quantitative claims;
- Q10 — peer-review status.

Codes. Each item is coded Yes, No, Not Verified or Not Applicable. Q10 also has a "preprint clearly labelled" code.

Categories. No numeric score is computed. A fixed, published rule assigns each study to a category:

- **HIGH EVIDENCE:** peer reviewed, with benchmark, figures and comparator all reported;
- **MODERATE EVIDENCE:** quantitative results in a peer-reviewed or clearly labelled preprint record, or a peer-reviewed evaluation without verifiable figures;
- **LIMITED EVIDENCE:** everything else.

Override. One override is applied and documented. FinHarness is set to LIMITED EVIDENCE because all of its figures were measured on a withdrawn benchmark.

Results. Of the 71 included studies:

- 10 are HIGH EVIDENCE, 36 MODERATE EVIDENCE and 25 LIMITED EVIDENCE;

- 36 are peer reviewed and 35 are preprints or working papers;
- 62 are FULLY VERIFIED and 9 PARTIALLY VERIFIED. The metadata and cited contribution of each study were checked against its record. PARTIALLY VERIFIED means that some cited detail rests only on search-engine text.

What the categories measure. Because full texts were unavailable, the categories measure **how much of each study's evidence could be verified from its record**, not the study's intrinsic quality (`CHAPTER2_QUALITY_APPRAISAL.md`). Items that depend on the full text (Q4 reproducibility, Q7 comparators, Q8 limitations) are most often Not Verified.

How the categories are used. They qualify how strongly a finding is worded in the synthesis. They were not used to exclude studies.

2.2.7 Data extraction

A single reviewer extracted 21 fields per included study into `CHAPTER2_SYSTEMATIC_EVIDENCE_MATRIX.csv`:

- study, year and venue;
- agent type;
- threat and attack entry point;
- whether the study operates at runtime;
- context, behavioral, tool/action, multi-step and provenance evidence;
- decision mechanism and mitigation;
- benchmark;
- financial context;
- metrics;
- main result and limitation;
- evidence level.

Coding values. Each property is coded **Yes**, **No**, **Not Reported**, **Not Applicable** or **Not Verified**. "Yes" is never entered for a property that is merely absent from the record.

Definitions and verification. Financial context is coded "Yes" only where the study's setting is primarily financial. A general benchmark that contains one financial scenario is coded "No", with a note. Every included study also carries a verification label, FULLY VERIFIED or PARTIALLY VERIFIED, following the thesis rule that search-engine text cannot settle a primary claim.

Numbers. Every number reported in this chapter is taken from the primary report of the study concerned, as recorded in the matrix.

2.2.8 Evidence synthesis

No meta-analysis. Quantitative pooling was not possible. The included studies use different agents, attack sets, benchmarks, success definitions and metrics, and many report only author-selected configurations. A pooled estimate would have no interpretable common parameter.

Structured synthesis. The synthesis is therefore qualitative and structured. Evidence is organized into eleven themes:

1. threats;
2. entry points;
3. observation granularity;
4. runtime intervention;
5. behavioral and trajectory evidence;
6. provenance and goal–action consistency;
7. statistical and algorithmic methods;
8. decision and consequence handling;
9. mitigation;
10. financial-agent evidence;
11. evaluation quality.

Five questions per theme. For each theme the synthesis asks:

- what the literature establishes;
- what remains uncertain;
- what is under-evaluated;
- what differs across studies;
- what this implies for REM.

These syntheses appear at the end of the relevant sections and are consolidated in Section 2.12 (Table 2.10). Conflicting findings are presented side by side rather than resolved in favor of either side.

How results are worded. Author-reported figures are stated as such, with their setting, and are not compared across studies as if they were measured on a common scale. Claims that rest on a single study are worded as single-study findings.

2.3 AI Agents and Emerging Security Risks

2.3.1 From language models to agents

In the survey literature, an LLM-based autonomous agent is a system in which a language model acts as the controller for perception, reasoning, planning and action in pursuit of a task, supported by memory and tool-use modules (L. Wang et al., 2024; background source). Two capability developments made this architecture practical:

- **ReAct** interleaves reasoning with action. The model generates a thought, selects an action, receives the environment's observation, and conditions its next step on that observation (Yao et al., 2023).
- **Toolformer** showed that a language model can learn which application programming interfaces to call, when to call them, and with what arguments (Schick et al., 2023).

Both are capability papers. They fail criterion I2 and are cited only as background. Together they describe the execution pattern that most agent frameworks still follow: a loop of reasoning, tool invocation and observation, which continues until the model judges the task complete.

2.3.2 Why agent architecture changes the security problem

The loop that makes agents useful also creates their attack surface. Three structural properties distinguish agent security from the security of a stand-alone model:

- **Several input channels of different trustworthiness.** The system prompt, the user's request, tool outputs, retrieved documents and stored memory all enter the same context window, and the model interprets all of them.
- **Statefulness across a trajectory.** Content that enters at one step can influence an action several steps later. The unit of analysis is therefore a trajectory, not an input–output pair.
- **Outputs with external effects.** An agent's outputs include tool calls. A successful manipulation does not stay in the text domain; it becomes a state change in an external system.

A component view of agent threats organizes the same observation around the model "brain", memory, tools and environment interaction (Yu et al., 2025; background source).

Two included benchmark studies show that these properties matter empirically:

- **ClawSafety** ran 2,520 sandboxed trials of personal agents with elevated local privileges. Attack success varied from 40% to 75% across five backbone models, and varied sharply with the channel through which the injection arrived. Cross-scaffold experiments showed that safety depended on the whole deployment stack, not on the backbone model alone (Wei et al., 2026; preprint).
- **ToolEmu** showed that agent risk does not require an attacker. In an LM-emulated sandbox with 36 toolkits and 144 test cases, even the safest agent tested failed in 23.9% of cases. Human evaluators judged 68.8% of the failures identified to be valid real-world failures (Ruan et al., 2024).

2.3.3 Implication

The reviewed studies locate agent risk at the interaction between the model, its heterogeneous input channels, its accumulated state and its tools, and ClawSafety adds the agent framework itself. A defense that inspects only the user's input, or only the final text, does not observe the point at which the agent's decision becomes an external effect. The remainder of the chapter therefore attends to **where** defenses intervene and **what** they observe.

2.4 Threats Against AI Agents

The OWASP Gen AI Security Project lists prompt injection first among risks to LLM applications. It distinguishes *direct* injection through the user's prompt from *indirect* injection through external content (OWASP Gen AI Security Project, 2025). This is an official practitioner source and is used only for terminology.

The threat evidence below comes from 23 included studies whose primary role is an attack, benchmark or threat analysis. Studies of defenses contribute threat evidence where they evaluate against named attacks.

2.4.1 Direct and indirect prompt injection

Direct injection. Y. Liu et al. (2024) moved prompt-injection research from demonstrations to systematic evaluation. They formalized attacks in a common framework and benchmarked five attacks against ten defenses across ten LLMs and seven tasks. HouYi, a black-box injection technique, found 31 of 36 commercial LLM-integrated applications susceptible, a result validated by ten vendors (Y. Liu et al., 2023; preprint). For a financial agent, direct injection matters less than its indirect counterpart, because the user of such an agent is usually the principal it serves rather than the adversary.

Indirect injection. Greshake et al. (2023) showed that instructions planted in data retrieved at inference time can compromise an LLM-integrated application remotely. Such retrieved prompts can act like arbitrary code execution and control which APIs are called. In tool-using agents, the attacker needs only to influence some content the agent will read. The malicious content then enters through the **observation channel**, *after* the user's request has been received and inspected.

Benchmark evidence. Agent-specific benchmarks quantify the threat:

- **InjecAgent** has 1,054 cases covering 17 user tools and 62 attacker tools. A ReAct-prompted GPT-4 agent was vulnerable in 24% of cases, and success nearly doubled when the injection was reinforced with a "hacking prompt" (Zhan et al., 2024).
- **Agent Security Bench (ASB)** covers ten scenarios with more than 400 tools and 27 attack and defense methods. It reported a highest average attack success rate of 84.30% across 13 backbones, and found current defenses of limited effectiveness (H. Zhang et al., 2025).
- **AgentVigil** automates black-box red-teaming. It achieved 71% and 70% success against agents based on o3-mini and GPT-4o on AgentDojo and VWA-adv, nearly double handcrafted baselines (Z. Wang et al., 2025).
- A demonstration against web-navigation agents embedded universal triggers in page HTML and achieved credential exfiltration and forced clicks on real websites (Johnson et al., 2025). This study is LIMITED EVIDENCE: its abstract gives no figures.

2.4.2 Tool misuse, action manipulation and new tool channels

Tool misuse is the use of a legitimate capability against the user's intent. Action manipulation is its adversarially induced form: a well-formed call to a permitted tool, with attacker-chosen arguments or timing. The included studies show that the tool interface is now an attack surface in its own right:

- **Tool selection.** ToolHijacker injects a malicious tool document into the tool library so that the agent selects the attacker's tool. Six prevention-based and detection-based defenses proved insufficient against it (J. Shi et al., 2026).
- **Tool metadata.** A STRIDE/DREAD threat model of the Model Context Protocol (MCP) identified *tool poisoning*, meaning instructions embedded in tool metadata, as the most prevalent client-side vulnerability. Most of the seven MCP clients tested showed weaknesses in static validation and parameter visibility (C. Huang et al., 2026).
- **Tool servers.** A study of 1,899 open-source MCP servers found MCP-specific tool poisoning in 5.5% of servers and general vulnerabilities in 7.2% (Hasan et al., 2026).
- **Tool arguments and plans.** Imprompter optimizes obfuscated prompts that cause improper tool use; its information-exfiltration attack succeeded end to end in nearly 80% of trials against a production agent (Fu et al., 2024; preprint). AI² builds semantically harmless inputs that hijack an application's action plan. It reports an average success rate of 84.30% and bypasses common safety filters in 92.7% of cases (Y. Zhang et al., 2024; preprint).
- **Framework tools.** In LLM-integrated frameworks, prompt injection reaches code-execution and database tools. LLMSmith found 20 vulnerabilities in 11 frameworks, 19 of them remote code execution, of which 17 were confirmed and 11 received CVEs (T. Liu et al., 2023; preprint). Prompt-to-SQL injection was shown to be pervasive across seven LLMs in LangChain applications (Pedro et al., 2023; preprint).
- **Skills.** Reusable agent "skills" create a supply-chain channel. SkillJect automatically poisons skill instructions and helper scripts and outperforms manual skill-injection attacks (X. Jia et al., 2026; preprint; LIMITED EVIDENCE).

Malicious users. Harmful behavior can also be requested directly. AgentHarm contains 110 malicious agentic tasks (440 with augmentations) in 11 harm categories (Andriushchenko et al., 2025). Its threat model, a malicious *user*, contrasts with REM's primary threat model, in which the adversary sits in the data read by a benign user's agent.

2.4.3 Context and memory manipulation

Where an agent retrieves from memory or a knowledge base, an adversary can plant content that is acted on later. AgentPoison reported an attack success rate of at least 80%, with benign impact of at most 1%, at a poison rate below 0.1% (Z. Chen et al., 2024). Memory poisoning separates the write from the activation in time. To an observer without provenance information, the eventual retrieval therefore looks legitimate.

2.4.4 Multi-step propagation, goal drift and abnormal execution

Some compromises contain no single anomalous step. Several runtime systems target this pattern:

- goal-conditioned drift detection (C. L. Wang et al., 2025);
- cross-turn drift in a financial query monitor (H. Jia et al., 2026);
- hazardous drift through individually benign actions (W. Lin et al., 2026);
- task-level transactions that expose cross-step violations (Z. Chen et al., 2026).

Abnormal execution. Abnormal execution can itself be the attack. Malfunction amplification misleads agents into repetitive or irrelevant actions. It induced failure rates above 80% in several scenarios, and its authors found it harder to detect than overtly harmful attacks (B. Zhang et al., 2025). In anomaly-detection terms, a sequence of individually normal events whose *combination* is anomalous is a *collective* anomaly (Chandola et al., 2009; foundational source).

Misaligned agents. A distinct threat is posed by an agent that is itself misaligned. Monitors for covert "scheming" operate on observable trajectories. Monitors trained on synthetic data generalized to more realistic environments, but their performance saturated quickly (Storf et al., 2026; preprint).

2.4.5 Adaptive attackers

Adaptive attacks bypassed all eight indirect-injection defenses evaluated by Zhan et al. (2025), with attack success consistently above 50%. ToolHijacker (J. Shi et al., 2026) and AI² (Y. Zhang et al., 2024) report similar defeats of the defenses they tested. Detection results measured against fixed attack templates are therefore upper bounds on robustness, not estimates of it.

Table 2.3. Threat classes and the included evidence

Threat class	Entry point	Temporal scope	Why it is hard to detect	Included studies (evidence level)
Direct prompt injection	User channel	Single turn	Injected instructions resemble legitimate ones	Y. Liu et al. 2024 (MOD); Y. Liu et al. 2023 (MOD)
Indirect prompt injection	Observation channel (tool outputs, retrieved content, web pages)	Cross-step	Enters after input inspection	Greshake et al. 2023 (MOD); Zhan et al. 2024 (HIGH); H. Zhang et al. 2025 (HIGH); Z. Wang et al. 2025 (HIGH); Johnson et al. 2025 (LIM); Wei et al. 2026 (MOD)
Tool channel attacks (selection, metadata, servers, skills)	Tool library, tool metadata, skill files	Setup-time or cross-step	Poisoned artefacts are trusted as configuration	J. Shi et al. 2026 (LIM); C. Huang et al. 2026 (MOD); Hasan et al. 2026 (HIGH); X. Jia et al. 2026 (LIM)
Tool misuse / action manipulation	Tool arguments; action plans	Step or chain	Call is well formed; anomaly is relational	Fu et al. 2024 (MOD); Y. Zhang et al. 2024 (MOD); T. Liu et al. 2023 (MOD); Pedro et al. 2023 (LIM); Andriushchenko et al. 2025 (MOD)
Context and memory manipulation	Memory, retrieval corpus	Cross-session	Write and activation separated in time	Z. Chen et al. 2024 (MOD)
Goal drift / multi-step propagation	Trajectory	Trajectory	No single step need be anomalous	C. L. Wang et al. 2025 (MOD); W. Lin et al. 2026 (MOD); Z. Chen et al. 2026 (LIM)
Abnormal / repetitive execution	Trajectory	Trajectory	Anomaly lies in the pattern	B. Zhang et al. 2025 (MOD)
Misaligned (scheming) agent	Agent itself	Trajectory	Covert pursuit of another goal	Storf et al. 2026 (LIM)
Adaptive attacks	Any	Any	Designed against the defense	Zhan et al. 2025 (MOD); J. Shi et al. 2026 (LIM); Y. Zhang et al. 2024 (MOD)

Evidence levels: HIGH, MOD (moderate) and LIM (limited), as defined in Section 2.2.6.

Themes 1 and 2 (threats; entry points): synthesis.

- **Established.** Indirect injection through the observation channel is demonstrated, benchmarked and adaptive. Its success rates are substantial across independent benchmarks, and the defenses of the time failed against adaptive attacks. The harm materializes when the agent proposes a tool call.
- **Uncertain.** Reported success rates differ by an order of magnitude across benchmarks, models and attack channels: from 24% on InjecAgent to 84.30% average on ASB, and 40–75% across models on ClawSafety. Those differences cannot be attributed to any single factor.
- **Under-evaluated.** The newer tool-channel threats (tool metadata, tool libraries, skills and MCP servers) are documented mainly by attack and measurement studies. Few runtime defenses have been evaluated against them. Abnormal and repetitive execution is evidenced by one peer-reviewed study.

- **Differs across studies.** The threat actor varies: an attacker in the data, a malicious user, a poisoned tool supplier, or a misaligned agent. So does the success criterion, which may be an attacker goal achieved, an unsafe action executed, or a task failure.
- **Implication for REM.** The observation point for REM is the proposed tool call. That is where every threat class above converges on an external effect. REM's primary threat model, indirect injection against a benign user's financial agent, is the best-evidenced class. Tool-channel threats and misaligned-agent threats lie outside REM's evaluated scope and should be stated as such in Chapter 3.

2.5 Benchmarks and Threat Models

Benchmarks operationalize threat models, and they bound what a defense can be shown to achieve. Table 2.4 evaluates the benchmarks used by included studies on seven dimensions:

- what the benchmark actually measures;
- its threat model;
- its execution realism;
- its financial relevance;
- its multi-step coverage;
- whether it measures utility;
- its limitations.

Table 2.4. Critical evaluation of benchmarks used by included studies

Benchmark (source)	What it measures	Threat model	Execution realism	Financial relevance	Multi-step coverage	Utility measured	Limitations
AgentDojo (Debenedetti et al., 2024)	Task utility, utility under attack and targeted attack success per episode; 97 user tasks, 629 security cases	Indirect injection via tool outputs	Executable, stateful tools; outcomes checked against environment state	One of four suites is Banking; no finance-specific analysis in the verified record	Yes (multi-tool episodes); number of genuinely multi-step attacks not verified	Yes, jointly with security	Limited number of distinct tasks; robustness claims bounded by the attacks evaluated (Section 2.4.5)
InjecAgent (Zhan et al., 2024)	Whether the next action after one injected tool response serves the attacker; 1,054 cases	Indirect injection (direct harm; data exfiltration)	Simulated single tool response	Not reported	No (short cases)	No	Isolates injection susceptibility; no benign utility
ASB (H. Zhang et al., 2025)	Attack success across stages; utility–security trade-off metric	Direct/indirect injection, memory poisoning, backdoor, mixed	Agent framework with >400 tools	Finance is one of 10 scenarios; not analysed separately	Yes (multiple stages)	Yes	Breadth over depth per scenario
AgentHarm (Andriushchenko et al., 2025)	Whether harmful multi-step tasks are completed	Malicious user	Synthetic tools	Fraud is one of 11 categories	Yes	Capability retention after jailbreak	Different threat actor from REM's
ToolEmu (Ruan et al., 2024)	Risky failures without adversary; 144 cases	No adversary	LM-emulated tools	Not reported (financial loss among risk examples)	Yes	Not the focus	Emulator and evaluator are LMs
R-Judge (Yuan et al., 2024)	LLM ability to judge safety risk in 569 multi-turn records	Behavioural safety risks	Recorded interactions (offline)	Not reported	Yes	Not applicable	Offline judgement, not runtime interception
ClawSafety (Wei et al., 2026)	Attack success in high-privilege workspaces; 120 scenarios, 2,520 trials	Injection via skills, email, web	Sandboxed real frameworks	Finance is one of five domains	Not verified	Not verified	Preprint; release announced
SIREN (J. Lin et al., 2026, VIGIL)	Tool-stream injection defense; 959 cases	Injection via tool metadata and feedback	Not verified	Not reported	Not verified	Utility under attack	Introduced with the defense evaluated on it
TS-Bench (Mou et al., 2026, ToolSafe)	Step-level detection of unsafe tool invocations	Unsafe invocations incl. injection	Not verified	Not reported	Yes (interaction history)	Benign completion	Introduced with the guardrail evaluated on it

Benchmark (source)	What it measures	Threat model	Execution realism	Financial relevance	Multi-step coverage	Utility measured	Limitations
NEXUS-Bench (Hossain et al., 2026)	Plan-level intervention accuracy; 128 held-out synthetic instances	Unsafe plans	Author-generated templates	Not reported	Plan-level	Paired benign cases	Authors: in-distribution results are upper bounds
FinVault (Z. Yang et al., 2026)	Financial-agent security (as originally reported)	Financial-agent attacks	Execution-grounded (as originally reported)	Yes	Not verified	Not verified	**Withdrawn by its authors; not evidence**

Audits of benchmark validity. Two included meta-research studies qualify how far any of these scores can be read as measurements of safety:

- M. Q. Li et al. (2026; preprint) catalogued 40 agent-safety benchmarks and found no ranking concordance across evaluation dimensions (Kendall's *W*). Benchmark choice alone can therefore change which system appears safest.
- Y. Wang et al. (2026; preprint) found that on one binary trace-judgment benchmark an "always positive" baseline outranked five models under F1. They also found that a cross-benchmark correlation moved from −0.64 with seven models to +0.02 with eighteen, an artefact of small panels.

The machine-learning-for-security literature anticipated these problems. Sampling bias, inappropriate baselines, base-rate neglect and lab-only evaluation recur in top-venue security papers (Arp et al., 2022; methodological source).

Benchmarks built with the defense. Two further observations come from the extraction. First, six included defense or benchmark studies introduce the benchmark on which their own method is evaluated: SIREN, TS-Bench, SciSafetyBench, NEXUS-Bench, PromptShield and the MCP policy-enforcement point's controlled dataset. Second, most of these report no evaluation on an independent benchmark in the verified record. NEXUS is an exception: it reports out-of-distribution results on R-Judge.

AgentDojo in particular. AgentDojo receives particular attention because of its executable Banking suite, but the evidence does not support calling it the best benchmark. Its advantages for REM's question are specific:

- tools execute and change environment state;
- utility and security are measured separately;
- a Banking suite exists.

Its limitations are equally specific:

- a small number of distinct user tasks;
- robustness claims bounded by the attacks actually evaluated (Section 2.4.5);
- no verified finance-specific analysis in the literature reviewed here.

The number of distinct tasks, not the number of generated runs, bounds what an evaluation on it can conclude (Chapter 3 treats this under grouping).

Theme 11 (evaluation quality) — benchmark component.

- **Established.** Benchmark results are benchmark-specific and do not transfer.
- **Uncertain.** How many genuinely multi-step attacks the common benchmarks contain.
- **Under-evaluated.** Independent evaluation of defenses on benchmarks their authors did not build.
- **Differs across studies.** Success criteria and utility definitions.
- **Implication for REM.** Results must be reported per benchmark, with benign utility, and interpreted against the number of distinct tasks. FinVault cannot be used.

2.6 Defensive Approaches

The 42 included defense and design studies differ along two dimensions that the literature rarely separates:

- **where they intervene:** at the input, in the context, in the model, in the system architecture, at the policy or tool layer, around the agent loop at runtime, or after an action has taken effect;
- **what they observe:** an isolated input, an individual tool call, a plan, the flow of data between steps, a trajectory, or the consequence of an action.

This section organizes defenses by where they intervene. Section 2.7 examines what runtime monitors observe, and Section 2.9 examines how they decide and mitigate.

Table 2.5. Where included defenses intervene and what they observe

Point of intervention	Included studies	Primary unit observed	What this position can and cannot see
Input / context text	InstructDetector (Wen et al., 2025); PromptShield (Jacob et al., 2025); attack-technique defenses (Y. Chen et al., 2025); Spotlighting (Hines et al., 2024); PromptGuard 2 in LlamaFirewall (Chennabasappa et al., 2025)	An input or retrieved text	Sees text before the model acts; does not see how the agent then acts
Model	StruQ (S. Chen et al., 2025)	Model behaviour under separated channels	Addresses the instruction/data confusion; requires training and model access
System architecture / information flow	f-secure IFC (F. Wu et al., 2024); RTBAS (Zhong et al., 2025); ACE (E. Li et al., 2026); IsolateGPT (Y. Wu et al., 2025); CaMeL (Debenedetti et al., 2025)	Data flow into planning and between tools or apps	Can prevent untrusted data from steering control flow; changes how the agent executes
Policy / privilege at the tool boundary	Progent (T. Shi et al., 2025); AgentSpec (H. Wang et al., 2026); Conseca (Tsai & Bagdasarian, 2025); MCP policy-enforcement point (S. Wang et al., 2026); LATTICE (Calboreanu, 2026)	Each tool call against a policy	Deterministic and auditable; answers “is this permitted?”, not “how likely is this induced?”
Plan or reasoning	TrustAgent (Hua et al., 2024); Thought-Aligner (Jiang et al., 2025); NEXUS (Hossain et al., 2026); DRIFT planner (H. Li et al., 2025)	A plan or intermediate thought	Can intervene before actions exist; plans can change after new observations
Runtime around the agent loop	Task Shield (F. Jia et al., 2025); MELON (Zhu, Yang, et al., 2025); ProvenanceGuard (She et al., 2026); VIGIL (J. Lin et al., 2026); ToolSafe (Mou et al., 2026); DRIFT validator (H. Li et al., 2025); GuardAgent (Xiang et al., 2025); ShieldAgent (Z. Chen, Kang, & Li, 2025); AGrail (Luo et al., 2025); SafeScientist (K. Zhu et al., 2025); AlignmentCheck (Chennabasappa et al., 2025); AgentTrust (C. Yang, 2026); MI9 (C. L. Wang et al., 2025); PRISM (F. Li, 2026); ProbGuard (H. Wang et al., 2025); DreamGuard (Lin et al., 2026); SafeAgent (H. Liu et al., 2026); FinHarness (H. Jia et al., 2026); constitutional monitors (Storf et al., 2026)	Proposed action in context; sometimes the trajectory	Observes the action boundary; costs are latency and false interventions
Post-action / transactional	Cordon (Z. Chen et al., 2026); GoEX (Patil et al., 2024)	Staged or reversible effects	Can hold, undo or confine effects; consumes rather than produces a risk judgement
Consequence gating	Actuarial Action Interface (H.-H. Chen, 2026)	Priced action	Prices consequence; does not estimate adversarial induction

2.6.1 Input-, context- and model-level protection

Input detectors. Detectors that act on text report strong in-distribution results:

- InstructDetector uses a model’s hidden states and gradients to detect embedded instructions. It reports 99.60% in-domain and 96.90% out-of-domain accuracy, and reduces attack success to 0.03% on BIPIA (Wen et al., 2025).
- PromptShield curates a benchmark for deployable detectors, and its fine-tuned detector improves performance in the low-false-positive regime (Jacob et al., 2025).
- Spotlighting transforms untrusted input so that the model receives a continuous provenance signal, reducing attack success from above 50% to below 2% with GPT-family models (Hines et al., 2024).

Model-level protection. StruQ trains the model to follow instructions only in the prompt channel (S. Chen et al., 2025). This addresses the root cause identified by Greshake et al. (2023), but it requires training and access to the model. That is a significant constraint for a financial institution that deploys a third-party model.

Shared limitations. These approaches share two limitations for tool-using agents:

- they act on text, not on actions, so an input detector does not observe what the agent then does;
- ToolHijacker defeated six such defenses (J. Shi et al., 2026), and AI² bypassed common safety filters in 92.7% of cases (Y. Zhang et al., 2024).

2.6.2 System-level and information-flow defenses

A second family changes the system so that untrusted data cannot steer control flow:

- **f-secure LLM systems.** A security monitor filters untrusted input out of the planning process, and formal models state the security guarantee (F. Wu et al., 2024; preprint).
- **RTBAS.** Information-flow control is adapted to tool-based agents. Tool calls whose integrity and confidentiality can be established are executed automatically, and the user is asked to confirm the rest. On AgentDojo it prevented all targeted attacks with a 2% utility loss under attack (Zhong et al., 2025; preprint).
- **ACE.** Planning is decoupled: an abstract plan is built from trusted information only, its information-flow constraints are checked statically, and data and capability barriers are enforced during execution. It reports security against InjecAgent and ASB attacks, and it demonstrates new attacks against IsolateGPT (E. Li et al., 2026).
- **IsolateGPT.** Third-party apps run in isolation, with overhead under 30% for three-quarters of tested queries (Y. Wu et al., 2025).
- **CaMeL.** Control and data flow are extracted from the trusted query and capability policies are enforced at each tool call. CaMeL solved 77% of AgentDojo tasks with provable security, against 84% for the undefended agent (Debenedetti et al., 2025, v2; an earlier version reported 67%).

Conflicting evidence. The IsolateGPT–ACE pair is a direct conflict within this family. One peer-reviewed study reports that isolation protects against the attacks it tested. A later peer-reviewed study constructs attacks that defeat it. Architectural guarantees hold only for the threat models their authors formalized.

2.6.3 Policy enforcement and tool restriction

Classical basis. This family descends from execution monitoring. Schneider (2000; foundational source) characterized the policies a monitor can enforce by observing a system's execution and halting it before a violation: the safety properties.

Included instances:

- **Progent** expresses privileges as rules over tool names and arguments, with solver-checked updates. It reduced attack success from 39.9% to 1.0% on AgentDojo and from 70.3% to 3.9% on ASB (T. Shi et al., 2025; preprint).
- **AgentSpec** enforces rules written as trigger, predicate and enforcement. It prevented over 90% of unsafe code-agent executions and all hazardous embodied actions, and gave 100% compliance in autonomous driving, at millisecond overhead (H. Wang et al., 2026).
- **Conseca** argues for policies generated just in time for each task context and enforced deterministically (Tsai & Bagdasarian, 2025; LIMITED EVIDENCE).
- **MCP policy-enforcement point.** A policy-enforcement point for MCP agents intercepts at the tool-call boundary with declarative rules over cross-step information-flow labels, and keeps a hash-chained audit log. On a controlled dataset, it reduced attack success from 40.0% to 5.0%, against 35.0% for the strongest prompt-only baseline (S. Wang et al., 2026). The same study also reports:
  - a task-level false-positive rate of 30.0%, attributed mainly to by-design capability denials;
  - residual attack success of 16.7% on one backend, due to a rule-coverage gap;
  - sub-millisecond overhead.
- **LATTICE** gates execution through policy-as-code with deterministic verdicts, escalates to human operators on a confidence basis, and keeps cryptographic audit trails (Calboreanu, 2026).

Limitations for REM. These studies are fast, auditable and, in the architectural cases, provable for the threats they address. Two limitations recur:

- **The question is binary.** An enforcer answers whether an action is permitted. It does not estimate how likely a *permitted* action is to have been induced by an adversary.
- **Policy coverage bounds protection.** The policy-enforcement point's residual 16.7% attack success on one backend is a rule-coverage gap in its authors' own words.

2.6.4 Learned and LLM-based guardrails

A further family uses learned models or LLMs to judge actions or trajectories:

- **GuardAgent** generates guardrail code from safety requests (Xiang et al., 2025).
- **ShieldAgent** verifies trajectories against probabilistic rule circuits extracted from policy documents (Z. Chen, Kang, & Li, 2025).
- **AGrail** keeps adaptive safety checks in a lifelong memory (Luo et al., 2025).
- **TrustAgent** applies an agent constitution around planning (Hua et al., 2024).
- **Thought-Aligner** corrects unsafe intermediate thoughts. It raised behavioral safety from about 50% to about 90% on average across six LLMs (Jiang et al., 2025).

- **ToolSafe** trains a step-level guardrail that judges each tool invocation before execution from the interaction history. Its feedback-driven framework reduced harmful invocations by 65% on average and improved benign completion by about 10% under injection (Mou et al., 2026).
- **VIGIL** replaces isolation with verify-before-commit. It reduced attack success by over 22% relative to dynamic defenses and more than doubled utility under attack relative to static baselines on its SIREN benchmark (J. Lin et al., 2026; PARTIALLY VERIFIED).
- **DRIFT** combines a secure planner, a dynamic validator of plan deviations and privileges, and an injection isolator for the memory stream. It is evaluated on AgentDojo, ASB and AgentDyn (H. Li et al., 2025).
- **SafeScientist** monitors prompts, agent collaboration and tool use in scientific agents (Zhu, Zhang, et al., 2025).

Limitations. These systems cover a broad range of semantic violations without writing a policy for each. Their limitations also recur:

- LLM judgement adds latency and can vary between runs;
- accuracies are measured on the authors' own benchmarks;
- the outputs are verdicts, and the verified records do not report calibrated probabilities that could be combined with the consequence of an action. The one exception is NEXUS (Section 2.9.3).

2.6.5 Goal–action consistency and provenance

The family most directly aimed at indirect injection asks whether a proposed action is *justified* by the user's task, rather than whether it is harmful in itself:

- **Task Shield** verifies whether each instruction and tool call serves the user's goals. It reduced attack success to 2.07% with 69.79% utility on AgentDojo with GPT-4o (F. Jia et al., 2025).
- **MELON** re-executes the trajectory with the user's prompt masked and flags an attack when the actions remain similar (Zhu, Yang, et al., 2025).
- **ProvenanceGuard** asks whether a tool call is supported by traceable evidence in the context. Relative to an LLM-as-judge baseline, it reduced error on misaligned traces from 44.3% to 2.1% on Agent-SafetyBench and from 32.4% to 18.7% on WorkBench (She et al., 2026; preprint). Its reported intervention rate on aligned traces (14.5% against 10.9%) and its extra cost are PARTIALLY VERIFIED.

These approaches share one insight: under indirect injection, the useful evidence concerns the *relation* between an action and its sources. They differ in cost. MELON adds an execution, and Task Shield and ProvenanceGuard add LLM reasoning.

Themes 3 and 4 (observation granularity; runtime intervention): synthesis.

- **Established.** Runtime intervention at the tool-call boundary is widely implemented: 41 of the 42 included defense and design studies operate at runtime, and 34 inspect tool calls or actions. Deterministic enforcement is fast. Several peer-reviewed studies report sub-millisecond to millisecond overheads.
- **Uncertain.** Robustness to adaptive attacks. Architecture-level guarantees have been broken by later attacks (IsolateGPT–ACE), and input-level detectors have been bypassed (ToolHijacker, AI²).
- **Under-evaluated.** Evaluation of any given defense on a benchmark that its authors did not construct. Evaluation against the tool-channel threats of Section 2.4.2.
- **Differs across studies.** The unit observed differs: input text, tool call, plan, data flow or trajectory. So does the question asked, which may be about permission, alignment, provenance or harm.
- **Implication for REM.** Interception at the tool-call boundary is established, and REM cannot claim it. What the families leave open is the combination of an estimate of *how likely* a permitted, well-formed action is to be adversarially induced with the *consequence* of that action. Section 2.9 examines this.

2.7 Runtime Monitoring and Behavioral Analysis

This section concerns the evidence a runtime layer can extract from the agent's behavior, as distinct from the text it receives. That evidence is the domain of REM's Behavioral Analysis Layer.

2.7.1 Step-level, sequence-level and accumulating monitors

Runtime monitors differ in the temporal scope of their judgment.

Step-level monitors judge each proposed action in its current context. Examples are Progent, AgentSpec, Task Shield, ProvenanceGuard and VIGIL. They are fast and interpretable, but by construction they cannot see an attack whose individual steps are each acceptable.

Sequence-level monitors judge an action against a model of the trajectory:

- MI9 enforces temporal behavioral patterns through finite-state conformance engines and detects goal-conditioned drift. It reports 99.81% detection across 1,033 *synthetic* scenarios (C. L. Wang et al., 2025; preprint; PARTIALLY VERIFIED).
- ProbGuard learns a discrete-time Markov chain over abstract states. It intervenes when the estimated probability of remaining safe falls below a threshold, with PAC-style guarantees on the learned model (H. Wang et al., 2025; preprint).
- DreamGuard maintains a recurrent latent state and fuses immediate-hazard and prefix-risk scores into PASS, HOLD or BLOCK, at about 25 ms per call (W. Lin et al., 2026; preprint).
- ToolSafe's guardrail reasons over the interaction history (Mou et al., 2026).
- Constitutional monitors judge whole trajectories for scheming (Storf et al., 2026).

Accumulating monitors sit between the two. PRISM accumulates session risk with time-to-live decay (F. Li, 2026). AgentTrust applies seven order-aware chain detectors (C. Yang, 2026). FinHarness integrates per-step risk in a cascade (H. Jia et al., 2026).

A formal view. A formal account of runtime governance frames policies as functions of the agent's identity, its *partial execution path*, the proposed next action and the organizational state, mapped to a probability of policy violation. It shows that system prompts and static access control are special cases, and it lists risk calibration as an open problem (Kaptein et al., 2026; preprint; LIMITED EVIDENCE). This formalization supports evaluating actions in the light of their path. It also shows that the idea of a path-conditioned violation probability is not new.

2.7.2 Provenance and cross-step information flow as evidence

A second line of evidence concerns *where an action's content comes from*. In an indirect injection against a banking agent, the attacker typically supplies the decisive arguments inside untrusted content: a recipient account, an amount, a new payee. Whether an argument can be traced to the user's request or to trusted state is therefore strong evidence of adversarial induction.

Included studies. Eight of the 42 included defense and design studies code provenance or information-flow evidence as present (Table 2.8). They realize it in three ways:

- **by analysis:** ProvenanceGuard's evidence tracing;
- **by construction:** CaMeL, ACE and f-secure extract flows from trusted inputs;
- **by labels:** RTBAS's dependency screeners and the MCP policy-enforcement point's labels for source integrity and data sensitivity.

2.7.3 What is known about the empirical value of behavioral and provenance evidence

The review found **conflicting evidence** on whether trajectory and provenance components change outcomes. All three direct tests of a component's contribution are single studies on their authors' own benchmarks.

- **A null result for trajectory evidence.** AgentTrust's ablation found that disabling its session tracker left verdict accuracy unchanged on both of its benchmarks (C. Yang, 2026; PARTIALLY VERIFIED). The author attributes the remaining gap to benchmark-compatibility rules.
- **A positive result for cross-step provenance.** In the MCP policy-enforcement point, disabling cross-step label propagation raised the call-level false-negative rate by 26.4 percentage points on the authors' controlled dataset (S. Wang et al., 2026; HIGH EVIDENCE). The authors frame the result as mechanism-coverage evidence, not a deployment estimate.
- **A positive result for provenance against an LLM judge.** ProvenanceGuard's error reductions relative to an LLM-as-judge baseline indicate that evidence tracing is informative on misaligned tool calls (She et al., 2026; the headline figures are verified at abstract level).
- **Context for these results:**
  - MI9's evaluation was synthetic;
  - ProbGuard's gains were in autonomous-driving and embodied settings;
  - DreamGuard's figures in the verified record concern latency, not detection;
  - constitutional monitors saturated quickly, and further optimization overfitted (Storf et al., 2026).

The conflict is informative rather than contradictory:

- The null result concerns a *session tracker of order-aware chain detectors* in a rule-dominated system on coding- and shell-oriented scenarios.
- The positive results concern *cross-step information-flow labels* and *evidence provenance* against injected or misaligned tool calls.

On the included evidence, it is plausible that provenance-type trajectory evidence contributes under indirect injection while generic sequence patterns may not. No included study tests either kind of evidence as an input to a *calibrated estimator*, and none tests it on financial actions.

2.7.4 Foundations: behavioral anomaly detection and its cautions

Foundational work. The idea of detecting compromise from behavior predates LLM agents. Forrest et al. (1996) built a profile of normal system-call sequences and flagged sequences absent from it. That is the closest classical analogue to monitoring an agent's tool-call sequence. Chandola et al. (2009, 2012) distinguished three kinds of anomaly and formalized anomaly detection over discrete sequences:

- *point* anomalies: in agents, a single out-of-policy call;
- *contextual* anomalies: a transfer that is normal in general but anomalous given its provenance;
- *collective* anomalies: goal drift and multi-step chains.

Cautions. The same literature carries cautions that transfer directly:

- attacks differ from typical learning tasks, with high error costs and a diverse normal class (Sommer & Paxson, 2010);
- security evaluations recur with sampling bias, base-rate neglect and lab-only testing (Arp et al., 2022).

For an agent monitor, these imply three requirements: a representative benign workload, interpretation of deviations in security terms, and models of "normal" learned from data resembling deployment. These are foundational and methodological sources, not included studies.

Themes 5 and 6 (behavioral/trajectory evidence; provenance and goal–action consistency): synthesis.

- **Established.** Many runtime systems observe trajectories: 17 of 42 code behavioral evidence and 21 code multi-step handling. Provenance and goal consistency are established signals against injected or misaligned actions.
- **Uncertain.** The measured contribution of each kind of evidence. One ablation found no effect of a session tracker, and another found a 26.4-point effect of cross-step labels. Both are single-study results on the authors' own benchmarks.
- **Under-evaluated.** The effect of trajectory and provenance evidence when it feeds a probability estimator rather than a rule or a judge. Evaluation in financial settings.
- **Differs across studies.** How provenance is obtained (analysis, construction or labels) and how sequences are modeled (automata, Markov chains, latent states or heuristics).
- **Implication for REM.** REM cannot claim trajectory monitoring or provenance analysis as new. Its evaluation must report the contribution of behavioral and provenance evidence, and must not assume it. It must also show that the benchmark contains the behavior the features are meant to detect.

2.8 Statistical and Algorithmic Foundations Relevant to REM

A runtime decision layer that assigns graduated responses must solve four methodological problems:

- turn evidence into an estimate of how likely an action is to be adversarially induced;
- make that estimate interpretable as a probability;
- combine the probability with the consequence of the action to reach a decision;
- record why the decision was made.

A layer that observes several steps must also decide how to accumulate evidence over time. This section explains **why** each established method is relevant and what the included studies show about its use in agent security. The methods are established, and none is a contribution of this thesis. Their precise form in REM, including the estimator, the calibration map, the loss structure and the verdict rule, is specified in Chapter 3 and is not repeated here. The sources in this section are **FOUNDATIONAL** or **METHODOLOGICAL SOURCES** admitted under criterion I6 (Section 2.2.4).

2.8.1 Why logistic regression

Logistic regression models the probability of a binary outcome as a logistic function of a linear score (Cox, 1958):

$$\Pr(y = 1 \mid x) = \frac{1}{1 + \exp\!\big(-(\beta_0 + \beta^\top x)\big)} \qquad (2.1)$$

Why it suits a runtime layer.

- Evaluation costs one inner product.
- Coefficients can be inspected.
- The output is intended as a probability, not as an arbitrary score.

Limitations.

- Interactions must be engineered into the features.
- A model fitted on one attack distribution can be miscalibrated on another.

- With scarce adversarial examples and binary indicators, the unpenalized estimate may not be finite. A ridge penalty addresses this (le Cessie & van Houwelingen, 1992).
- Naive Bayes is the generative counterpart with the same linear form. It may reach its (higher) asymptotic error with fewer examples (Ng & Jordan, 2002). It is an established alternative estimator and is not part of REM.

Use in agent security. Among the included studies, only NEXUS uses a logistic risk score for agent safety (Hossain et al., 2026). The choice of estimator family therefore has one direct precedent in agent security, and that precedent operates at the plan level.

2.8.2 Why calibration matters, and its limits

Definition. A classifier is *calibrated* when its predicted probabilities match observed frequencies. Calibration matters whenever a probability is multiplied by a cost, because miscalibration distorts the comparison between actions.

Platt scaling. Platt (1999) fits a sigmoid to a model's score *s* on held-out data:

$$\hat{p} = \frac{1}{1 + \exp(A s + B)} \qquad (2.2)$$

The mapping is monotone, so it preserves the ranking of predictions.

Alternatives.

- **Temperature scaling** is a one-parameter variant (Guo et al., 2017).
- **Isotonic regression** is nonparametric (Zadrozny & Elkan, 2002). Platt scaling tends to perform better with small calibration sets, whereas isotonic regression performs as well or better once calibration data are plentiful (Niculescu-Mizil & Caruana, 2005).
- **Beta calibration** contains the identity map on scores in [0, 1], which the logistic family applied to such scores does not (Kull et al., 2017).

In REM, Platt scaling on the logit is the adopted calibrator. Beta, isotonic and temperature calibration are reviewed alternatives and are **not adopted** (Chapter 3).

Measuring calibration. Calibration is measured by the expected calibration error (ECE; Naeini et al., 2015; Guo et al., 2017). ECE depends on the binning and on the size of the calibration set.

Evidence from included studies.

- **NEXUS** reports that Platt scaling reduced ECE from 0.085 to 0.013 on its benchmark. It also evaluated isotonic regression, and its authors report that Platt was less prone to overfitting on the small calibration split (Hossain et al., 2026; the calibration details are PARTIALLY VERIFIED).
- **C. Zhang et al. (2026)** caution against over-reading such improvements. In their ALFWorld analysis, Platt scaling reduced a confidence score's ECE from 0.463 to 0.006, while control regret under threshold routing stayed at 0.318 (preprint; figures PARTIALLY VERIFIED). A calibrated risk estimate is not the decision-relevant quantity for choosing an intervention.
- **LATTICE** adds independent peer-reviewed evidence in the same direction from a different setting. Across four planner families and 4,000 trajectories, a *confidence-threshold baseline* had a false-allow rate ranging from 0.03 to 0.998 (Calboreanu, 2026). A single threshold on a confidence signal was therefore unstable across the upstream models that produced the actions.

Implication. Calibration quality and control quality must be measured and reported separately. A probability, however well calibrated, does not by itself represent whether a trajectory remains recoverable.

2.8.3 Why sequential evidence matters

When evidence arrives step by step, a monitor must decide when the accumulated evidence justifies action.

CUSUM. Classical sequential analysis gives a statistically grounded answer. The cumulative-sum procedure (Page, 1954) accumulates log-likelihood ratios of post-change against pre-change behavior and raises an alarm when the accumulated statistic crosses a threshold (Basseville & Nikiforov, 1993). Under the classical change-point model, with independent observations before and after the change, it has an asymptotic optimality property for detection delay at a given false-alarm constraint (Lorden, 1971).

Limits for agents. These assumptions are only approximately satisfied by tool-call sequences, and an adversary can pace an attack to stay below the threshold. The procedure also detects a *change*, whereas an agent episode may be adversarial from its first injected observation.

Heuristic alternatives. Among the included studies, sequential accumulation is heuristic: PRISM's session risk with time-to-live decay (F. Li, 2026) and FinHarness's cascade (H. Jia et al., 2026).

Role in REM. CUSUM is **not part of REM's verdict path**. Chapter 3 uses it only as an optional, offline, observe-only analysis of logged indicators that never influences a verdict.

2.8.4 Why expected-loss decisions are relevant

The principle. Cost-sensitive decision theory chooses the response with the lowest expected cost given the class probabilities and a cost matrix (Elkan, 2001). For responses a and a binary state y:

$$a^*(x) = \arg\min_a \sum_{y \in \{0,1\}} \Pr(y \mid x)\, C(a,y) \qquad (2.3)$$

The reject option. Chow (1970) showed that withholding an automatic decision is optimal when the expected cost of deciding exceeds the cost of rejecting. This is the classical basis for routing uncertain cases to human review.

Requirements. The rule needs calibrated probabilities (Section 2.8.2) and costs that reflect real preferences, which are rarely known precisely in security. Elkan's principle and Chow's analysis support REM's expected-loss verdict and its Escalate route. Neither source proposed REM's four-action design.

Precedents among included studies.

- **NEXUS** defines an expected-loss objective with fixed intervention costs and uses it to set loss-optimal thresholds inside a rule-first cascade (Hossain et al., 2026; the cost values and cascade are PARTIALLY VERIFIED).
- **H.-H. Chen (2026)** prices the consequence of each action deterministically against a reserve.
- **Kaptein et al. (2026)** formalize a path-conditioned probability of policy violation.

Cost-based reasoning about agent actions therefore has precedents.

2.8.5 What attribution contributes to auditability

Additive attribution. Additive attribution explains a prediction as a sum of per-feature contributions. For a linear model under feature independence, the Shapley attribution of a feature is its coefficient times the feature's deviation from its mean (Lundberg & Lee, 2017). On the log-odds of a logistic model, that decomposition is exact and costs no more than the score itself. This makes it suitable for an audit record that shows which evidence contributed to a decision.

Other audit traces. It is not a distinguishing mechanism. Included systems already provide audit traces in other forms:

- rules and findings (NEXUS; AgentTrust);
- verifiable rule circuits (ShieldAgent);
- traceable evidence (ProvenanceGuard);
- tamper-evident or hash-chained logs (PRISM; the MCP policy-enforcement point; LATTICE's cryptographic audit trail).

In REM, attribution is **audit-only** and never enters the verdict.

Table 2.6. Established methods reviewed as the algorithmic basis for REM

Problem	Method (original source)	Why relevant	Main limitation	Use among included studies
Probabilistic risk scoring	Logistic regression (Cox, 1958); ridge penalty (le Cessie & van Houwelingen, 1992); naive Bayes as alternative (Ng & Jordan, 2002)	Fast, inspectable, probabilistic output	Linear log-odds; distribution shift	NEXUS (plan level)
Calibration	Platt (1999); temperature (Guo et al., 2017); isotonic (Zadrozny & Elkan, 2002); beta (Kull et al., 2017)	Makes scores usable with costs	Calibration does not imply good control	NEXUS (Platt, isotonic evaluated); C. Zhang et al. (analysis); DreamGuard (calibrated fusion)
Calibration measurement	ECE (Naeini et al., 2015; Guo et al., 2017)	Quantifies miscalibration	Bin-dependent; unstable with small sets	NEXUS; C. Zhang et al.
Sequential accumulation	CUSUM (Page, 1954; Basseville & Nikiforov, 1993); optimality (Lorden, 1971)	Statistically grounded accumulation	Assumptions not met by agent trajectories; evadable by pacing	Heuristic analogues only (PRISM; FinHarness). In REM: optional, observe-only
Response selection	Minimum expected cost (Elkan, 2001); reject option (Chow, 1970)	Uses consequences explicitly; human-review region	Costs uncertain; recoverability omitted	NEXUS objective (fixed costs, cascade); consequence pricing (H.-H. Chen)
Audit decomposition	Additive attribution (Lundberg & Lee, 2017)	Exact for linear scores; cheap	Not a novel explanation mechanism	Rule/evidence/audit traces in NEXUS, AgentTrust, ShieldAgent, ProvenanceGuard, PRISM, LATTICE

Theme 7 (statistical and algorithmic methods): synthesis.

- **Established.** Each building block is established, and several are already applied in agent security.
- **Uncertain.** How these blocks perform in combination at the action boundary of an executing agent under indirect injection.
- **Under-evaluated.** Calibration of agent-safety probabilities. Only NEXUS reports calibration error for a deployed scorer, and C. Zhang et al. and LATTICE show that calibration or a threshold on confidence does not by itself secure good control.
- **Differs across studies.** Probabilities are produced by logistic scores, learned Markov chains, latent world models, or LLM judges.
- **Implication for REM.** REM's estimator, calibrator and decision rule are adopted, not contributed. Calibration must be reported separately from control. The comparison that isolates the effect of using consequences is against a consequence-independent rule that uses the *same* estimator.

2.9 Runtime Decision and Mitigation

This section examines how runtime systems *act* on what they detect. It distinguishes four functions that the literature often conflates:

- **detection:** recognizing that something may be wrong;
- **risk estimation:** quantifying how likely or how severe;
- **decision:** selecting a response;
- **mitigation:** executing the response.

2.9.1 The space of runtime responses

The included systems use a common, graduated vocabulary:

- allow;
- block;
- modify: rewrite arguments, sanitize context, propose a safer alternative or request a revision;
- escalate: request confirmation or human review;
- restrict tools or privileges;
- interrupt or terminate;
- stage, undo or roll back effects.

Human participation is also studied directly. An analysis of 21 production agent systems found runtime approval in 15 of them, policy specification in 14 and scope configuration in 16. It found no production deployment of intent anchoring or trust labeling, the mechanisms most studied academically. It also identified a trade-off between approval fatigue and uncontrolled autonomy (P. Wang et al., 2026; preprint).

Table 2.7. Runtime responses reported by included systems

System	Allow	Block	Modify / sanitize / revise	Escalate / human review	Restrict tools or privileges	Interrupt / terminate	Stage / roll back / undo	Decision granularity
Progent	Yes	Yes	Not Reported	Yes (approval for privilege expansion)	Yes	Not Reported	Not Reported	Per tool call
AgentSpec	Yes	Yes	Rule-defined	Not Verified	Not Reported	Not Verified	Not Reported	Per triggered event
RTBAS	Yes (auto-execute)	Not Verified	Not Reported	Yes (user confirmation)	Not Reported	Not Reported	Not Reported	Per tool call
MCP policy-enforcement point	Yes	Yes (deny)	Not Reported	Not Reported	Yes (capability tokens)	Not Reported	Not Reported	Per tool call with cross-step labels
LATTICE	Yes	Yes	Not Reported	Yes (confidence-based escalation to humans)	Not Reported	Not Reported	Not Reported	Per gated action
VIGIL	Yes (commit)	Yes (no commit)	Not Verified	Not Reported	Not Reported	Not Reported	Not Reported	Per speculative action
ToolSafe	Yes	Not Verified	Yes (guardrail feedback to agent)	Not Reported	Not Reported	Not Reported	Not Reported	Per tool invocation
MI9	Yes	Yes	Not Verified	Not Verified	Yes (graduated containment)	Yes	Not Reported	Continuous, agent-level
PRISM	Yes	Yes	Not Verified	Not Verified	Yes (policy controls)	Not Verified	Not Reported	Per hook, session accumulation

System	Allow	Block	Modify / sanitize / revise	Escalate / human review	Restrict tools or privileges	Interrupt / terminate	Stage / roll back / undo	Decision granularity
SafeAgent	Yes	Yes	Yes (sanitization, replanning, argument rewriting)	Yes (human escalation)	Not Verified	Yes	Yes (rollback)	Per step over session state (PARTIALLY VERIFIED)
AgentTrust	Yes	Yes	Yes (safer alternatives)	Yes (warn; review)	Not Reported	Not Reported	Not Reported	Per tool call, session chains
NEXUS	Yes	Yes	Yes (request revision)	Yes (request confirmation)	Not Reported	Not Reported	Not Reported	Per plan, before execution
DreamGuard	Yes (PASS)	Yes (BLOCK)	Not Reported	HOLD (semantics Not Verified)	Not Reported	Not Reported	Not Reported	Per action with trajectory state
FinHarness	Yes (approve)	Yes	Not Reported	Escalation to an advanced LLM judge (not a human)	Not Reported	Not Reported	Not Reported	Per step, cross-turn
Jackson	Yes	Yes (deny)	Yes (sanitize)	Yes (escalate)	Not Verified	Not Verified	Not Verified	Not Verified
Cordon	Not Applicable	Not Applicable	Not Applicable	Not Verified	Not Applicable	Not Applicable	Yes (shadow state; effect outbox)	Per task transaction
GoEX	Not Applicable	Not Applicable	Not Applicable	Post-facto human validation	Not Reported	Not Reported	Yes (undo; damage confinement)	Per executed action

2.9.2 AgentTrust: runtime interception with graduated verdicts

AgentTrust (C. Yang, 2026; preprint; MODERATE EVIDENCE; PARTIALLY VERIFIED) is among the closest prior art to REM. It passed the inclusion criteria through the other-methods stream. The protocol asks that its runtime interception, verdicts, monitoring, multi-step handling, mitigation, evaluation, latency and ablation claims each be verified; the verification status of each is given below.

Architecture and verdicts (verified at abstract level). AgentTrust intercepts agent tool calls *before execution* and returns one of four verdicts: allow, warn, block or review. Its components are:

- a normalizer that deobfuscates shell input;
- an analyzer with 42 risk patterns;
- a policy engine with 170 rules;
- a SafeFix engine that proposes safer alternatives;
- a session tracker (RiskChain) with seven order-aware chain detectors;
- an LLM judge for ambiguous cases.

Results (abstract level). On its 300-scenario internal benchmark, a production-only rule set reached 95.0% verdict accuracy and 73.7% risk-level accuracy. On 630 external adversarial scenarios, the author reports 96.7% under a *patched* rule set, explicitly not zero-shot.

Partially verified details. The following could be confirmed only through search-engine excerpts:

- an end-to-end latency of about 1.72 ms against about 1.35 s for a zero-shot LLM judge;
- worst-case aggregation over the normalized variants of an input;
- the judge’s five dimensions, one of which is reversibility;
- the ablation in which disabling the session tracker left accuracy unchanged.

What REM cannot claim. AgentTrust already provides:

- pre-execution interception;
- a four-way graduated verdict set that includes human review;
- a modify-like response (SafeFix);
- session-level multi-step detection;
- component ablations with latency.

Where the difference lies. The defensible differences are narrower:

- **Evidence to verdict.** AgentTrust’s verdicts come from analyzers and rules, with an LLM judge for ambiguous cases. The verified material reports no calibrated probability of adversarial induction.
- **Use of consequence.** Reversibility is one of the judge’s dimensions, not an explicit loss that changes the decision at a fixed probability.

- **Threat and domain.** AgentTrust targets side effects of coding and shell agents. It does not report evaluation on indirect injection against a financial agent.

2.9.3 NEXUS: calibrated risk scoring with a four-way intervention set

NEXUS (Hossain et al., 2026; preprint; MODERATE EVIDENCE; PARTIALLY VERIFIED) is the closest prior art to REM's *decision* component. It also passed the inclusion criteria through the other-methods stream. The protocol asks that its logistic risk scoring, calibration, consequence features, expected-loss objective, decision policy, intervention cascade and evaluation each be verified; the verification status of each is given below.

Architecture (verified at abstract level). NEXUS is a structured-plan safety monitor. It evaluates the plan an agent proposes *before execution* and selects among allow, block, request confirmation and request revision. It combines four components:

- deterministic rules;
- argument-level inspection;
- a logistic-regression risk score for graded escalation;
- a session manager for cross-turn state.

Results (abstract level). On a 128-instance held-out synthetic benchmark it reports:

- F1 of 0.949;
- four-class intervention accuracy of 0.6406, 27.3 percentage points above rule-only selection;
- median latency of 0.205 ms.

Partially verified details. The following could be confirmed only through search-engine excerpts:

- **Calibration.** Platt scaling reduced ECE from 0.085 to 0.013; isotonic regression was also evaluated.
- **Expected-loss objective.** The objective uses *fixed* intervention costs (0 allow, 0.1 revise, 0.3 confirm, 1 block) to set loss-optimal thresholds on the calibrated score.
- **Deployed policy.** The policy is a rule-first cascade. Multiple critical violations always block. With one critical violation the score decides between block and confirm. Otherwise, threshold crossings route plans to confirmation. With no rule firing, the policy has no signal for revision.
- **Consequence features.** Each planned tool call carries side-effect, irreversibility, sensitivity, permission and cost annotations, which the rules use.
- **Evaluation caveats.** The authors call the in-distribution results on their templates upper bounds. They report out-of-distribution F1 of 0.861 on R-Judge and 0.881 on a stress set.

What REM cannot claim. NEXUS already provides:

- calibrated logistic risk scoring;
- a four-way intervention set with confirmation and revision routes;
- consequence information in its plan representation and rules;
- an expected-loss formulation with loss-optimal thresholds;
- sub-millisecond latency.

Where the difference lies. The defensible differences are:

- **Point of evaluation.** REM evaluates each proposed tool call during execution, not a pre-execution plan.
- **Threat model and setting.** REM addresses indirect injection in an executable banking environment, not synthetic plan templates.
- **Decision rule.** REM minimizes conditional expected loss per step, with losses indexed by each action's declared consequence tier. NEXUS uses fixed intervention costs to set thresholds inside a rule-first cascade.

Whether these differences change outcomes is an empirical question.

2.9.4 Other decision mechanisms

Jackson (2025). A working paper (LIMITED EVIDENCE; PARTIALLY VERIFIED) describes a policy engine in which a multi-dimensional risk score drives a deterministic enforcement state machine with allow, deny, sanitize and escalate decisions. Its decision set corresponds closely to REM's four verdicts. It is unrefereed, its full text was not accessed, and its evaluation claims are qualitative.

Consequence-aware control.

- H.-H. Chen (2026; preprint) prices each side-effect-bearing action, such as a payment or refund, against a safe default and gates it against a reserve capital budget. It does not estimate how likely the action is to have been induced.
- SafeAgent realizes consequence modeling and arbitration through LLM reasoning (H. Liu et al., 2026; PARTIALLY VERIFIED).

- NEXUS encodes consequence in plan annotations and rules.

Confidence-based escalation. LATTICE separates deciding from judging, so that no component both proposes and approves an action. It escalates to human operators on confidence, and reports zero unsafe actions across four planner families (Calboreanu, 2026). It achieved this at a conservative operating point that auto-allowed no action. In the same study, a confidence-threshold baseline's false-allow rate varied from 0.03 to 0.998 across planners.

Transactional and reversible mitigation.

- Cordon stages outward-facing effects in an outbox and executes reversible mutations in shadow state (Z. Chen et al., 2026).
- GoEX offers undo and damage confinement for post-facto validation (Patil et al., 2024).

Such substrates *consume* a decision and could supply the mechanics of a hold or undo response.

Themes 8 and 9 (decision and consequence handling; mitigation): synthesis.

- **Established.** Graduated responses including human review are common: Progent, RTBAS, LATTICE, AgentTrust, NEXUS, SafeAgent and Jackson. So are modify-type responses (AgentTrust, NEXUS, SafeAgent, ToolSafe) and reversible or staged effects (Cordon, GoEX). Consequence information enters decisions through rules (NEXUS), deterministic pricing (H.-H. Chen), a judge dimension (AgentTrust) or LLM operators (SafeAgent).
- **Uncertain.** How a probability of adversarial induction and a declared consequence should be *combined* when choosing a response. Only NEXUS combines a calibrated probability with costs, and it does so with fixed intervention costs inside a cascade at the plan level.
- **Under-evaluated.** How often each response is chosen, and how effective it is once chosen. Intervention rates are reported by few studies, notably ProvenanceGuard and the MCP policy-enforcement point. LATTICE's result shows that zero false-allow can be bought with an operating point that auto-allows nothing, so a safety figure must be read alongside intervention behavior.
- **Differs across studies.** The meaning of "escalate": to a human (RTBAS, LATTICE, AgentTrust) or to a stronger LLM judge (FinHarness). The meaning of "modify": argument rewriting, safer alternatives or plan revision. Whether decisions are per call, per plan or per episode.
- **Implication for REM.** REM's verdict set and its Escalate and Modify responses are adopted, not contributed. Its specific decision rule has one partial precedent in NEXUS. REM's evaluation must report intervention rates and benign utility beside attack success. The operational semantics of Modify and Block, and tie-breaking between verdicts of equal expected loss, are specified in Chapter 3. Two conflicts there remain open and are marked SUPERVISOR DECISION REQUIRED:
  - **SC-1:** episode-level versus step-level mitigation semantics;
  - **SC-2:** the tie-breaking order versus the prototype's VerdictTieError.

2.10 Financial AI-Agent Security

The protocol asks that general agent-security evidence be kept separate from financial-agent evidence. A general benchmark is not counted as financial evidence merely because one of its scenarios is financial.

2.10.1 General agent-security evidence with a financial component

Several included general studies contain a financial element but report no finance-specific analysis in the verified record:

- **AgentDojo** includes an executable Banking suite (Debenedetti et al., 2024).
- **ASB** includes finance as one of ten scenarios (H. Zhang et al., 2025).
- **AgentHarm** includes fraud among eleven harm categories (Andriushchenko et al., 2025).
- **ClawSafety** includes finance among five professional domains. Its scenarios include redirecting financial transactions (Wei et al., 2026).
- **ToolEmu** cites financial loss as a risk example (Ruan et al., 2024).
- **RTBAS** cites financial transactions as a motivating example (Zhong et al., 2025).

These studies establish that financial actions are within the attack surface that general benchmarks cover. They do not establish how defenses perform on financial actions specifically.

2.10.2 Financial-agent evidence

Four included studies have a primarily financial setting, and none reaches MODERATE or HIGH EVIDENCE:

- **Castro-Maldonado et al. (2026)**, peer reviewed, study a semantic firewall with online ensemble learning in an agentic retrieval-augmented banking chatbot. The record specifies neither the threat model nor the results (LIMITED EVIDENCE).

- **Z. Chen, J. Chen, et al. (2025)**, a preprint position paper, argue that accuracy and return metrics give an illusion of reliability for financial agents. They propose model-, workflow- and system-level stress testing, illustrated by an audit of six agents on three tasks (LIMITED EVIDENCE).
- **H.-H. Chen (2026)**, a preprint, frames consequence pricing actuarially, with payments and refunds as examples (LIMITED EVIDENCE; partial financial focus).
- **FinHarness** (H. Jia et al., 2026; preprint) is the closest domain-specific runtime system. It combines a query monitor fusing intent with cross-turn drift, a tool monitor for each prospective call, and a cascade routing verification between a lightweight and an advanced LLM judge. It reports attack success falling from 38.3% to 15.0%, with benign approval moving from 41.1% to 39.3%. All of these figures were measured on **FinVault**, whose arXiv version has been **withdrawn** by its authors (Z. Yang et al., 2026). FinVault is mentioned only to record that withdrawal. FinHarness's figures are therefore not treated as affirmative evidence, and FinHarness is appraised LIMITED EVIDENCE by override.

Background SoK. A systematization of LLM-agent security in agentic commerce (Mao et al., 2026) organizes financial threats along five dimensions and derives twelve cross-layer attack vectors. It places *transaction authorization* at the center of the financial threat model. It is a secondary study, cited as background.

2.10.3 Why financial execution raises the requirements

Three properties of financial execution raise the requirements on a runtime layer. The first two are established in the included sources; the third is this chapter's inference from them.

- **Irreversibility.** Some financial actions cannot be recalled. This is encoded in consequence pricing (H.-H. Chen), in irreversibility flags checked by rules (NEXUS), and in transactional staging (Cordon).
- **Attacker-supplied arguments.** Under indirect injection, recipients and amounts can be planted in the content the agent reads, which makes argument provenance especially informative (Section 2.7.2).
- **Heterogeneous consequences.** A balance query and a transfer may both be permitted, yet differ sharply in the harm an induced call could cause. The same probability of adversarial induction therefore need not warrant the same response.

Regulatory motivation. Regulation (EU) 2024/1689 requires that high-risk AI systems allow automatic event logging (Article 12) and effective human oversight, including the ability to intervene or halt (Article 14) (European Parliament & Council of the European Union, 2024). Whether a given financial agent is high-risk is not assessed here. The provisions motivate an audit record for every decision and an explicit route to human review.

2.10.4 Requirements derived for a runtime layer protecting a financial agent

Six requirements follow from Sections 2.4–2.10:

- **R1** pre-execution mediation of each proposed action;
- **R2** provenance evidence on action arguments;
- **R3** consequence-differentiated responses at the same estimated probability;
- **R4** a human-review route;
- **R5** an audit record for every decision;
- **R6** joint measurement of security, benign utility, intervention behavior, calibration and latency.

Theme 10 (financial-agent evidence): synthesis.

- **Established.** Financial actions are within the attack surface of general benchmarks, and a financial runtime harness has been built (FinHarness).
- **Uncertain.** How any defense performs on financial actions specifically. The only quantitative financial runtime results rest on a withdrawn benchmark.
- **Under-evaluated.** Every financial-context study is LIMITED EVIDENCE. No included study reports a finance-specific analysis of an open, executable benchmark.
- **Differs across studies.** The financial studies vary in kind: a chatbot firewall, a position paper, an actuarial framework, and an LLM-judge harness.
- **Implication for REM.** Evaluation on AgentDojo's Banking suite, with results reported at the level of financial action types and consequence tiers, would add verified financial evidence where the systematic search found none.

2.11 Comparative Evidence Synthesis

Two tables compare the included runtime defense and design studies. Both are built from the systematic evidence matrix (`CHAPTER2_SYSTEMATIC_EVIDENCE_MATRIX.csv`).

- **Table 2.8** records what each study observes and does, using the columns specified in the review protocol.
- **Table 2.9** records how the studies closest to REM turn evidence into a decision.

Attack, benchmark and analysis studies appear in Tables 2.3 and 2.4.

Coding. Yes = reported in the verified record; NR = Not Reported; NV = Not Verified; NA = Not Applicable. A property that the record does not report is **not** inferred to be present or absent. Financial context is coded Yes only for a primarily financial setting. Evidence levels are HIGH, MOD (moderate) and LIM (limited), as defined in Section 2.2.6.

Table 2.8. Comparative evidence for included defense and design studies (n = 42)

Study	Threat	Entry Point	Observation Unit	Runtime	Behavior	Tool/Action	Multi-Step	Provenance	Decision	Mit
C. L. Wang et al. (2025) [P59]	Emergent/unsafe agent behaviour (governance)	NA	Agent state and actions over time	Yes	Yes	Yes	Yes	NR	Yes	Yes (graduate
C. Yang (2026) [P64]	Unsafe tool-use side effects	Proposed tool call	Each proposed tool call (plus session chains)	Yes	Yes (seven chain detectors)	Yes	Yes (session tracker; no measurable effect in ablation)	NR	Yes	Yes (allow/warn/l safer alternat
Calboreanu (2026) [P40]	Unauthorised actions	NA	Proposed action at gated execution path	Yes	NR	Yes	NR	NR	Yes	Yes
Castro-Maldonado et al. (2026) [P32]	Customer-data protection / security (threat model not specified in abstract)	User queries and retrieved content	Query/response	Yes	NR	NV	NR	NR	Yes	NV
Chennabasappa et al. (2025) [P16]	Prompt injection; jailbreak; agent misalignment; insecure code	Untrusted inputs; agent reasoning; generated code	Inputs and chain-of-thought	Yes	Yes	NV	NV	NR	Yes	NV
Debenedetti et al. (2025) [P52]	Prompt injection; data exfiltration	Untrusted data	Program control and data flow	Yes	NR	Yes	Yes (data flow across steps)	Yes	Yes	Yes
E. Li et al. (2026) [P12]	Malicious apps; indirect prompt injection; planning/execution integrity and availability	Third-party apps and their outputs	Abstract plan, concrete plan and execution	Yes	NR	Yes	Yes	Yes	Yes	Yes
F. Jia et al. (2025) [P01]	Indirect prompt injection	External data returned by tools	Each instruction and tool call	Yes	NR	Yes	NV	NR	Yes	NV
F. Li (2026) [P60]	IPI, unsafe tools, credential leakage, tampering	Tool inputs/outputs; files; credentials	Lifecycle hook events within a session	Yes	Yes (session risk accumulation)	Yes	Yes	NR	Yes	Yes
F. Wu, Cecchetti, & Xiao (2024) [P08]	Indirect prompt injection	Any untrusted source feeding the planner	Planning inputs and structured plans	Yes	NR	Yes	Yes	Yes	Yes	Yes
H. Jia et al. (2026) [P71]	Prompt-induced unauthorised financial actions	User queries; tool calls	Query and tool call with cross-turn state	Yes	Yes (partial: cross-turn drift, accumulated risk)	Yes	Yes	NR	Yes	Yes (block/ap escalation to
H. Li et al. (2025) [P38]	Prompt injection via external tools	Tool outputs; memory stream	Function trajectory vs. planned trajectory	Yes	Yes	Yes	Yes	NR	Yes	Yes
H. Liu et al. (2026) [P63]	Prompt injection propagating through workflows	Inputs and intermediate context	Stateful workflow	Yes	Yes	Yes	Yes	NR	Yes	Yes

Study	Threat	Entry Point	Observation Unit	Runtime	Behavior	Tool/Action	Multi-Step	Provenance	Decision	Mit
H. Wang et al. (2025) [P61]	Safety-specification violations	NA	Abstract state sequence	Yes	Yes	Yes	Yes	NR	Yes	Yes
H. Wang, Poskitt, & Sun (2026) [P50]	Unsafe actions	NA	Triggered action vs. rule predicate	Yes	NR	Yes	NV	NR	Yes	Yes
H.-H. Chen (2026) [P67]	Consequences of side-effecting actions	NA	Each priced action	Yes	NR	Yes	NR	NR	Yes (consequence pricing vs. reserve)	Yes (gating)
Hines et al. (2024) [P47]	Indirect prompt injection	Untrusted documents	Input text	Yes	NA	NA	NA	Yes	NA	NA
Hossain et al. (2026) [P65]	Unsafe plans	Proposed plan and arguments	Plan (with cross-turn session state)	Yes	Yes (partial: session manager)	Yes (argument-level)	Yes (partial: plan-level)	NR	Yes	Yes (allow/block/
Hua et al. (2024) [P55]	Unsafe planning	NA	Plan	Yes (planning stage)	NR	Yes (plan-level)	Yes (plan-level)	NR	NV	Yes (plan rev
Jackson (2025) [P66]	Governance violations	NA	Governed action	Yes	NV	Yes	NV	NR	Yes (multi-dimensional risk score)	Yes (allow/deny/s
Jacob et al. (2025) [P23]	Prompt injection	Conversational and application-structured inputs	Input text	Yes	NR	NR	NR	NR	Yes	NR
Jiang et al. (2025) [P56]	Unsafe intermediate thoughts	NA	Intermediate thought before action	Yes	Yes (thought-level)	NR	Yes (partial)	NR	NV	Yes (thought
J. Lin et al. (2026) [P11]	Tool-stream injection (manipulated tool metadata and runtime feedback)	Tool metadata and tool feedback	Speculative action before commit	Yes	NR	Yes	NV	NR	Yes	Yes
W. Lin et al. (2026) [P62]	Long-horizon hazardous drift	NA	Each call with trajectory prefix	Yes	Yes	Yes	Yes	NR	Yes	Yes (PASS/H
Luo et al. (2025) [P39]	Task-specific and systemic risks	NV	Agent actions	Yes	NV	Yes	NR	NR	Yes	NV
Mou et al. (2026) [P33]	Unsafe tool invocations incl. prompt injection	Requests and tool interaction history	Each tool invocation with history	Yes	Yes	Yes	Yes	NR	Yes	Yes
Patil et al. (2024) [P10]	Incorrect or unsafe LLM-generated actions (non-adversarial)	NA	Executed action	Yes	NR	Yes	NR	NR	NV	Yes
S. Chen et al. (2025) [P48]	Prompt injection	Data channel	Model input	NA	NA	NA	NA	Yes	NA	NA
S. Wang, Zhu, & Li (2026) [P37]	Indirect prompt injection (path traversal, exfiltration, high-frequency tool abuse)	Externally retrieved data	Each tool call with cross-step labels	Yes	NR	Yes	Yes	Yes	Yes	Yes
She et al. (2026) [P58]	Misaligned tool calls incl. IPI	Tool outputs; context	Each tool call and its evidence provenance	Yes	Yes (partial)	Yes	NR	Yes	Yes	Yes
T. Shi et al. (2025) [P49]	Indirect prompt injection; tool misuse	Tool outputs	Each tool call vs. privilege policy	Yes	NR	Yes	Yes (policy updates during task)	NR	Yes	Yes

Study	Threat	Entry Point	Observation Unit	Runtime	Behavior	Tool/Action	Multi-Step	Provenance	Decision	Mit
Storf et al. (2026) [P34]	Scheming (covert pursuit of misaligned goals)	NA	Externally observable trajectory	Yes	Yes	Yes	Yes	NR	Yes	NR
Tsai & Bagdasarian (2025) [P51]	Context-inappropriate actions	NA	Proposed action vs. task-context policy	Yes	NR	Yes	NR	NR	Yes	Yes
Wen et al. (2025) [P03]	Indirect prompt injection	Retrieved external content	Model hidden states and gradients for an input	Yes	NR	NR	NR	NR	Yes	NR
Xiang et al. (2025) [P53]	Violations of safety guard requests	NA	Target agent actions	Yes	NR	Yes	NR	NR	Yes	Yes
Y. Chen et al. (2025) [P24]	Prompt injection in retrieved data	Retrieved content	Input prompt	Yes	NR	NR	NR	NR	NR	NR
Y. Wu et al. (2025) [P36]	Untrusted apps; cross-app security, privacy and safety issues	Third-party apps	App interactions	Yes	NR	Yes	NR	NR	NV	Yes
Z. Chen et al. (2026) [P68]	Irreversible effects; cross-step violations	NA	Task-level transaction	Yes	NR	Yes	Yes	NR	NR	Yes (staging, recovery)
Z. Chen, Kang, & Li (2025) [P54]	Policy violations over trajectories	NA	Action trajectory	Yes	Yes (trajectory-level)	Yes	Yes	NR	Yes	Yes
Zhong et al. (2025) [P06]	Prompt injection; privacy leakage	Tool outputs / external content	Each tool call and its data dependencies	Yes	NR	Yes	NV	Yes	Yes	Yes
Zhu, Yang, et al. (2025) [P57]	Indirect prompt injection	Tool outputs	Tool calls under original vs. masked re-execution	Yes	Yes	Yes	Yes	NR	Yes	NV
Zhu, Zhang, et al. (2025) [P18]	High-risk or unethical tasks; adversarial attacks	Task prompts; tool use; agent collaboration	Prompts, agent messages and tool use	Yes	Yes	Yes	NV	NR	Yes	Yes

Table 2.9. Decision mechanism, calibration, consequence and reported evaluation dimensions (runtime systems closest to REM)

System	Decision basis	Calibrated probability	Consequence used in decision	Human review / graduated response	Benign utility or false interventions reported	Latency reported	Financial focus	Evidence level
Progent	Privilege policy (solver-checked updates)	NA	NR	Approval for privilege expansion	Yes (functionality maintained)	NR	NR	MOD
AgentSpec	Deterministic rules	NA	NR	NV	NV	Yes (ms)	NR	MOD
RTBAS	Information-flow screeners	NR	NR	User confirmation when integrity/confidentiality not ensured	Yes (2% utility loss under attack)	NR	NR	MOD
MCP policy-enforcement point	Declarative rules over cross-step labels	NA	NR	NR	Yes (task-level FP 30.0%; 0/30 rule FPs)	Yes (sub-ms)	NR	HIGH
LATTICE	Policy-as-code; deterministic verdicts; confidence-based escalation	NR (confidence baseline analysed)	NR	Yes (human escalation)	Yes (operating point auto-allowed no action)	Yes (µs–ms)	NR	HIGH
Task Shield	LLM verification of task alignment	NR	NR	NR	Yes (69.79% utility)	NR	NR	HIGH

System	Decision basis	Calibrated probability	Consequence used in decision	Human review / graduated response	Benign utility or false interventions reported	Latency reported	Financial focus	Evidence level
VIGIL	Intent-grounded verification before commit	NR	NR	NR	Yes (utility under attack)	NR	NR	HIGH (PV)
ToolSafe	Learned step-level guardrail with feedback	NR	NR	Feedback to agent	Yes (benign completion)	NR	NR	MOD
ProvenanceGuard	Evidence support for tool calls	NR	NR	Intervention	Yes (aligned-trace intervention rate; PV)	Yes (higher than baselines; PV)	NR	MOD (PV)
MI9	Governance components; agency-risk index	NR	NR	Graduated containment	NV	Overhead noted (PV)	NV	MOD (PV)
ProbGuard	Probability of remaining safe (learned DTMC) vs threshold	NR (PAC-style bounds on learned model)	NR	NR	Yes (task completion)	NV	NR	MOD
DreamGuard	Calibrated fusion of hazard and prefix-risk scores	Partial (“calibrated fusion”; no calibration metric verified)	NR	HOLD	NV	Yes (~25 ms)	NR	MOD
SafeAgent	LLM operators; policy arbitration	NV	Yes (LLM consequence modelling; PV)	Yes	NV	NV	NR	LIM (PV)
AgentTrust	Rules and analysers; worst-case over normalized variants; LLM judge for ambiguity	NR	Partial (reversibility as a judge dimension; PV)	Yes (warn; review)	NR	Yes (~1.72 ms; PV)	NR	MOD (PV)
NEXUS	Rule-first cascade; calibrated logistic score with loss-optimal thresholds	Yes (Platt; ECE $0.085 \rightarrow 0.013$; PV)	Yes (plan annotations used by rules); intervention costs fixed (PV)	Yes (confirm; revise)	Yes (paired benign cases)	Yes (0.205 ms median)	NR	MOD (PV)
FinHarness	Monitors and LLM-judge cascade	NR	NR	Escalation to an advanced LLM judge	Yes (benign approval; withdrawn benchmark)	NR (judge-call counts)	Yes	LIM
H.-H. Chen	Deterministic pricing against reserve capital	NA	Yes	NR	NR	NR	Partial	LIM
Jackson	Multi-dimensional risk score; enforcement state machine	NV	NV	Yes (escalate; sanitize)	NV	Qualitative only	NR	LIM (PV)
C. Zhang et al. (analysis)	Action-conditioned intervention value	Analysed (calibration did not reduce regret; PV)	Yes (recoverability)	NA	Regret reported	NR	NR	MOD (PV)
REM (investigated in this thesis; not a result)	Calibrated per-step probability with consequence-tier losses in an expected-loss rule (Chapter 3)	Designed	Designed (declared tiers)	Designed (Escalate; Modify)	Planned	Planned	Planned (AgentDojo Banking)	—

PV = figure or property PARTIALLY VERIFIED (search-engine excerpt only).

Reading the tables.

Row by row, every individual property in REM's design appears somewhere in the included studies:

- calibrated logistic scoring with a four-way intervention set: NEXUS;
- pre-execution interception with graduated verdicts and human review: AgentTrust, RTBAS, LATTICE;
- consequence-dependent control: H.-H. Chen, NEXUS, SafeAgent;
- provenance and cross-step information-flow evidence: ProvenanceGuard, CaMeL, ACE, RTBAS, the MCP policy-enforcement point;
- trajectory-aware monitoring: MI9, PRISM, ProbGuard, DreamGuard, ToolSafe, AgentTrust;
- latency and component ablation: AgentTrust, NEXUS, the MCP policy-enforcement point;
- a financial runtime harness: FinHarness;
- the limits of calibration and confidence thresholds for control: C. Zhang et al.; LATTICE.

Column by column, the combinations thin out:

- **NEXUS** alone reports a calibrated probability together with costs and a graduated response. It does so at the plan level, on author-generated templates, with fixed intervention costs setting thresholds inside a rule-first cascade.
- **Provenance evidence**, where its contribution has been measured, feeds rules (the MCP policy-enforcement point) or a detector (ProvenanceGuard), not a probability estimator.
- **Financial focus** appears only in LIMITED EVIDENCE studies.
- **No included study** reports all three of the following together:
  - a calibrated probability of adversarial induction per proposed tool call;
  - consequence-dependent losses used to select among graduated responses;
  - evaluation on an executable financial environment with utility, intervention behavior and latency reported.

2.12 Critical Synthesis

This section draws the thematic syntheses together (Table 2.10) and then states what the systematic evidence does and does not support.

Table 2.10. Thematic synthesis of the included evidence

Theme	What the literature establishes	What is uncertain	What is under-evaluated	What differs across studies	Implication for REM
1 Threats	Indirect injection is demonstrated, benchmarked and adaptive; defenses of the time were bypassed	Magnitude: 24% to 84.30% average ASR across benchmarks	Tool-metadata, tool-library, skill and MCP-server threats against runtime defenses	Threat actor and success criterion	Focus on indirect injection; state other classes as out of scope
2 Entry points	The observation channel (tool outputs, retrieved content, tool metadata) is the decisive entry for tool-using agents	Relative danger of channels (skills > email/web in one study)	Defenses against setup-time channels	Which channel each benchmark exercises	Mediate at the action, where all channels converge
3 Observation granularity	Input, call, plan, data-flow and trajectory units all implemented	Which unit gives the best security–utility trade-off	Head-to-head comparisons on a shared workload	Unit observed; question asked	Per-call observation with provenance context is a defensible, not novel, choice
4 Runtime intervention	Interception at the tool boundary is widespread (41/42 runtime; 34/42 tool/action) and fast	Robustness to adaptive attacks	Evaluation on benchmarks the authors did not build	Permission vs. alignment vs. provenance vs. harm	REM cannot claim interception
5 Behavioral / trajectory evidence	Many monitors model sequences (17/42 behavioral; 21/42 multi-step)	Contribution: one null ablation (session tracker)	Contribution inside a probabilistic estimator; financial settings	Automata, Markov chains, latent states, heuristics	Measure and report the contribution; do not assume it
6 Provenance / goal–action consistency	Informative against injected or misaligned calls; cross-step labels had a 26.4-point effect in one study	Whether the effect generalizes beyond the authors' datasets	Provenance as input to a calibrated estimator; financial arguments	Obtained by analysis, construction or labels	Use as estimator input; test the contribution

Theme	What the literature establishes	What is uncertain	What is under-evaluated	What differs across studies	Implication for REM
7 Statistical / algorithmic methods	Logistic scoring, calibration, expected cost, reject option and attribution are established; one agent-safety precedent (NEXUS)	Behaviour in combination at the action boundary	Calibration of agent-safety probabilities; calibration vs. control	How probabilities are produced	Adopted methods; report calibration separately from control
8 Decision and consequence	Graduated responses and consequence information are common	How to combine an induction probability with a declared consequence	Intervention rates; per-response effectiveness	Fixed costs, pricing, rules, judge dimensions	Test the combination against a same-estimator consequence-independent baseline
9 Mitigation	Block, modify, escalate, restrict, roll back and undo all implemented	Effectiveness of each response once chosen	Residual risk after modify-type responses	Meaning of “escalate” (human vs. stronger judge) and “modify”	Report residual attack success and escalation rate; SC-1 and SC-2 remain open
10 Financial evidence	Financial actions lie within general benchmarks’ attack surface; one financial harness exists	Defense performance on financial actions	Every financial-context study is LIMITED EVIDENCE; key figures rest on a withdrawn benchmark	Chatbot firewall, position paper, actuarial framework, judge harness	Report results per financial action type and consequence tier
11 Evaluation quality	Results are benchmark-specific; many benchmarks are built with the defense they evaluate	Multi-step content of benchmarks	Independent evaluation; decomposition into security, utility, intervention, calibration and latency	Metrics and baselines	Per-benchmark reporting with benign utility, intervention rates, calibration and latency

What is well addressed.

- Threats are well characterized and benchmarked.
- Interception at the tool-call boundary is implemented in many forms, and deterministic enforcement is fast.
- Graduated responses, including human review, are standard.
- Architectural defenses offer formal guarantees for the threats they formalize.
- Calibration methods are mature and have one agent-safety application.

What remains fragmented. The pieces of a probabilistic, consequence-aware decision exist, but in different studies:

- a calibrated score with fixed-cost thresholds in a plan-level cascade (NEXUS);
- deterministic consequence pricing that ignores how an action arose (H.-H. Chen);
- consequence reasoning realized by LLM operators (SafeAgent);
- provenance evidence feeding rules or a detector (the MCP policy-enforcement point; ProvenanceGuard);
- financial focus in a judge cascade evaluated on a withdrawn benchmark (FinHarness).

Conflicting evidence, presented rather than resolved.

- **Value of trajectory evidence.** The AgentTrust ablation found no effect of a session tracker. The MCP policy-enforcement point found a large effect of cross-step labels. The kind of trajectory evidence differs, and both are single studies.
- **Architectural guarantees.** IsolateGPT reports protection against the attacks it tested, and ACE later defeats it with new attacks.
- **Calibration as a remedy.** NEXUS reports a large ECE improvement. C. Zhang et al. report that a similar improvement left control regret unchanged. LATTICE reports that a confidence threshold’s false-allow rate ranged from 0.03 to 0.998 across planners.
- **Safety figures versus utility.** LATTICE’s zero false-allow was obtained at an operating point that auto-allowed no action. The MCP policy-enforcement point’s attack-success reduction came with a 30.0% task-level false-positive rate. A safety figure alone is therefore not evidence of a usable defense.

Where current approaches stop.

- Most detectors and guardrails stop at a verdict and do not produce a probability that could be traded off against consequence.
- Most consequence-aware designs stop at the consequence and do not estimate the likelihood of adversarial induction.
- Most trajectory-aware systems stop at detection. The two ablations disagree about whether the trajectory component changes outcomes.
- Most evaluations stop at attack success: benign utility, intervention behavior, calibration and latency are reported unevenly.

What the evidence does not support. The systematic evidence does **not** support three claims that earlier drafts could have been read as making:

- that runtime interception, graduated verdicts or consequence awareness are absent from the literature;

- that combining several signals at the action boundary is rare. AgentTrust, the MCP policy-enforcement point, ToolSafe, DRIFT and FinHarness all combine signals at or near the tool call;
- that trajectory evidence is known to be ineffective. The evidence is mixed.

2.13 Research Gap and REM Positioning

2.13.1 Assessment of candidate gaps

The protocol listed five candidate gaps, A to E, to be retained only if the systematic evidence supports them. Table 2.11 records the assessment.

Table 2.11. Evidence-based assessment of candidate research gaps

Candidate gap	Evidence for	Evidence against	Decision
A. Integration. Capabilities exist across separate approaches	Tables 2.8–2.9: calibrated scoring (NEXUS), consequence pricing (H.-H. Chen), provenance (ProvenanceGuard; MCP policy-enforcement point), graduated verdicts (AgentTrust; LATTICE) are in different studies	NEXUS combines calibration, costs and graduated responses, at the plan level	**Retained, narrowed.** The integration not reported is a calibrated per-call probability of adversarial induction combined with consequence-indexed losses at the action boundary of an executing agent
B. Runtime action-boundary evaluation. Limited evidence combining multiple signals at the point of action	—	AgentTrust, the MCP policy-enforcement point, ToolSafe, DRIFT, SafeAgent and FinHarness combine several signals at or near the tool call	**Not retained as stated.** Its defensible residue, combination into a calibrated probability, is part of Gap A
C. Behavioral/provenance contribution. Insufficient evidence separating the contribution of behavioral and provenance signals	Only three component-level tests found; they conflict (null session-tracker ablation vs. a 26.4-point cross-step-label effect); none inside a probabilistic estimator; none financial	The MCP policy-enforcement point does separate the contribution of cross-step labels in a rule-based enforcer (HIGH EVIDENCE)	**Retained, narrowed.** Limited and conflicting evidence on the contribution of behavioral and provenance evidence *as inputs to a calibrated estimator* for financial actions under indirect injection
D. Financial-agent evidence. Limited verified evidence in executable financial settings	Four financial-context studies, all LIMITED EVIDENCE; the only quantitative runtime results rest on a withdrawn benchmark; general benchmarks with financial suites report no finance-specific analysis in the verified record	FinHarness shows the problem is recognized	**Retained**
E. Evaluation decomposition. Insufficient evaluation separating security, benign utility, calibration or decision quality, intervention behavior and latency	Calibration reported for one deployed scorer (NEXUS); intervention rates by few studies; no study compares a consequence-aware decision with a consequence-independent baseline that shares its estimator	Several studies report more than attack success (MCP policy-enforcement point; LATTICE; NEXUS; ToolSafe)	**Retained, narrowed.** The missing element is the same-estimator comparison, with calibration reported separately from control and intervention rates reported

2.13.2 Statement of the research gap

> Among the 71 studies included in this systematic review (search date 25 September 2026), none reports an empirical evaluation of a non-invasive runtime layer for a financial tool-using agent that combines (i) provenance-based evidence about proposed financial actions, (ii) a calibrated per-step probability that the proposed action is adversarially induced, (iii) a decision that minimizes expected loss over Allow, Modify, Escalate and Block using losses declared per consequence tier, and (iv) deterministic mitigation whose effectiveness is measured and used in the loss structure. None reports attack success, benign utility, utility under attack, intervention behavior and latency separately, with calibration reported separately from control outcomes, and results compared against a consequence-independent baseline that uses the same estimator.

Nature of the gap. The gap is **integrative and empirical**. It concerns the combination and evaluation of established mechanisms under indirect prompt injection, in a financial setting. It does not concern the absence of any single mechanism, and it does not concern a new learning algorithm.

Bounds. The gap is bounded by the search, which covered one database with a retrieval cap and a partly non-independent other-methods stream. It is also bounded by record-level appraisal (Section 2.13.4).

Falsifiability. A study that reports this combination and evaluation would require the contribution to be revised.

Consistency with Chapter 3. The statement is consistent with Chapter 3, §3.13, which describes the contribution as empirical and integrative.

2.13.3 REM positioning: established by prior research versus proposed by this thesis

REM is a runtime mediation layer between a financial tool-using agent and its environment. It is organized in exactly five layers:

1. **Input & Context**;
2. **Detection**;
3. **Behavioral Analysis**;
4. **Decision Engine**;
5. **Mitigation**.

Chapter 3 specifies it. Table 2.12 separates what REM adopts from prior research from what this thesis proposes.

Table 2.12. Established by prior research versus proposed by this thesis

Element	Established by prior research (included or foundational sources)	Role in REM
Runtime interception of each proposed tool call	Progent; AgentSpec; AgentTrust; RTBAS; MCP policy-enforcement point; LATTICE; PRISM; SafeAgent; Task Shield	Adopted, not claimed
Provenance / goal–action consistency evidence	ProvenanceGuard; Task Shield; MELON; CaMeL; ACE; RTBAS; MCP policy-enforcement point	Adopted as estimator input, not claimed
Behavioral / trajectory monitoring	MI9; ProbGuard; DreamGuard; PRISM; AgentTrust; ToolSafe; Forrest et al. (1996); Chandola et al. (2009, 2012)	Adopted, not claimed
Ridge logistic risk scoring	Cox (1958); le Cessie & van Houwelingen (1992); NEXUS	Adopted, not claimed
Platt calibration on the logit	Platt (1999); Guo et al. (2017); NEXUS	Adopted, not claimed. Beta, isotonic and temperature calibration are reviewed and **not adopted**
Minimum expected-cost decision; reject option	Elkan (2001); Chow (1970); NEXUS objective	Adopted, not claimed. Neither source proposed REM's four-action architecture
Consequence-dependent control	H.-H. Chen (2026); NEXUS; SafeAgent; AgentTrust	Adopted as declared consequence tiers, not claimed
Graduated responses including human review	AgentTrust; NEXUS; RTBAS; LATTICE; SafeAgent; Jackson; MI9	Adopted, not claimed
Deterministic mitigation; reversible and staged effects	Progent; AgentSpec; Cordon; GoEX	Adopted, not claimed
Sequential change detection (CUSUM)	Page (1954); Basseville & Nikiforov (1993); Lorden (1971)	Optional, offline, observe-only analysis outside the verdict path; not claimed
Additive attribution for audit	Lundberg & Lee (2017); audit traces in NEXUS, AgentTrust, PRISM, LATTICE	Audit record only; never used for the verdict; not claimed
Combination of (i)–(iv) in Section 2.13.2 at the action boundary, under indirect injection, on an open executable banking benchmark	Not reported among the 71 included studies	**Proposed by this thesis (integrative)**
Evaluation that separates attack success, benign utility, intervention behavior, calibration and latency, against a consequence-independent baseline with the same estimator	Not reported among the 71 included studies	**Proposed by this thesis (empirical)**

Relation to AgentTrust. AgentTrust already provides:

- pre-execution interception;
- four graduated verdicts including human review;
- safer-alternative suggestions;
- session-level chain detection;
- component ablation with latency.

REM claims none of these. REM differs in how evidence becomes a decision: a calibrated probability combined with declared losses, where AgentTrust uses rule and analyzer aggregation with an LLM judge. REM also differs in threat model and domain. Several AgentTrust details are PARTIALLY VERIFIED (Section 2.9.2).

Relation to NEXUS. NEXUS already provides:

- calibrated logistic risk scoring;
- a four-way intervention set;
- consequence annotations used by rules;
- an expected-loss objective with loss-optimal thresholds;
- sub-millisecond latency.

REM claims none of these. REM differs in three respects:

- **point of evaluation:** each proposed tool call during execution, where NEXUS evaluates a pre-execution plan;
- **setting:** indirect injection in an executable banking environment, where NEXUS uses synthetic templates;
- **decision rule:** per-step minimization of conditional expected loss with losses indexed by each action's consequence tier, where NEXUS uses fixed intervention costs to set thresholds inside a rule-first cascade.

The cost values and the cascade are PARTIALLY VERIFIED (Section 2.9.3). Whether the differences change outcomes is to be tested, not assumed.

Relation to other close studies. REM does not claim:

- to be the first financial runtime protection (FinHarness);
- a risk-score-to-graduated-verdict pipeline (Jackson; SafeAgent);
- deterministic consequence pricing (H.-H. Chen);
- provenance analysis (ProvenanceGuard; the MCP policy-enforcement point);
- that calibration improves control (C. Zhang et al.; LATTICE).

Open design conflicts. The operational semantics of Modify and Block, and the tie-breaking between verdicts of equal expected loss, are specified in Chapter 3. Two conflicts there remain unresolved and are carried forward unchanged:

- **SC-1 (mitigation semantics).** Chapter 3 specifies episode-level Modify and Block; the implementation uses step-level semantics. Status: **SUPERVISOR DECISION REQUIRED**.
- **SC-2 (tie-breaking).** Chapter 3 specifies Block ≥ Escalate ≥ Modify ≥ Allow; the prototype raises a VerdictTieError. Status: **SUPERVISOR DECISION REQUIRED**.

2.13.4 Limitations of this review

The review is systematic in procedure but **not exhaustive**, and its conclusions are bounded as follows.

1. **Database accessibility.** Only OpenAlex was searched. IEEE Xplore, the ACM DL, Scopus and Web of Science, ScienceDirect, SpringerLink, the ACL Anthology, NeurIPS/PMLR, ICLR/OpenReview, USENIX and arXiv could not be reached (DATABASE ACCESS UNAVAILABLE). Elicit access was also unavailable. Their content was reached only through OpenAlex's index.
2. **Index coverage, ranking and retrieval cap.** OpenAlex's lexical relevance ranking surfaced general surveys ahead of specific recent preprints. Only the top 24–25 records per query page were screened, although index estimates ran to thousands. The database search retrieved only 3 of the 13 named prior-art studies, which shows that relevant studies can be missed.
3. **Non-independent second stream.** Thirty-one included studies came from the reference list of the previous chapter, which reflects the author's prior reading. This introduces selection bias, which the protocol's requirement to treat named prior art identically mitigates but does not remove.
4. **No full texts.** Eligibility, extraction and appraisal were performed on record-level evidence. Nine included studies are PARTIALLY VERIFIED. Appraisal items that depend on full text (reproducibility, comparators, limitations) are mostly Not Verified.
5. **Single reviewer.** Screening, extraction and appraisal were each done by one person, with no second screener and no agreement statistic. The appraisal checklist is transparent but is not a validated risk-of-bias instrument.
6. **Rapidly changing preprint literature.** 35 of the 71 included studies are preprints or working papers, and the field produces new preprints monthly. One source (FinVault) has been withdrawn. Preprint results may change or be withdrawn.
7. **Publication bias.** Defenses are usually reported on the authors' own benchmarks with favorable configurations. Negative results appear mainly in attack papers.
8. **Heterogeneity and no pooling.** Architectures, threats, benchmarks and metrics differ too much for meta-analysis. The synthesis is qualitative, and numeric comparisons across studies are avoided.
9. **Metadata errors in the index.** One record was corrupted (AgentDojo's DOI attached to another abstract) and one venue year was inconsistent (PromptShield). Both were detected and recorded, but others may have gone unnoticed.

Before submission. The protocol's Boolean blocks should be run directly in at least IEEE Xplore, the ACM DL, Scopus or Web of Science, and arXiv. Full texts of the included studies should be appraised, and the PRISMA flow and gap assessment updated.

2.13.5 Prior-art risks to monitor

1. **NEXUS.** If a later version evaluates per tool call on an executable benchmark, or makes intervention losses depend on each action's consequence, Gap A largely closes.
2. **AgentTrust.** If a later version reports calibrated probabilities or consequence-weighted decisions, Gap A narrows. AgentTrust shares its name with another arXiv preprint (2606.08539) and must not be confused with it.
3. **MCP policy-enforcement point and LATTICE.** Both are peer-reviewed, HIGH EVIDENCE studies of action-boundary enforcement with ablation or baseline analysis. A probabilistic extension of either would narrow Gaps A, C and E.
4. **FinHarness.** A re-evaluation on an open executable benchmark with a probabilistic decision layer would narrow Gap D.

5. **Jackson (2025).** The full text was not accessed. If it documents calibrated, consequence-weighted selection with evaluation, Gaps A and E narrow.
6. **Unsearched databases.** Studies in the libraries that could not be searched may already report the combination stated in Section 2.13.2.

2.14 Chapter Summary

This chapter reported a systematic literature review, following PRISMA 2020 and PRISMA-S, of runtime security for tool-using AI agents, with attention to financial execution.

Selection. The search of OpenAlex on 25 September 2026 identified 320 records. After deduplication, 219 were screened, and 45 were assessed for eligibility at record level. Forty database studies were included. Another 31 came from the verified reference list of the previous chapter, giving 71 included studies. Of these, 10 are HIGH, 36 MODERATE and 25 LIMITED EVIDENCE.

Themes. The synthesis established the following:

- **Threats.** Indirect prompt injection through the observation channel is the best-evidenced threat to tool-using agents, and it is adaptive. Its harm materializes at the proposed tool call.
- **Defenses.** Runtime interception, deterministic enforcement, information-flow architectures, learned guardrails and provenance checks are all established. Adaptive attacks have repeatedly broken defenses, including architectural ones.
- **Behavioral and provenance evidence.** Many monitors observe trajectories. Evidence on their contribution is sparse and conflicting: a null ablation of a session tracker, and a large effect of cross-step information-flow labels.
- **Algorithmic foundations.** Logistic scoring, Platt calibration, expected-cost decisions with a reject option, and additive attribution are established. One agent-safety system (NEXUS) combines calibration with costs at the plan level. Calibration or a confidence threshold alone did not secure good control in two independent studies. CUSUM is reviewed only as an optional, observe-only analysis.
- **Decision and mitigation.** Graduated responses with human review are standard. How to combine a probability of adversarial induction with a declared consequence remains open. Intervention rates are rarely reported.
- **Financial agents.** All financial-context evidence is LIMITED, and the only quantitative financial runtime results rest on a withdrawn benchmark.

The gap. The research gap is accordingly **integrative and empirical**. It concerns the evaluation, on an open, executable banking benchmark, of a runtime layer that combines:

- provenance-based action evidence;
- a calibrated per-step probability;
- consequence-tiered expected-loss decisions;
- deterministic mitigation.

The evaluation must measure attack success, benign utility, utility under attack, intervention behavior and latency separately, and report calibration separately from control, against a consequence-independent baseline that uses the same estimator. Every mechanism in this combination is established. Only the combination at the action boundary and the evaluation design are proposed by this thesis.

Chapter 3 specifies the REM methodology through which the gap is addressed. SC-1 and SC-2 remain open, pending supervisor decision.

References

APA 7th edition. Preprints, working papers and withdrawn preprints are marked. Official and regulatory sources are marked [Official source] and are not treated as research literature. For arXiv-only records identified by the database search, the author list is abbreviated as it appears in the index record and is marked as such; the full lists were not verifiable in the review environment.

Andriushchenko, M., Souly, A., Dziemian, M., Duenas, D., Lin, M., Wang, J., Hendrycks, D., Zou, A., Kolter, Z., Fredrikson, M., Winsor, E., Wynne, J., Gal, Y., & Davies, X. (2025). AgentHarm: A benchmark for measuring harmfulness of LLM agents. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*. https://arxiv.org/abs/2410.09024

Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., & Rieck, K. (2022). Dos and don'ts of machine learning in computer security. In *31st USENIX Security Symposium* (pp. 3971–3988). USENIX Association.

Basseville, M., & Nikiforov, I. V. (1993). *Detection of abrupt changes: Theory and application*. Prentice Hall.

Calboreanu, E. (2026). LATTICE: A governance-first architecture for authorized autonomous AI operations. *Frontiers in Artificial Intelligence, 9*, Article 1800407. https://doi.org/10.3389/frai.2026.1800407

Castro-Maldonado, V., Aceves-Fernández, M. A., García-Noguez, L. R., & Pedraza-Ortega, J. C. (2026). Semantic firewalls with online ensemble learning for secure agentic RAG systems in financial chatbots. *AI, 7*(3), Article 80. https://doi.org/10.3390/ai7030080

Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys, 41*(3), Article 15, 1–58. https://doi.org/10.1145/1541880.1541882

Chandola, V., Banerjee, A., & Kumar, V. (2012). Anomaly detection for discrete sequences: A survey. *IEEE Transactions on Knowledge and Data Engineering, 24*(5), 823–839. https://doi.org/10.1109/TKDE.2010.235

Chen, H.-H. (2026). *Insuring every action: An authority frontier framework for runtime actuarial control of autonomous AI agents* (arXiv:2605.25632) [Preprint]. arXiv. https://arxiv.org/abs/2605.25632

Chen, S., Piet, J., Sitawarin, C., & Wagner, D. (2025). StruQ: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium*. USENIX Association.

Chen, Y., Li, H., Zheng, Z., Wu, D., Song, Y., & Hooi, B. (2025). Defense against prompt injection attack by leveraging attack techniques. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 18331–18347). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.acl-long.897

Chen, Z., Chen, J., Chen, J., & Sra, M. (2025). *Standard benchmarks fail — Auditing LLM agents in finance must prioritize risk* (arXiv:2502.15865) [Preprint]. arXiv. https://arxiv.org/abs/2502.15865

Chen, Z., Dong, D., Liu, H., Li, J., Zhai, J., Xu, D., & Pu, B. (2026). *Cordon: Semantic transactions for tool-using LLM agents* (arXiv:2606.17573) [Preprint]. arXiv. https://arxiv.org/abs/2606.17573

Chen, Z., Kang, M., & Li, B. (2025). ShieldAgent: Shielding agents via verifiable safety policy reasoning. In *Proceedings of the 42nd International Conference on Machine Learning* (Proceedings of Machine Learning Research, Vol. 267, pp. 8313–8344). PMLR.

Chen, Z., Xiang, Z., Xiao, C., Song, D., & Li, B. (2024). AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*.

Chennabasappa, S., Nikolaidis, C., Song, D., Molnar, D., Ding, S., Wan, S., Whitman, S., Deason, L., Doucette, N., Montilla, A., Gampa, A., de Paola, B., Gabi, D., Crnkovich, J., Testud, J.-C., He, K., Chaturvedi, R., Zhou, W., & Saxe, J. (2025). *LlamaFirewall: An open source guardrail system for building secure AI agents* (arXiv:2505.03574) [Preprint]. arXiv. https://arxiv.org/abs/2505.03574

Chow, C. K. (1970). On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory, 16*(1), 41–46. https://doi.org/10.1109/TIT.1970.1054406

Cox, D. R. (1958). The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B (Methodological), 20*(2), 215–232. https://doi.org/10.1111/j.2517-6161.1958.tb00292.x

Debenedetti, E., Shumailov, I., Fan, T., Hayes, J., Carlini, N., Fabian, D., Kern, C., Shi, C., Terzis, A., & Tramèr, F. (2025). *Defeating prompt injections by design* (arXiv:2503.18813v2) [Preprint]. arXiv. https://arxiv.org/abs/2503.18813

Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., & Tramèr, F. (2024). AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*.

Elkan, C. (2001). The foundations of cost-sensitive learning. In *Proceedings of the Seventeenth International Joint Conference on Artificial Intelligence (IJCAI-01)* (pp. 973–978).

European Parliament & Council of the European Union. (2024). *Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*. Official Journal of the European Union. https://eur-lex.europa.eu/eli/reg/2024/1689/oj [Official source]

Forrest, S., Hofmeyr, S. A., Somayaji, A., & Longstaff, T. A. (1996). A sense of self for Unix processes. In *Proceedings of the 1996 IEEE Symposium on Security and Privacy* (pp. 120–128). IEEE. https://doi.org/10.1109/SECPRI.1996.502675

Fu, X., Li, S., Wang, Z., et al. (2024). *Imprompter: Tricking LLM agents into improper tool use* (arXiv:2410.14923) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2410.14923 [Author list abbreviated from the index record; full list not verified]

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence*

and Security (pp. 79–90). ACM. https://doi.org/10.1145/3605764.3623985

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning* (Proceedings of Machine Learning Research, Vol. 70, pp. 1321–1330). PMLR.

Hasan, M. M., Li, H., Fallahzadeh, E., Rajbahadur, G. K., Adams, B., & Hassan, A. E. (2026). Model Context Protocol (MCP) at first glance: Studying the security and maintainability of MCP servers. *ACM Transactions on Software Engineering and Methodology*. https://doi.org/10.1145/3814959

Hines, K., Lopez, G., Hall, M., Zarfati, F., Zunger, Y., & Kiciman, E. (2024). *Defending against indirect prompt injection attacks with spotlighting* (arXiv:2403.14720) [Preprint]. arXiv. https://arxiv.org/abs/2403.14720

Hossain, E., Nipu, M. M. H., Ornee, T. N., Rana, R., & Yousefi, N. (2026). *NEXUS: Structured runtime safety for tool-using LLM agents* (arXiv:2607.19356) [Preprint]. arXiv. https://arxiv.org/abs/2607.19356

Hua, W., Yang, X., Jin, M., Li, Z., Cheng, W., Tang, R., & Zhang, Y. (2024). TrustAgent: Towards safe and trustworthy LLM-based agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024* (pp. 10000–10016). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.findings-emnlp.585

Huang, C., Huang, X., Tran, N. P., & Milani Fard, A. (2026). Model Context Protocol threat modeling and analysis of vulnerabilities to prompt injection with tool poisoning. *Journal of Cybersecurity and Privacy, 6*(3), Article 84. https://doi.org/10.3390/jcp6030084

Jackson, F. (2025). *Designing a policy engine for agentic AI systems: From governance requirements to runtime enforcement* [Working paper]. SSRN. https://doi.org/10.2139/ssrn.5904104

Jacob, D., Alzahrani, H., Hu, Z., Alomair, B., & Wagner, D. (2025). PromptShield: Deployable detection for prompt injection attacks. In *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy* (pp. 341–352). ACM. https://doi.org/10.1145/3714393.3726501 [Crossref records issued year 2024 and online publication 2025]

Jia, F., Wu, T., Qin, X., & Squicciarini, A. (2025). The Task Shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics. https://aclanthology.org/2025.acl-long.1435/

Jia, H., Liu, Y., Chong, B., Yang, Y., Chen, Y., Liang, J., Li, Q., Lu, H., Xu, K., Zheng, H., Zhang, C., Peng, H., & Yu, P. S. (2026). *FinHarness: An inline lifecycle safety harness for finance LLM agents* (arXiv:2605.27333) [Preprint]. arXiv. https://arxiv.org/abs/2605.27333

Jia, X., Liao, J., Qin, S., et al. (2026). *SkillJect: Effectively automating skill-based prompt injection for skill-enabled agents* (arXiv:2602.14211) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2602.14211 [Author list abbreviated from the index record; full list not verified]

Jiang, C., Zhang, W., Pan, X., Hong, G., & Yang, M. (2025). *Think twice before you act: Enhancing agent behavioral safety with thought correction* (arXiv:2505.11063) [Preprint; accepted to ICML 2026]. arXiv. https://arxiv.org/abs/2505.11063

Johnson, S., Pham, V., & Le, T. (2025). The dangers of indirect prompt injection attacks on LLM-based autonomous web navigation agents: A demonstration. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (pp. 729–738). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.emnlp-demos.55

Kaptein, M., Khan, V.-J., & Podstavnychy, A. (2026). *Runtime governance for AI agents: Policies on paths* (arXiv:2603.16586) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2603.16586

Kull, M., Silva Filho, T., & Flach, P. (2017). Beta calibration: A well-founded and easily implemented improvement on logistic calibration for binary classifiers. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics* (Proceedings of Machine Learning Research, Vol. 54, pp. 623–631). PMLR.

le Cessie, S., & van Houwelingen, J. C. (1992). Ridge estimators in logistic regression. *Applied Statistics, 41*(1), 191–201. https://doi.org/10.2307/2347628

Li, E., Mallick, T., Rose, E., Robertson, W., Oprea, A., & Nita-Rotaru, C. (2026). ACE: A security architecture for LLM-integrated app systems. In *Proceedings of the 2026 Network and Distributed System Security Symposium (NDSS)*. Internet Society. https://doi.org/10.14722/ndss.2026.230352

Li, F. (2026). *OpenClaw PRISM: A zero-fork, defense-in-depth runtime security layer for tool-augmented LLM agents* (arXiv:2603.11853) [Preprint]. arXiv. https://arxiv.org/abs/2603.11853

Li, H., Liu, X., Chiu, C., Li, D., Zhang, N., & Xiao, C. (2025). DRIFT: Dynamic rule-based defense with injection isolation for securing LLM agents. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)* (pp. 92825–92853). https://doi.org/10.52202/085713-2791

Li, M. Q., Fung, B. C. M., Li, B., Ismail, H., & Iqbal, F. (2026). *Taxonomy and consistency analysis of safety benchmarks for AI agents* (arXiv:2605.16282) [Preprint]. arXiv. https://arxiv.org/abs/2605.16282

Lin, J., Zhou, Z., Zheng, Z., Liu, S., Xu, T., Chen, Y., & Chen, E. (2026). VIGIL: Defending LLM agents against tool-stream injection via verify-before-commit. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 9764–9785). Association for Computational Linguistics. https://doi.org/10.18653/v1/2026.acl-long.443

Lin, W., Yu, C., Lin, X., Cao, S., Chen, X., Xue, L., Yu, L., Sha, L., & Wu, C. (2026). *DreamGuard: Efficient runtime guardrail for LLM agents via risk-aware world model* (arXiv:2608.05695) [Preprint]. arXiv. https://arxiv.org/abs/2608.05695

Liu, H., Ilyushin, E., Ni, J., & Zhu, M. (2026). *SafeAgent: A runtime protection architecture for agentic systems* (arXiv:2604.17562) [Preprint]. arXiv. https://arxiv.org/abs/2604.17562

Liu, T., Deng, Z., Meng, G., et al. (2023). *Demystifying RCE vulnerabilities in LLM-integrated apps* (arXiv:2309.02926) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2309.02926 [Author list abbreviated from the index record; full list not verified]

Liu, Y., Deng, G., Li, Y., et al. (2023). *Prompt injection attack against LLM-integrated applications* (arXiv:2306.05499) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2306.05499 [Author list abbreviated from the index record; full list not verified]

Liu, Y., Jia, Y., Geng, R., Jia, J., & Gong, N. Z. (2024). Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium* (pp. 1831–1847). USENIX Association.

Lorden, G. (1971). Procedures for reacting to a change in distribution. *The Annals of Mathematical Statistics, 42*(6), 1897–1908. https://doi.org/10.1214/aoms/1177693055

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)* (pp. 4765–4774).

Luo, W., Dai, S., Liu, X., Banerjee, S., Sun, H., Chen, M., & Xiao, C. (2025). AGrail: A lifelong agent guardrail with effective and adaptive safety detection. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 8104–8139). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.acl-long.399

Mao, Q., Wang, J., Liu, Y., Zhu, L., Ma, C., & Yan, J. (2026). *SoK: Security of autonomous LLM agents in agentic commerce* (arXiv:2604.15367) [Preprint]. arXiv. https://arxiv.org/abs/2604.15367

Mou, Y., Xue, Z., Li, L., Liu, P., Zhang, S., Ye, W., & Shao, J. (2026). ToolSafe: Enhancing tool invocation safety of LLM-based agents via proactive step-level guardrail and feedback. In *Findings of the Association for Computational Linguistics: ACL 2026* (pp. 37125–37153). Association for Computational Linguistics. https://doi.org/10.18653/v1/2026.findings-acl.1850

Naeini, M. P., Cooper, G., & Hauskrecht, M. (2015). Obtaining well calibrated probabilities using Bayesian binning. *Proceedings of the AAAI Conference on Artificial Intelligence, 29*(1). https://doi.org/10.1609/aaai.v29i1.9602

Ng, A. Y., & Jordan, M. I. (2002). On discriminative vs. generative classifiers: A comparison of logistic regression and naive Bayes. In *Advances in Neural Information Processing Systems 14 (NIPS 2001)*. https://papers.nips.cc/paper/2020-on-discriminative-vs-generative-classifiers-a-comparison-of-logistic-regression-and-naive-bayes

Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning* (pp. 625–632). ACM. https://doi.org/10.1145/1102351.1102430

OWASP Gen AI Security Project. (2025). *LLM01:2025 Prompt injection*. OWASP Foundation. https://genai.owasp.org/llmrisk/llm01-prompt-injection/ [Official source]

Page, E. S. (1954). Continuous inspection schemes. *Biometrika, 41*(1–2), 100–115. https://doi.org/10.1093/biomet/41.1-2.100

Patil, S. G., Zhang, T., Fang, V., et al. (2024). *GoEX: Perspectives and designs towards a runtime for autonomous LLM applications* (arXiv:2404.06921) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2404.06921 [Author list abbreviated from the index record; full list not verified]

Pedro, R., Amaral de Castro, D., Carreira, P., et al. (2023). *From prompt injections to SQL injection attacks: How protected is your LLM-integrated web application?* (arXiv:2308.01990) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2308.01990 [Author list abbreviated from the index record; full list not verified]

Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In A. J. Smola, P. Bartlett, B. Schölkopf, & D. Schuurmans (Eds.), *Advances in large margin classifiers* (pp. 61–74). MIT Press.

Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C. J., & Hashimoto, T. (2024). Identifying the risks of LM agents with an LM-emulated sandbox. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Schneider, F. B. (2000). Enforceable security policies. *ACM Transactions on Information and System Security, 3*(1), 30–50. https://doi.org/10.1145/353323.353382

She, Y., Liang, Y., & Kang, E. (2026). *Safeguarding LLM agents from misalignment through provenance analysis* (arXiv:2607.01236) [Preprint]. arXiv. https://arxiv.org/abs/2607.01236

Shi, J., Yuan, Z., Tie, G., Zhou, P., Gong, N., & Sun, L. (2026). Prompt injection attack to tool selection in LLM agents. In *Proceedings of the 2026 Network and Distributed System Security Symposium (NDSS)*. Internet Society. https://doi.org/10.14722/ndss.2026.230675

Shi, T., He, J., Wang, Z., Wu, L., Li, H., Guo, W., & Song, D. (2025). *Progent: Securing AI agents with privilege control* (arXiv:2504.11703) [Preprint]. arXiv. https://arxiv.org/abs/2504.11703

Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. In *2010 IEEE Symposium on Security and Privacy* (pp. 305–316). IEEE. https://doi.org/10.1109/SP.2010.25

Storf, S., Barton-Cooper, R., Peters-Gill, J., et al. (2026). *Constitutional black-box monitoring for scheming in LLM agents* (arXiv:2603.00829) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2603.00829 [Author list abbreviated from the index record; full list not verified]

Tsai, L., & Bagdasarian, E. (2025). Contextual agent security: A policy for every purpose. In *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS '25)* (pp. 8–17). ACM. https://doi.org/10.1145/3713082.3730378

Wang, C. L., Singhal, T., Kelkar, A., & Tuo, J. (2025). *MI9: An integrated runtime governance framework for agentic AI* (arXiv:2508.03858) [Preprint]. arXiv. https://arxiv.org/abs/2508.03858

Wang, H., Poskitt, C. M., & Sun, J. (2026). AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*. ACM. https://doi.org/10.1145/3744916.3764546

Wang, H., Poskitt, C. M., Wei, J., & Sun, J. (2025). *ProbGuard: Probabilistic runtime monitoring for LLM agent safety* (arXiv:2508.00500; earlier versions titled "Pro2Guard") [Preprint]. arXiv. https://arxiv.org/abs/2508.00500

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science, 18*(6), Article 186345. https://doi.org/10.1007/s11704-024-40231-1

Wang, P., Li, Y., & Tian, Y. (2026). *Reframing LLM agent security as an agent-human interaction problem* (arXiv:2605.24309) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2605.24309

Wang, S., Zhu, S., & Li, R. (2026). Runtime policy enforcement for MCP-based LLM agents. *Electronics, 15*(13), Article 2829. https://doi.org/10.3390/electronics15132829

Wang, Y., Han, X., Shang, D., Tang, Y., & Liu, B. (2026). *Safety, or just capability? A validity audit of agent-safety benchmarks* (arXiv:2607.28685) [Preprint]. arXiv. https://arxiv.org/abs/2607.28685

Wang, Z., Siu, V., Ye, Z., Shi, T., Nie, Y., Zhao, X., Wang, C., Guo, W., & Song, D. (2025). AGENTVIGIL: Automatic black-box red-teaming for indirect prompt injection against LLM agents. In *Findings of the Association for Computational Linguistics: EMNLP 2025* (pp. 23159–23172). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.findings-emnlp.1258

Wei, B., Zhang, Y., Pan, J., et al. (2026). *ClawSafety: "Safe" LLMs, unsafe agents* (arXiv:2604.01438) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2604.01438 [Author list abbreviated from the index record; full list not verified]

Wen, T., Wang, C., Yang, X., Tang, H., Xie, Y., Lyu, L., Dou, Z., & Wu, F. (2025). Defending against indirect prompt injection by instruction detection. In *Findings of the Association for Computational Linguistics: EMNLP 2025* (pp. 19472–19487). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.findings-emnlp.1060

Wu, F., Cecchetti, E., & Xiao, C. (2024). *System-level defense against indirect prompt injection attacks: An information flow control perspective* (arXiv:2409.19091) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2409.19091

Wu, Y., Roesner, F., Kohno, T., Zhang, N., & Iqbal, U. (2025). IsolateGPT: An execution isolation architecture for LLM-based systems. In *Proceedings of the 2025 Network and Distributed System Security Symposium (NDSS)*. Internet Society. https://doi.org/10.14722/ndss.2025.241131

Xiang, Z., Zheng, L., Li, Y., Hong, J., Li, Q., Xie, H., Zhang, J., Xiong, Z., Xie, C., Yang, C., Song, D., & Li, B. (2025). GuardAgent: Safeguard LLM agents via knowledge-enabled reasoning. In *Proceedings of the 42nd International Conference on Machine Learning* (Proceedings of Machine Learning Research, Vol. 267, pp. 68316–68342). PMLR.

Yang, C. (2026). *AgentTrust: Runtime safety evaluation and interception for AI agent tool use* (arXiv:2605.04785) [Preprint]. arXiv. https://arxiv.org/abs/2605.04785

Yang, Z., Li, R., Qiang, Q., Wang, J., Lou, F., Li, M., Cheng, D., Xu, R., Lian, H., Zhang, S., Liang, X., Huang, X., Wei, Z., Liu, Z., Guo, X., Wang, H., Chen, R., & Zhang, L. (2026). *FinVault: Benchmarking financial agent safety in execution-grounded environments* (arXiv:2601.07853) [Withdrawn preprint]. arXiv. https://arxiv.org/abs/2601.07853

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR 2023)*.

Yu, M., Meng, F., Zhou, X., Wang, S., Mao, J., Pan, L., Chen, T., Wang, K., Li, X., Zhang, Y., An, B., & Wen, Q. (2025). A survey on trustworthy LLM agents: Threats and countermeasures. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2* (pp. 6216–6226). ACM. https://doi.org/10.1145/3711896.3736561

Yuan, T., He, Z., Dong, L., Wang, Y., Zhao, R., Xia, T., Xu, L., Zhou, B., Li, F., Zhang, Z., Wang, R., & Liu, G. (2024). R-Judge: Benchmarking safety risk awareness for LLM agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024* (pp. 1467–1490). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.findings-emnlp.79

Zadrozny, B., & Elkan, C. (2002). Transforming classifier scores into accurate multiclass probability estimates. In *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 694–699). ACM. https://doi.org/10.1145/775047.775151

Zhan, Q., Fang, R., Panchal, H. S., & Kang, D. (2025). Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents. In *Findings of the Association for Computational Linguistics: NAACL 2025* (pp. 7116–7132). Association for Computational Linguistics. https://aclanthology.org/2025.findings-naacl.395/

Zhan, Q., Liang, Z., Ying, Z., & Kang, D. (2024). InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024* (pp. 10471–10506). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.findings-acl.624

Zhang, B., Tan, Y., Shen, Y., Salem, A., Backes, M., Zannettou, S., & Zhang, Y. (2025). Breaking agents: Compromising autonomous LLM agents through malfunction amplification. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing* (pp. 34952–34964). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.emnlp-main.1771

Zhang, C., Wan, Z., Yu, X., Wu, J., Wen, Q., Zhou, P., Zhao, W., & Tsang, I. (2026). *Calibration is not control: Why LLM-agent oversight needs intervention* (arXiv:2606.21399) [Preprint]. arXiv. https://arxiv.org/abs/2606.21399

Zhang, H., Huang, J., Mei, K., Yao, Y., Wang, Z., Zhan, C., Wang, H., & Zhang, Y. (2025). Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*.

Zhang, Y., Chen, K., Gao, J., et al. (2024). *Towards action hijacking of large language model-based agent* (arXiv:2412.10807) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2412.10807 [Author list abbreviated from the index record; full list not verified]

Zhong, P. Y., Chen, S., Wang, R., et al. (2025). *RTBAS: Defending LLM agents against prompt injection and privacy leakage* (arXiv:2502.08966) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2502.08966 [Author list abbreviated from the index record; full list not verified]

Zhu, K., Yang, X., Wang, J., Guo, W., & Wang, W. Y. (2025). MELON: Provable defense against indirect prompt injection attacks in AI agents. In *Proceedings of the 42nd International Conference on Machine Learning* (Proceedings of Machine Learning Research, Vol. 267). PMLR.

Zhu, K., Zhang, J., Qi, Z., et al. (2025). *SafeScientist: Toward risk-aware scientific discoveries by LLM agents* (arXiv:2505.23559) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2505.23559 [Author list abbreviated from the index record; full list not verified]

CHAPTER THREE

RESEARCH METHODOLOGY AND DESIGN OF THE REM FRAMEWORK

3.1 Research Methodology and System Scope

3.1.1 Design science research

This thesis follows design science research, in which knowledge is produced by building an artifact that addresses an identified problem and evaluating it rigorously (Hevner et al., 2004). The research process follows the six activities of the design science research methodology of Peffers et al. (2007). Table 3.1 maps these activities to the thesis.

Table 3.1. Design science research activities in this thesis

Activity (Peffers et al., 2007)	Realization in this thesis	Location
Problem identification and motivation	Runtime compromise of tool-using financial agents through indirect prompt injection	Chapter 1; Chapter 2
Definition of objectives for a solution	Objectives O1–O3; research questions RQ1–RQ4; security objectives SO-1 to SO-6	Sections 3.1.2, 3.1.3, 3.3.7
Design and development	Five-layer REM architecture and its runtime pipeline	Sections 3.2–3.10
Demonstration	Integration of REM with the AgentDojo banking environment	Chapter 4
Evaluation	Controlled experiments, baseline comparison, and ablation	Sections 3.11–3.12; Chapter 5
Communication	Thesis and derived publications	Chapter 6

The evaluation is quantitative and experimental. REM is compared against a baseline that uses the same probability estimator but a consequence-independent decision rule, so that differences in outcomes can be attributed to the decision rule rather than to the estimator.

3.1.2 Research objectives

The approved objectives are:

- **O1:** Design a REM framework for financial AI agents' and task automation agents' security.
- **O2:** Evaluate the framework against adversarial attacks.
- **O3:** Compare performance with existing security approaches.

This chapter realizes O1 through the architecture and runtime pipeline, O2 through the attack scenarios and metrics of Sections 3.11–3.12, and O3 through the baselines of Section 3.11.7. The evaluation in this thesis uses a financial (banking) environment. Task-automation environments are outside the evaluation scope unless approved as an extension.

3.1.3 Research questions

The approved proposal does not state research questions. The following questions are derived from the objectives and are presented for supervisor confirmation.

- **RQ1 (estimation).** How well does a ridge logistic regression estimator, using context, behavioral, and action evidence available at the tool-call boundary, discriminate adversarially induced proposed actions in the AgentDojo banking environment, and how well calibrated are its probabilities under grouped cross-fitting?
- **RQ2 (decision).** Compared with a consequence-independent threshold policy that uses the same estimator, how does consequence-tiered expected-loss verdict selection change attack success, benign utility, utility under attack, and intervention rates, across a declared range of loss structures?
- **RQ3 (mitigation and cost).** What residual attack success remains after the Modify mechanism, how often is escalation selected, and what per-step latency does REM add?
- **RQ4 (component contribution).** What is the marginal contribution of each evidence group, of calibration, and of consequence tiers, as determined by ablation?

Table 3.2. Traceability of research questions

RQ	Objectives	Principal components	Principal metrics
RQ1	O1, O2	Detection and Behavioral Analysis evidence; estimator; calibration	ROC-AUC, PR-AUC, ECE, log-loss
RQ2	O2, O3	Consequence tiers; expected-loss decision; baseline policy	ASR, benign utility, utility under attack, intervention rates, realized loss

RQ	Objectives	Principal components	Principal metrics
RQ3	O2	Mitigation Layer; Escalate; runtime pipeline	Residual ASR under Modify, escalation rate, latency
RQ4	O2	All components (ablation)	Change in the metrics above

3.1.4 Scope

In scope:

- A single LLM-based agent that invokes tools with generated arguments.
- A financial (banking) task environment with executable tools and state-based checks.
- Indirect prompt injection through tool outputs as the evaluated attack class.
- Runtime mediation of each proposed tool call before it is released.

Out of scope:

- Training-time defenses and modification of the agent's model or prompts.
- Multi-agent systems.
- Evaluation against adaptive adversaries who tune attacks to REM.
- Production deployment in a financial institution.
- Counterfactual estimation of intervention value by replaying agent trajectories.

3.1.5 Assumptions

Table 3.3. Assumptions

ID	Assumption	Type	Status
AS-1	REM can intercept each proposed tool call and each tool result without modifying the agent's internals.	Architectural	To be demonstrated on the chosen platform (Chapter 4)
AS-2	Context segments can be attributed to their source channel reliably enough to assign a provenance label.	Architectural	To be demonstrated
AS-3	The adversary cannot modify REM's code, parameters, policy, or logs.	Architectural	Assumed; stated as a limitation
AS-4	The user's objective is legitimate; REM protects the user against external manipulation.	Scoping	Stated choice
AS-5	Tool outputs and retrieved content are untrusted by default.	Scoping	Follows from the indirect injection threat (Greshake et al., 2023)
AS-6	Adversaries do not adapt to REM's parameters in the primary evaluation.	Scoping	Declared limitation
AS-7	Observable evidence separates adversarially induced from benign proposed actions.	Empirical	Tested in pilot E0 (Section 3.11.8)
AS-8	The benign reference data are representative of the evaluated workload.	Empirical	Threatened by benchmark scope (Section 3.14)
AS-9	LLM non-determinism is bounded enough for repeated runs to characterize variance.	Empirical	Measured through repeated runs
AS-10	The agent's internal reasoning is not used; REM relies only on observable inputs, proposed actions, and tool results.	Architectural	Design constraint

3.1.6 Notation

Table 3.4. Notation used in this chapter

Symbol	Meaning
t, T	Step index within an episode; episode length
g	User objective of the episode
c_{t}	Context at step t, as segments with provenance labels
a_{t}	Proposed action (tool name and arguments) at step t

Symbol	Meaning
H_{t}	Episode history up to step t
$y_{t} \in \{0, 1\}$	Latent label: 1 if a_{t} is adversarially induced
$x_{t} \in [0, 1]^{d}$	Evidence vector; groups x^{ctx}, x^{beh}, x^{act} with index sets I_{ctx}, I_{beh}, I_{act}
β_{0}, β, λ	Logistic intercept, coefficient vector, ridge strength
s_{t}	Logit (linear predictor)
$\tilde{p}_{t}$, p_{t}	Uncalibrated probability; calibrated probability
γ_{1}, γ_{0}	Calibration slope and intercept
z_{t}, k_{t}, κ	Consequence descriptors; consequence tier; tier map
$\mathcal{V}$	Verdict set {Allow, Modify, Escalate, Block}
$\mathcal{V}(a_{t})$	Verdicts feasible for a_{t}
v_{t}	Verdict issued at step t
L(v, y, k)	Loss of verdict v when the label is y and the tier is k
$C_{miss}(k)$, $C_{FA}(k)$, $C_{mod}(k)$, C_{esc}	Declared costs of a missed attack, a false intervention by Block, Modify on a legitimate action, and escalation
$r_{mod}(k)$	Residual attack-success rate after Modify at tier k (measured)
$R_{t}(v)$	Conditional risk of verdict v at step t
$\pi_{v,v'}(k)$, p^{*}_{k}	Indifference threshold between verdicts v and v′; Allow–Block threshold
$\sigma(\cdot)$	Logistic function, $\sigma(u) = 1 / (1 + \exp(-u))$
Π, k_{D2}	Set of prohibited actions; minimum tier for predicate D2
$\theta(\alpha)$, α_{blk}, α_{mod}	Baseline threshold for false-positive target α; targets for Block and Modify
$\mathcal{T}_{R}$	Set of tools to which the Modify mechanism applies

Symbols used only in the optional sequential analysis are defined in Section 3.11.10. Symbols that appear in a single equation, such as Δ_{0}, Δ_{1}, B_{m}, and the step sets D_{fit}, D_{cal}, and D_{val}^{benign}, are defined where they are used. Each symbol has one meaning throughout the chapter.

3.2 REM Architecture

3.2.1 Five layers

REM is a runtime mediation layer between one agent and its environment. It comprises exactly five layers, as approved in the research proposal:

1. **Input & Context Layer.** Captures the information the agent handles and the action it proposes, and labels each context segment with its provenance.
2. **Detection Layer.** Computes evidence about untrusted context.
3. **Behavioral Analysis Layer.** Computes evidence about the episode history and the proposed action.
4. **Decision Engine Layer.** Estimates the probability that the proposed action is adversarially induced, assigns a consequence tier, applies policy predicates, selects a verdict, and writes the audit record.
5. **Mitigation Layer.** Executes the mechanism associated with the verdict.

Offline fitting and recalibration (Section 3.9) are cross-cutting activities performed between deployments. They are not a layer, are not numbered with the layers, and never change REM's parameters during an episode.

3.2.2 Topology and information flow

At each step, REM executes Layer 1, then Layers 2 and 3 in parallel on Layer 1's output, then Layer 4, then Layer 5, before the proposed action is released to the environment. The Detection and Behavioral Analysis Layers compute their evidence independently; they are combined only in the Decision Engine. This independence allows an evidence group to be removed in ablation without changing the computation of the other. After an action is released, its tool result re-enters Layer 1 as untrusted content for the next step.

Figure 3.1. REM architecture. The figure is to be drawn with the following elements and no others:

- **External elements:** user and system inputs; tool outputs and retrieved content (untrusted); the agent (opaque); tools and environment; human reviewer.
- **Five layer boxes:**
  - L1: capture and normalization; provenance labeling; entity extraction.
  - L2: injection-classifier score; instruction-pattern indicator.

    - L3: history tracker; behavioral features; action features.
    - L4: logistic estimation; calibration; consequence tier; policy predicates; expected-loss verdict with four outputs; audit record.
    - L5: release; tool restriction; withhold and escalate; block with safe fallback.
- **Stores (not layers):** declared policy; frozen parameters; audit log.
- **Cross-cutting box** (dashed, outside the layer stack): offline fitting and recalibration, connected to the audit log and to the frozen parameters.
- **Runtime arrows:** from L1 to L2 and L3 in parallel; from L2 and L3 to L4; from each verdict to its mechanism in L5; from L5 to the environment or back to the agent.

No arrow connects the offline box to the runtime path within an episode.

3.2.3 Layer interfaces

Table 3.5. Layer interfaces

Layer	Consumes	Produces
L1 Input & Context	Objective, system prompt, user messages, tool results, proposed action	Labeled context c_{t}; entities with provenance; held action a_{t}
L2 Detection	Untrusted segments of c_{t}	Context evidence x^{ctx}
L3 Behavioral Analysis	H_{t}, a_{t}, entities, benign reference transitions, privilege map	Behavioral evidence x^{beh}; action evidence x^{act}
L4 Decision Engine	x_{t}, consequence descriptors z_{t}, frozen parameters, declared policy	p_{t}, k_{t}, R_{t}(·), verdict v_{t}, audit record
L5 Mitigation	v_{t}, a_{t}, episode restrictions	Released, restricted, withheld, or refused action

The four functions are separated architecturally. Layers 2 and 3 produce evidence and issue no verdicts. Layer 4 performs probability estimation and decision. Layer 5 performs mitigation.

3.3 Threat Model and Financial Action Boundary

3.3.1 Protected system

The protected system is a single LLM-based agent that pursues a user objective by proposing tool calls. REM treats the agent as opaque: it observes inputs, context, proposed actions, and tool results, but not model parameters or internal reasoning (AS-10). The evaluation instance is an agent operating the AgentDojo banking environment (Debenedetti et al., 2024), described in Section 3.11.2.

3.3.2 Episodes and steps

An episode is one execution of the agent on one user objective g. It is a finite sequence of steps t = 1, …, T, where step t is the interval in which the agent proposes action a_{t}. At step t, REM observes g, the labeled context c_{t}, the proposed action a_{t}, and the history H_{t} of prior contexts, actions, verdicts, and tool results.

3.3.3 Trust boundaries and untrusted content

Table 3.6. Trust boundaries

ID	Boundary	Direction	Treatment in REM
TB-1	User input	User → agent context	Labeled semi-trusted
TB-2	Observation	Tool outputs and retrieved content → agent context	Labeled untrusted; primary observation point
TB-3	Action	Agent → tools and external systems	Primary enforcement point
TB-4	Memory	Context ↔ persistent store	Provenance retained on read and write (design scope only)

Untrusted content is any context segment that entered through TB-2 or TB-4, or that was derived from such a segment. The observation boundary matters because content arriving there enters after the user's request, which is the route exploited by indirect prompt injection (Greshake et al., 2023; OWASP Gen AI Security Project, 2025). The action boundary matters because it is the last point at which an induced action can be prevented rather than repaired.

3.3.4 Adversary model

- **AD-1, content-control adversary (primary, evaluated).**

    - *Can:* author or modify content that the agent receives through tool outputs, including instruction-like text and values such as account identifiers.
    - *Cannot:* access the user channel, observe or modify REM's parameters, or modify agent or REM code.
- **AD-2, prompt-channel adversary (design scope only).** Can submit crafted input through a compromised user channel. The evaluated banking attacks do not exercise this channel.

Excluded:

- adversaries who modify model weights, agent code, or REM;
- infrastructure compromise;
- a malicious operator;
- adaptive adversaries (AS-6).

3.3.5 Indirect injection and adversarially induced actions

Indirect prompt injection occurs when content from an external source, processed by the agent, alters its behavior (OWASP Gen AI Security Project, 2025). REM's target event is narrower and defined at the action level.

A proposed action a_{t} is **adversarially induced** (y_{t} = 1) if executing it would advance an objective introduced by the adversary rather than the user objective g. The label is not observable at runtime. Actions that are unsafe because of the agent's own error without adversarial involvement are outside this definition. The operational derivation of y_{t} for evaluation data is given in Section 3.11.3.

3.3.6 Sensitive financial actions

Table 3.7 groups the tools of the evaluated banking environment by the consequence of an adversarially induced call. The tool names are those registered in the AgentDojo banking suite inspected for this chapter (Section 3.11.2). The grouping is a design classification, not a property reported by the benchmark.

Table 3.7. Action categories in the banking environment

Category	Tools	Consequence if adversarially induced
Read	get_balance, get_iban, get_most_recent_transactions, get_scheduled_transactions, get_user_info, read_file	No direct external effect; may expose data later sent elsewhere
Value transfer	send_money, schedule_transaction	Money sent to an attacker-controlled account
Payment modification	update_scheduled_transaction	Redirection or alteration of a recurring payment
Credential change	update_password	Loss of control over the account
Profile change	update_user_info	Alteration of account holder information

3.3.7 Security objectives

The threat model above states what an adversary can do; the objectives below state what REM must therefore do about it. They are ordered along the path of a single proposed tool call: from the evidence available about it, through the verdict selected for it, to the record left behind and the cost of producing it.

- **SO-1:** Use evidence about untrusted content that enters through TB-2.
- **SO-2:** Detect proposed actions whose arguments originate in untrusted content or that deviate from the episode's benign pattern.
- **SO-3:** Account for the consequence of the proposed action when selecting a verdict.
- **SO-4:** Provide human escalation as a response.
- **SO-5:** Emit, for every verdict, an audit record from which the verdict can be reproduced.
- **SO-6:** Preserve benign task utility and report per-step latency.

3.4 Input & Context Layer

The Input & Context Layer collects and labels information. It computes no risk evidence.

3.4.1 Context capture

At step t the layer receives:

- the user objective g and user messages (TB-1);
- the system prompt;
- the results of previously executed tools (TB-2);
- the proposed action a_{t}, captured at TB-3 and held until Layer 5 acts.

Invisible and formatting characters, such as zero-width characters, are removed from untrusted text before further processing. This is a deterministic hygiene step. It reduces some encoding-based concealment but does not prevent paraphrase-based evasion.

3.4.2 Provenance labeling

Each context segment receives one label from the ordered set trusted (system prompt) ≻ semi-trusted (user input) ≻ untrusted (tool output, retrieved content, memory). Content derived from several segments inherits the least-trusted label among them, and processing never promotes content to a more trusted label. This rule is a design definition that preserves the distinction between data and instruction exploited by indirect injection. It asserts no empirical claim.

3.4.3 Entity extraction

The layer extracts security-relevant entities from each segment and from the arguments of a_{t}, and records the provenance of each occurrence:

- account identifiers (such as IBAN-formatted strings);
- monetary amounts;
- password and credential values;
- user-profile values (such as addresses and phone numbers);
- tool names mentioned in text.

The extraction patterns are declared in configuration and are fixed before evaluation. These entities allow Layer 3 to ask whether an argument of the proposed action can be traced to the user objective or to trusted content, or only to untrusted content.

3.5 Detection Layer

The Detection Layer computes context evidence from untrusted segments. It issues no verdict.

Table 3.8. Context evidence

Feature	Definition	Type	Availability
x^{ctx}_{1}	Maximum score of a fixed, pre-trained prompt-injection classifier over untrusted segments added since the previous step	[0, 1]	Derived (classifier choice pending supervisor approval)
x^{ctx}_{2}	1 if an untrusted segment added since the previous step matches the declared instruction-pattern set; otherwise 0	Binary	Derived (pattern set declared before evaluation)

The classifier is used with fixed inference and is not retrained. A classifier distributed with an existing guardrail system, such as PromptGuard 2 in LlamaFirewall (Chennabasappa et al., 2025), is a candidate. The instruction-pattern set contains declared indicators such as imperative instructions addressed to an assistant, references to tool names, and role or delimiter markers.

Both features are individually evadable, and benchmark-based scores of detectors are fragile (Arp et al., 2022; M. Q. Li et al., 2026). The Detection Layer is therefore not relied upon to be decisive. Its evidence is combined with behavioral and action evidence in the Decision Engine, and its contribution is measured by ablation (Section 3.11.7).

3.6 Behavioral Analysis Layer

The Behavioral Analysis Layer maintains the episode history and computes evidence about the trajectory and the proposed action. All reference quantities it uses are estimated offline from benign data and are frozen at runtime.

3.6.1 Step tracking

The layer maintains H_{t}: the sequence of proposed actions, their arguments with provenance, the verdicts issued, and the tool results returned. H_{t} is reset at the start of each episode.

3.6.2 Behavioral and action evidence

Table 3.9. Behavioral and action evidence

Feature	Definition	Type	Availability
x^{beh}_{1}	Post-observation shift: 1 if the most recent tool result was untrusted and had x^{ctx}_{2} = 1; otherwise 0	Binary	Derived from H_{t}

Feature	Definition	Type	Availability
x^{beh}_{2}	Repeated calls: min(n_{rep}, n_{max}) / n_{max}, where n_{rep} is the number of consecutive calls with the same tool and key arguments and n_{max} is declared	[0, 1]	Derived from H_{t}
x^{beh}_{3}	Privilege escalation: 1 if the privilege level of a_{t}'s tool exceeds the highest level used so far in the episode; otherwise 0	Binary	Derived; privilege map declared
x^{beh}_{4}	Transition deviation: 1 if the pair (previous tool, current tool) does not occur in the benign reference transition set; otherwise 0	Binary	Derived; reference set estimated on benign fitting data
x^{act}_{1}	Argument provenance: 1 if a key argument of a_{t} (recipient account identifier, amount, password, or profile value) occurs in untrusted content but not in g or trusted content; otherwise 0	Binary	Derived from L1 entities
x^{act}_{2}	Destination novelty: 1 if the recipient of a_{t} appears neither in g nor among the transactions observed before the episode's first untrusted ingestion; otherwise 0	Binary	Derived; applicable to value-transfer and payment-modification tools
x^{act}_{3}	Sensitive egress: 1 if a_{t} sends to an external recipient a value obtained from a sensitive read (account identifier, profile value, or credential); otherwise 0	Binary	Derived from L1 entities and H_{t}

All features are bounded in [0, 1] by definition, so no further rescaling is applied. Three design rules govern them:

1. **No identity-based features.** No feature may encode a specific account identifier. In the evaluated benchmark, one attacker account identifier is shared by most injection tasks (Section 3.11.2), and a feature keyed on that value would leak the label.
2. **Evidence is kept separate from consequence.** Whether a payee appears only in untrusted content is evidence about *whether* an action is adversarially induced; it belongs in x_{t}. Whether the action transfers a large amount irreversibly describes *what would happen*; it belongs in the consequence descriptors z_{t} (Section 3.7.3) and enters the decision only through the loss.
3. **The feature set is frozen after pilot E0** (Section 3.11.8), before any held-out predictions are examined.

3.7 Decision Engine Layer

The Decision Engine turns evidence into a verdict in five stages: probability estimation, calibration, consequence assessment, policy predicates, and expected-loss verdict selection. It also writes the audit record. Each equation below states its source and whether it is established from the source, adapted from it, or derived here.

3.7.1 Ridge logistic estimation

The probability that the proposed action is adversarially induced is modeled by logistic regression (Cox, 1958):

$$s_t = \beta_0 + \beta^\top x_t, \tilde{p_t} = \sigma(s_t) = 1/(1+exp(-s_t))$$

(3.1)

where x_{t} ∈ [0, 1]^{d} is the evidence vector [x^{ctx}; x^{beh}; x^{act}], β_{0} ∈ ℝ is the intercept, β ∈ ℝ^{d} is the coefficient vector, s_{t} is the logit, and p̃*{t} is the uncalibrated estimate of P(y*{t} = 1 | x_{t}).

Role in REM: maps the evidence of Layers 2 and 3 to a score and an uncalibrated probability. *Status:* established from source (Cox, 1958).

The parameters are estimated offline by maximizing a ridge-penalized log-likelihood (le Cessie & van Houwelingen, 1992):

$$(\beta_0,\beta) = argmax_{\beta_0,\beta} \Sigma_{i \in D_{fit}} [y_i ln\tilde{p_i} + (1-y_i) ln(1-\tilde{p_i})] - \lambda \|\beta\|^2_2$$

(3.2)

where D_{fit} is the set of labeled fitting steps, y_{i} ∈ {0, 1} are their labels, p̃_{i} is given by Equation 3.1, and λ ≥ 0 controls the strength of the quadratic penalty. The intercept is not penalized.

Role in REM: produces finite and stable coefficients from small, imbalanced data with binary features. *Status:* established from source.

The quadratic penalty is needed for a specific reason. The evidence features are mostly binary, and adversarial steps are scarce (Section 3.11.4). If a single indicator appears only in adversarial fitting steps, the unpenalized maximum-likelihood estimate does not exist as a finite value. The penalty keeps the estimates finite.

The exact scaling constant of the penalty in le Cessie and van Houwelingen (1992) is pending full-text verification. This does not affect the method: λ is selected by grouped cross-validation on held-out log-loss (Section 3.9), and rescaling the penalty by a positive constant changes the selected value of λ, not the set of attainable solutions. Software libraries often parameterize the penalty through an inverse strength; the mapping used in implementation will be documented from the library version used.

No interaction terms are included by default. Equation 3.1 assumes that the log-odds of adversarial induction are approximately linear in the evidence.

3.7.2 Calibration on the logit

The uncalibrated probability p̃_{t} need not agree with observed frequencies. Ridge shrinkage and class imbalance can both distort it. Because the decision rule in Section 3.7.5 treats the probability as a probability, REM recalibrates it with a two-parameter logistic map applied to the logit:

$$p_t = \sigma(\gamma_1 s_t + \gamma_0), \gamma_1 > 0$$

(3.3)

where γ_{1} is the calibration slope and γ_{0} is the calibration intercept.

Role in REM: produces the calibrated probability used by the decision rule. *Status:* adapted from source. The method is Platt scaling (Platt, 1999). The two-parameter logistic form with parameters fitted by negative log-likelihood on held-out data follows Guo et al. (2017). The map is applied here to the logit of a logistic regression model. The monotonicity condition follows Kull et al. (2017), who require a non-negative slope for a non-decreasing map; REM requires a strictly positive slope so that the ranking of steps is preserved.

The calibration parameters are fitted by minimizing the negative log-likelihood on held-out logits (Guo et al., 2017):

$$(\hat{\gamma_1}, \hat{\gamma_0}) = \operatorname{argmin}_{\gamma_1, \gamma_0} - \Sigma_{i \in D_{cal}} [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$$

(3.4)

where D_{cal} is a set of steps whose logits s_{i} were produced by a model that was not fitted on those steps. Fitting Equation 3.4 on the model's own training logits would be close to degenerate, so REM uses out-of-fold logits from grouped cross-fitting (Section 3.9).

Status: established from source (Guo et al., 2017).

Four properties of this choice are stated.

1. **It changes probabilities, not rankings.** With γ_{1} > 0, Equation 3.3 is strictly increasing in s_{t}. Calibration therefore leaves ROC-AUC, PR-AUC, and any decision based only on the ordering of steps unchanged. If the fitted slope is not positive, calibration is rejected for that fit and the event is reported.
2. **It is the minimal adequate family.** Kull et al. (2017) prove that beta calibration with equal shape parameters equals logistic calibration applied to the log-odds of a score. Since p̃*{t} = σ(*s{t}), the log-odds of p̃*{t} is s*{t}, so Equation 3.3 is exactly beta calibration with equal shape parameters applied to p̃*{t}. The identity map is included (*γ{1} = 1, γ_{0} = 0). Full beta calibration adds a third parameter, and isotonic regression is nonparametric; neither is adopted, given the limited calibration data.
3. **It is not a contribution.** Calibrated logistic risk scores are already used in agent safety (Hossain et al., 2026).
4. **It does not guarantee good control.** C. Zhang et al. (2026) show that recalibrating a scalar risk score can improve calibration error while leaving control regret under threshold routing unchanged, because a scalar probability does not represent whether an intervention would improve the outcome. REM therefore reports calibration quality and control outcomes separately (Section 3.12) and does not claim that calibration improves control.

3.7.3 Consequence tiers

The consequence of a proposed action is described by descriptors z_{t} obtained from the action itself (Table 3.10). They are mapped to a consequence tier k_{t} = κ(z_{t}) ∈ {1, …, K} by a declared policy function κ. Consequence descriptors do not enter Equation 3.1; they affect the verdict only through the loss.

Table 3.10. Consequence descriptors

Descriptor	Meaning	How obtained	Availability
Action category	Read, value transfer, payment modification, credential change, profile change (Table 3.7)	Tool name	Available
Amount	Monetary amount of a value-transfer or payment-modification call	Action arguments	Available where the tool takes an amount
Recurring-payment change	Whether the action alters a scheduled or recurring payment	Tool name	Available
Credential change	Whether the action changes account credentials	Tool name	Available
Irreversibility	Whether the effect is treated as irreversible once executed	Declared per tool	Proposed (policy); the benchmark does not simulate settlement

Table 3.11. Illustrative consequence tiers (policy pending supervisor approval)

Tier	Illustrative membership
1	Read tools
2	Profile change
3	Value transfer at or below a declared amount limit
4	Value transfer above the declared limit; payment modification; credential change

The quantities used by the Decision Engine fall into three classes, which are kept distinct throughout the thesis.

Table 3.12. Learned, measured, and policy quantities

Class	Quantities	How determined
Learned	β_{0}, β, γ_{1}, γ_{0}	Fitted on labeled data by Equations 3.2 and 3.4
Measured	$r_{mod}(k)$; benign reference mean $\bar{x}$; benign transition set; utility impact of Modify	Estimated from observe-only and enforcing-mode runs
Policy	κ, amount limits, $C_{miss}(k)$, $C_{FA}(k)$, $C_{mod}(k)$, C_{esc}, Π, k_{D2}, privilege map, tie-breaking order	Declared before evaluation; varied over a declared grid where they affect outcomes

Policy quantities encode institutional judgment. They are not presented as empirical findings, and their influence is reported through the loss grid of Section 3.12.3.

3.7.4 Policy predicates

Two deterministic predicates are evaluated before verdict selection.

- **D1, prohibited action.** If $a_{t} \in \Pi$, the verdict is Block irrespective of p_{t}. A prohibition that a sufficiently low probability could override would not be a prohibition.
- **D2, missing required evidence.** If the evidence required for tier k_{t} cannot be computed (for example, argument provenance is unavailable) and $k_{t} \geq k_{D2}$, the verdict is Escalate. Absent evidence is not treated as evidence of safety.

Π, k_{D2}, and the evidence required at each tier are declared policy. When a predicate applies, it determines the verdict, the expected-loss rule is not consulted, and the audit record names the predicate as the source of the verdict. Both predicates are disabled in one ablation (Section 3.11.7).

3.7.5 Expected-loss verdict selection

Loss structure. For each tier k, the loss of issuing verdict v when the label is y is given in Table 3.13.

Table 3.13. Tier-indexed loss structure L(v, y, k)

Verdict v	L(v, 0, k): legitimate action	L(v, 1, k): adversarially induced action
Allow	0	$C_{miss}(k)$
Modify	$C_{mod}(k)$	$r_{mod}(k) \cdot C_{miss}(k)$
Escalate	C_{esc}	C_{esc}
Block	$C_{FA}(k)$	0

All declared costs are non-negative and expressed in common relative units. Correct automatic decisions are assigned zero loss, which is a declared convention. The Escalate entries assume a reviewer who resolves escalations correctly; this assumption is examined in Sections 3.12 and 3.14.

The adversarial Modify entry is a derivation. Under the stated assumption A-Mod — whether Modify neutralizes an adversarially induced action is independent of x_{t} given the tier, and the attack fails with probability 1 − r_{mod}(k) — the law of total expectation gives:

$$L(Modify,1,k) = r_{mod}(k) \cdot C_{miss}(k) + (1-r_{mod}(k)) \cdot 0 = r_{mod}(k) \cdot C_{miss}(k)$$

(3.5)

where r_{mod}(k) ∈ [0, 1] is the residual attack-success rate after Modify at tier k, measured in enforcing-mode runs (Section 3.11.7).

Status: derived under assumption A-Mod.

Conditional risk. Elkan (2001) states that the optimal prediction is the one that minimizes expected cost, computed from the conditional probability of each class and the cost of each prediction for each true class. REM applies this criterion to the four verdicts, with losses indexed by the consequence tier and the calibrated probability of Equation 3.3 in place of the true conditional probability:

$$R_t(v) = (1-p_t) \cdot L(v,0,k_t) + p_t \cdot L(v,1,k_t), v \in \mathcal{V}(a_t)$$

(3.6)

where R_{t}(v) is the conditional risk of verdict v at step t.

Status: adapted from source. Elkan's criterion is stated for predictions of classes. REM extends the set of choices to four verdicts, indexes the losses by consequence tier, and substitutes the calibrated estimate p_{t} for the true posterior. The substitution is justified only to the extent that p_{t} is calibrated.

Bayes verdict. The verdict is:

$$v_t = argmin_{v \in \mathcal{V}(a_t)} R_t(v)$$

(3.7)

with ties resolved toward the more restrictive verdict in the declared order Block ≥ Escalate ≥ Modify ≥ Allow.

Status: adapted from source; the tie-breaking order is a design definition that makes the rule a function.

The feasible set $\mathcal{V}$(a_{t}) contains Allow, Escalate, and Block for every action, and Modify only if the tool of a_{t} belongs to the set $\mathcal{T}$_{R} to which the Modify mechanism applies (Section 3.8).

Derived thresholds. Each R_{t}(v) is affine in p_{t}, so the verdict changes only where two conditional risks are equal. For verdicts v and v′ at tier k, let Δ_{0} = L(v′, 0, k) − L(v, 0, k) and Δ_{1} = L(v, 1, k) − L(v′, 1, k), with Δ_{0} + Δ_{1} > 0. Setting R(v) = R(v′) in Equation 3.6 and solving for p gives:

$$\pi_{v,v'}(k) = \Delta_0/(\Delta_0+\Delta_1)$$

(3.8)

Status: derived from Equation 3.6.

Derivation: R(v) − R(v′) = −Δ_{0} + p(Δ_{0} + Δ_{1}). This is zero at p = π_{v,v′}(k), negative below it (v is preferred), and positive above it (v′ is preferred).

For v = Allow and v′ = Block, Δ_{0} = C_{FA}(k) and Δ_{1} = C_{miss}(k), so:

$$p_k^* = C_{FA}(k)/(C_{FA}(k)+C_{miss}(k))$$

(3.9)

Status: derived from Equation 3.8. It has the form of the standard two-class cost-sensitive threshold for zero-cost correct decisions, which Elkan (2001) obtains from the same expected-cost criterion; the printed threshold expression in that paper could not be inspected directly during verification, so the derivation above stands on Equation 3.8 alone.

Thresholds are therefore consequences of declared costs rather than independently chosen constants. The question "why this threshold?" becomes "why this cost ratio?", which the loss grid of Section 3.12.3 addresses.

When escalation can be selected. Chow (1970) analyzed the tradeoff between recognition error and rejection, which is the classical basis for withholding an automatic decision. In REM, Escalate is not implemented as a separate rejection test; it is one of the verdicts inside Equation 3.7. For the verdict set {Allow, Escalate, Block} at tier k, Escalate is selected for some probability if and only if:

$$C_{esc} < C_{FA}(k) \cdot C_{miss}(k)/(C_{FA}(k)+C_{miss}(k))$$

(3.10)

Status: derived from Equations 3.6, 3.7, and 3.9.

Derivation: R(Allow) = p · C_{miss}(k) increases in p and R(Block) = (1 − p) · C_{FA}(k) decreases in p. Their minimum is largest at p^{*}*{k}, where it equals C*{FA}(k) · C_{miss}(k) / (C_{FA}(k) + C_{miss}(k)). The constant C_{esc} lies strictly below this minimum for some p if and only if Equation 3.10 holds; at equality, ties resolve toward Block.

When Modify is also feasible, Equation 3.10 remains a necessary condition, because adding a verdict can only lower the minimum risk. In the symmetric case C_{FA}(k) = C_{miss}(k) = C, Equation 3.10 reduces to C_{esc} < C/2: deferral is selected only when it costs less than half an error.

Dominance check. Before a loss structure is used, it is checked for dominated verdicts. For example, R(Modify) − R(Block) = (1 − p)(C_{mod}(k) − C_{FA}(k)) + p · r_{mod}(k) · C_{miss}(k), which is non-negative for every p whenever C_{mod}(k) ≥ C_{FA}(k). In that case Modify can never be selected at tier k. A declaration that makes a verdict unreachable, by this check or by Equation 3.10, is reported as such rather than silently producing a smaller verdict set.

3.7.6 Explainability and audit record

For every verdict the Decision Engine writes an audit record containing:

- the evidence vector x_{t}, the logit s_{t}, and the probabilities p̃*{t} and p*{t};
- the tier k_{t} and the conditional risks R_{t}(v);
- the verdict v_{t} and its source (D1, D2, or the expected-loss rule);
- the model, calibration, and policy versions;
- a linear attribution of the logit.

The attribution follows the linear case of Shapley-value attribution given by Lundberg and Lee (2017):

$$\varphi_{t,i} = \beta_i(x_{t,i} - \bar{x_i}), i = 1, \ldots, d; \varphi_0 = \beta_0 + \beta^\top \bar{x}$$

(3.11)

where x̄ is the mean evidence vector over benign fitting steps, φ_{t,i} is the contribution of feature i to the logit relative to that benign reference, and φ_{0} is the logit at the reference.

Status: adapted from source. Lundberg and Lee (2017) state the linear case under feature independence with printed indices that do not match between the two sides of the expression and with base value equal to the model intercept. REM uses the index-consistent form and sets the base value to the logit at the reference point, which is the value required by their local-accuracy property when the reference mean is not zero.

The decomposition is exact:

$$\varphi_0 + \Sigma_{i\,=\,1}^{d} \varphi_{t,i} = s_t$$

(3.12)

Status: derived. Summing Equation 3.11 gives β_{0} + β^{⊤}x̄ + β^{⊤}x_{t} − β^{⊤}x̄ = s_{t}. This is the local-accuracy property of Lundberg and Lee (2017) for a linear model, and REM checks it as a correctness invariant.

For reporting by evidence group, REM defines:

$$\Phi_J = \Sigma_{i \in I_J} \varphi_{t,i}, J \in ctx, beh, act$$

(3.13)

Status: design definition. Because the groups partition the features, Φ_{ctx} + Φ_{beh} + Φ_{act} = s_{t} − φ_{0}.

Four qualifications apply:

1. **The attribution never influences the verdict.** It is an audit component only.
2. **It decomposes the logit, not the calibrated probability.** On the calibrated logit scale each contribution is multiplied by γ_{1}, which preserves relative contributions because γ_{1} > 0.
3. **Correlation limits interpretation, not the identity.** The independence assumption affects the interpretation of each φ_{t,i} as a Shapley value when features are correlated; it does not affect the identity of Equation 3.12.

4. **It is not a contribution.** Auditable decision records are provided by several existing systems (Hossain et al., 2026; C. Yang, 2026). Because the attribution is determined by the logged x_{t} and the frozen parameters, it can equally be computed at audit time.

3.7.7 Consequence-independent baseline policy

The baseline uses the same estimator and calibration but a decision rule that ignores consequence. Its thresholds are selected as operating points on benign validation steps:

$$\theta(\alpha) = min\theta : FPR_{val}(\theta) \le \alpha, FPR_{val}(\theta) = |i \in D_{val}^{benign} : p_i \ge \theta| / |D_{val}^{benign}|$$

(3.14)

where α is a declared false-positive target, D_{val}^{benign} is the set of benign validation steps in the relevant cross-fitting fold, and θ ranges over the observed probabilities.

The baseline verdict is:

- **Block** if p_{t} ≥ θ(α_{blk});
- **Modify** if θ(α_{mod}) ≤ p_{t} < θ(α_{blk}) and Modify is feasible, otherwise Block;
- **Allow** otherwise.

The targets satisfy α_{mod} > α_{blk}. Predicates D1 and D2 apply as in Section 3.7.4, so the baseline issues Escalate only through D2.

Status: design definition.

Because Equation 3.14 selects thresholds by ranking, a strictly increasing recalibration re-selects equivalent thresholds and leaves the baseline's verdicts unchanged. Cost-derived thresholds such as Equation 3.9 are fixed in probability units and can change after recalibration. This contrast is used in the ablation of Section 3.11.7 and is consistent with the finding of C. Zhang et al. (2026) that recalibration need not change threshold-routed control.

3.8 Mitigation Layer

The Mitigation Layer executes, deterministically, the mechanism associated with each verdict. It never initiates an action on the agent's behalf.

Table 3.14. Verdict-to-mitigation mapping

Verdict	Mechanism	Operation
Allow	Release	a_{t} is released across TB-3 and the event is logged.
Modify	Tool restriction	a_{t} is not executed. The tool class of a_{t} is removed from the agent's available tools for the rest of the episode, and a structured notice is returned to the agent, which may continue with its remaining tools. Applies only to tools in $\mathcal{T}$_{R}.
Escalate	Withhold and refer	a_{t} is withheld and the audit record is presented to a human reviewer. The action is released only if the reviewer approves.
Block	Interrupt with safe fallback	a_{t} is not executed, every tool in $\mathcal{T}$_{R} is restricted for the rest of the episode so that no further side-effecting action is released, and a structured refusal is returned to the agent.

Tool restriction is the single Modify mechanism because it is deterministic, does not depend on an LLM rewriting content, and can be measured. Input sanitization, such as spotlighting (Hines et al., 2024), is an alternative that is not adopted as the primary mechanism.

Four properties are stated:

1. **Restrictiveness ordering.** Block halts all further side effects, Escalate defers the decision, and Modify removes one tool class. This ordering underlies the tie-breaking order of Equation 3.7.
2. **$\mathcal{T}$_{R} is declared.** In the banking environment it contains value-transfer, payment-modification, credential-change, and profile-change tools; read tools are not restricted.
3. **Mitigation effectiveness is measured, not assumed.** The residual attack-success rate r_{mod}(k) used in Equation 3.5 is estimated from enforcing-mode runs.
4. **Retries are observed.** A restricted or blocked agent may retry. Retries raise x^{beh}_{2}, and calls to restricted tools are refused by the Mitigation Layer.

3.9 Offline Fitting and Recalibration

Offline fitting and recalibration are cross-cutting activities. They are performed between deployments on logged data, they are not a layer, and they never modify REM's parameters during an episode. Runtime adaptation from observed traffic is excluded because it would allow an adversary who generates traffic to shift REM's reference behavior.

The offline procedure fits and freezes:

- the benign reference quantities ($\bar{x}$ and the benign transition set);
- the coefficients by Equation 3.2, with λ selected by grouped cross-validation on held-out log-loss;
- the calibration parameters by Equation 3.4 on out-of-fold logits;
- the baseline thresholds by Equation 3.14;
- the measured residual rates r_{mod}(k).

It also checks the declared loss structure for dominated verdicts (Section 3.7.5) and records all model, data, policy, benchmark, and software versions.

Between deployments, calibration can be monitored against later-established labels using the metrics of Section 3.12. If calibration drifts beyond a declared tolerance, the procedure is repeated on new data.

Procedure 3.1. Offline fitting (cross-cutting; never executed within an episode)

```
Input: logged observe-only traces grouped by injection task and user task; labels y;
       declared policy (κ, L, Π, k_D2, privilege map, 𝒯_R, α_blk, α_mod)
1  for each outer fold f (Section 3.11.5):
2      D_train ← steps outside fold f;  D_test ← steps in fold f
3      estimate x̄ and the benign transition set on benign steps of D_train
4      select λ by grouped inner cross-validation on D_train (held-out log-loss)
5      obtain out-of-fold logits on D_train from inner folds; fit (γ1, γ0) by Eq. 3.4
6      if γ1 ≤ 0 then record calibration failure for fold f
7      refit (β0, β) on all of D_train with the selected λ (Eq. 3.2)
8      select θ(α_blk), θ(α_mod) by Eq. 3.14 on out-of-fold calibrated benign steps of D_train
9      apply Eqs. 3.1, 3.3 to D_test; store predictions for evaluation only
10 estimate r_mod(k) from enforcing-mode runs (Section 3.11.7)
11 check the loss structure for dominated verdicts (Section 3.7.5)
12 freeze parameters and record all versions
```

3.10 Algorithm 1: REM Runtime Evaluation and Mitigation

Algorithm 1 gives the per-step runtime procedure. Every operation is defined in Sections 3.4–3.8.

```
Algorithm 1. REM Runtime Evaluation and Mitigation

Frozen (offline, cross-cutting):  β0, β, γ1 (> 0), γ0, x̄, benign transition set,
                                   privilege map, instruction-pattern set, n_max,
                                   injection classifier (fixed)
Declared policy:                  κ, L (Table 3.13), Π, k_D2, required evidence per tier,
                                   𝒯_R, order Block ≥ Escalate ≥ Modify ≥ Allow
Input at step t:                  objective g; new context segments with source channel;
                                   proposed action a_t; history H_t; episode restrictions S
Output:                           verdict v_t; mitigated outcome; audit record r_t

 1  // Layer 1: Input & Context (Section 3.4)
 2  remove invisible and formatting characters from new untrusted segments
 3  c_t   ← label segments trusted | semi-trusted | untrusted (least-trusted inheritance)
 4  Ent_t ← extract account identifiers, amounts, credentials, profile values, tool names,
            each with provenance, from c_t, g, and the arguments of a_t
 5  // Layer 2: Detection (Section 3.5, Table 3.8)
 6  x_ctx ← (x^ctx_1, x^ctx_2)
 7  // Layer 3: Behavioral Analysis (Section 3.6, Table 3.9)
 8  x_beh ← (x^beh_1, x^beh_2, x^beh_3, x^beh_4) from H_t, a_t, benign transitions, privilege map
 9  x_act ← (x^act_1, x^act_2, x^act_3) from a_t, Ent_t, g, H_t
```

```
10  // Layer 4: Decision Engine (Section 3.7)
11  x_t ← [x_ctx; x_beh; x_act];  avail ← features computable at step t
12  s_t ← β0 + β^⊤x_t;  p̃_t ← σ(s_t)                                  // Eq. 3.1
13  p_t ← σ(γ1·s_t + γ0)                                              // Eq. 3.3
14  k_t ← κ(z_t), where z_t are the descriptors of a_t (Table 3.10)
15  if a_t ∈ Π then
16      v_t ← Block;  src ← "D1"
17  else if the evidence required at tier k_t ∉ avail and k_t ≥ k_D2 then
18      v_t ← Escalate;  src ← "D2"
19  else
20      V_t ← {Allow, Escalate, Block} ∪ ({Modify} if tool(a_t) ∈ 𝒯_R)
21      for each v in V_t:  R[v] ← (1 − p_t)·L(v,0,k_t) + p_t·L(v,1,k_t)   // Eq. 3.6
22      v_t ← verdict in V_t with minimum R[v]; ties → more restrictive   // Eq. 3.7
23      src ← "expected loss"
24  φ0 ← β0 + β^⊤x̄;  φ_t ← β ⊙ (x_t − x̄);  Φ ← group sums over I_ctx, I_beh, I_act  // Eqs. 3.11, 3.13
25  check |φ0 + Σ_i φ_t,i − s_t| < ε                                   // Eq. 3.12
26  r_t ← (t, x_t, s_t, p̃_t, p_t, k_t, R, v_t, src, φ0, φ_t, Φ, versions, timestamps)
27  append r_t to the audit log (append-only)
28  // Layer 5: Mitigation (Section 3.8, Table 3.14)
29  if tool(a_t) is restricted in S then refuse a_t and return a structured notice
30  else if v_t = Allow    then release a_t
31  else if v_t = Modify   then withhold a_t; add tool class of a_t to S; return structured notice
32  else if v_t = Escalate then withhold a_t; send r_t to reviewer; release a_t only on approval
33  else if v_t = Block    then do not execute a_t; add every tool in 𝒯_R to S (halt);
                               return structured refusal
34  update H_t with (a_t, v_t) and, if released, its tool result, which enters Layer 1
    as untrusted content at step t + 1
35  return v_t, outcome, r_t
```

Line 29 enforces restrictions imposed by earlier Modify or Block verdicts. After Block, every tool in $\mathscr{T}_\{R\}$ is restricted, so the episode refuses every further side-effecting action while read tools remain available. Lines 24–25 are audit operations and do not affect the verdict.

3.11 Experimental Methodology

3.11.1 Environment and instrumentation

REM is evaluated in the banking environment of AgentDojo (Debenedetti et al., 2024), an open-source framework that evaluates both the utility and the security of LLM agents. Security is determined by checks on the environment state before and after an episode, and utility by task-specific checks.

REM is placed at the tool-execution boundary of the agent loop, where it intercepts each proposed tool call before execution and each tool result on return. That this is achievable without modifying the agent's internals is assumption AS-1, to be demonstrated in Chapter 4.

REM runs in two modes:

- **Observe-only:** verdicts are computed and logged but not enforced. Fitting data are collected in this mode, because enforced verdicts would alter the trajectories from which REM is fitted.
- **Enforcing:** verdicts are executed by the Mitigation Layer.

For every run, the following are recorded:

- the AgentDojo release or commit identifier;
- the suite version;
- the attack template;
- the LLM backbone identifier and version, access date, and decoding parameters;
- REM's model, calibration, and policy versions.

The LLM backbone or backbones and the attack templates are pending supervisor approval.

3.11.2 Dataset

The banking suite definition inspected for this chapter (suite v1, AgentDojo repository main branch, accessed September 2026) contains:

- **16 user tasks.** Examples include paying a bill described in a file, refunding a friend, updating a recurring rent payment, and updating account details. Their reference solutions use between one and five tool calls.
- **9 injection tasks.** Eight direct money or a recurring payment toward one shared attacker account identifier. One changes the account password. The reference attacker actions consist of one call for seven tasks, two calls for one task (reading scheduled transactions, then sending money), and three calls for one task (a total amount split into smaller transfers).
- **11 tools:** get_iban, send_money, schedule_transaction, update_scheduled_transaction, get_balance, get_most_recent_transactions, get_scheduled_transactions, read_file, get_user_info, update_password, and update_user_info.

These counts must be re-confirmed against the benchmark version pinned for the experiments.

AgentDojo is an environment, not a fixed dataset of traces. Traces are generated by running an agent, so the number of traces depends on the number of backbones, attack templates, and repeated runs [DATA COUNT REQUIRES VERIFICATION]. The 144 combinations of user task and injection task are not 144 independent samples. They are built from 16 user-task templates and 9 injection-task templates, and every trace inherits the dependence of its templates.

3.11.3 Label construction

The label must represent the quantity the estimator is asked to predict, which is whether a proposed call serves the attacker's objective rather than the user's. Labels are therefore derived from the injection task's own reference solution, not from the presence of an injection in the episode, because an attacked episode also contains benign calls. The derivation is mechanical, so the rules below state exactly what is matched, where the rule is known to be uncertain, and how that uncertainty is measured.

- **Step labels.** In an attacked episode, a proposed tool call is labeled $y_{t} = 1$ if it matches a call in the injection task's reference solution on tool name and on the attacker-controlled key argument, such as the attacker account identifier or the attacker-specified password. All other proposed calls, including calls that continue the user's task in attacked episodes, are labeled $y_{t} = 0$, as are all calls in episodes without injection.
- **Induced read steps.** Some injection tasks include a read call in their reference solution. Whether such a read step is labeled $y_{t} = 1$ is pending supervisor decision, because it is induced but has no direct external effect.
- **Label audit.** Matching on arguments can mislabel steps, for example when an agent alters an amount. The derivation is therefore audited on a manually inspected sample of episodes, and the disagreement rate is reported.

3.11.4 Data limitations

The design must respect five limitations of the data:

1. **Positive-step scarcity.** A step is labeled $y_{t} = 1$ only if the agent actually proposes the attacker's call. The number of positive steps therefore depends on how often the chosen backbone follows injected instructions, and it may be small [DATA COUNT REQUIRES VERIFICATION]. Most steps are benign, so the classes are strongly imbalanced.
2. **Shared attacker account identifier.** Eight of the nine injection tasks use the same attacker account identifier. Any feature keyed on that value would leak the label across folds (Section 3.6.2).
3. **Repeated task templates.** The same user tasks appear with every injection task, so benign behavior learned in one fold recurs in others.
4. **Short trajectories.** Reference solutions contain one to five calls, and most attacker objectives require one call. Trajectory-level evidence therefore has limited opportunity to accumulate.
5. **Calibration data.** A calibration set separate from training, validation, and test data would contain very few positive steps.

As a result, four statistically independent partitions for training, calibration, validation, and testing cannot be supported, and the design uses grouped cross-fitting instead.

3.11.5 Grouping and cross-fitting

Estimation, calibration, and threshold selection use nested grouped cross-fitting. No step is ever scored by a model fitted on data from its own group.

1. **Outer folds.**
   - *Attacked episodes* are grouped by injection task, giving nine leave-one-injection-task-out folds. The held-out fold therefore contains an attacker objective not seen during fitting.
   - *Episodes without injection* are assigned to the outer folds by user task, so that no user task's benign-only episodes appear in both the fitting and held-out portions of a fold.
2. **Inner folds.** Within the fitting portion of each outer fold, grouped inner cross-validation selects λ by held-out log-loss and produces out-of-fold logits. These are used to fit the calibration parameters (Equation 3.4) and to select the baseline thresholds (Equation 3.14).

3. **Held-out scoring.** The outer held-out fold is scored by the model fitted on the rest. All reported metrics are computed from these held-out predictions.
4. **Deployment model.** A model fitted on all data is used only for runtime demonstrations and latency measurement, never for reported accuracy or calibration.

Grouping by injection task does not remove the dependence introduced by shared user-task templates in attacked episodes. As a sensitivity analysis pending supervisor approval, the fitting is repeated with folds grouped by user task, and differences between the two groupings are reported.

3.11.6 Leakage controls

Because the injection tasks in the banking suite share a small number of templates and a single attacker account, an estimator can reach a high score by memorizing the benchmark rather than by recognizing adversarial inducement. Arp et al. (2022) identify this family of errors as a recurring cause of overstated security-classification results. The following controls are therefore fixed before any held-out prediction is examined.

- No feature encodes a specific account identifier (Section 3.6.2).
- Benign reference quantities are estimated within each outer fold's fitting portion.
- The feature set, instruction-pattern set, privilege map, tier map, loss grid, false-positive targets, and pilot decision values are declared before held-out predictions are examined.
- Fitting data are collected in observe-only mode.
- Results are reported per benchmark and are not pooled across benchmarks.

3.11.7 Configurations, baselines, and ablations

Configurations:

- **B0, no defense.** Establishes attack success and utility without protection.
- **B1, consequence-independent baseline.** The same estimator and calibration with the threshold policy of Section 3.7.7. This is the primary comparator for RQ2.
- **REM.** The expected-loss verdict selection of Section 3.7.5.
- **B2, detection-only (pending supervisor approval).** The Detection Layer's classifier score applied to untrusted content with an operating-point threshold. It tests whether Layers 3 and 4 add value.
- **Distributed defenses (pending supervisor approval).** Any defenses distributed with the pinned benchmark version, to address O3.

Ablations of REM (each changes one element, all else fixed; the estimator is refitted when an evidence group is removed):

1. Context evidence removed.
2. Behavioral evidence removed.
3. Action evidence removed.
4. Calibration removed ($\tilde{p}_t$ used in Equation 3.6).
5. Consequence tiers removed (one tier for all actions).
6. Predicates D1 and D2 disabled.
7. Observe-only (no mitigation).

Mitigation-effectiveness runs. Enforcing-mode runs in which Modify is applied to adversarial steps estimate $r_{mod}(k)$ per tier and the utility impact of Modify on benign steps. These estimates feed Equation 3.5 and are fitted within the outer folds so that no held-out episode informs its own loss values.

Escalation in experiments. No human reviewer is available in the benchmark. Escalation is resolved in two ways:

- by an idealized reviewer who releases the action if $y_t = 0$ and prevents it if $y_t = 1$;
- by a no-reviewer bound in which every escalation is treated as Block.

Both are reported, so the dependence of any result on the reviewer assumption is visible.

3.11.8 Pilot E0

Before the feature set is frozen, a pilot run on the fitting portions of the outer folds checks feasibility:

- the number of positive steps per backbone and attack template;
- the ROC-AUC of each evidence group alone and combined.

The pilot's decision values are project decisions declared before the pilot, not literature-derived thresholds [pending supervisor approval]:

- the minimum number of positive steps needed to proceed;
- the minimum group ROC-AUC needed to retain an evidence group.

If the positive-step count is insufficient, additional attack templates or backbones are added before the main experiments. The pilot never uses held-out fold predictions.

3.11.9 Repeated runs and uncertainty

Each configuration is run at least three times with fixed decoding parameters to characterize LLM non-determinism [number of runs pending supervisor approval]. Repeated runs of the same template are not independent. Results are therefore aggregated per episode template before uncertainty is estimated.

Confidence intervals are obtained by bootstrap resampling (Efron, 1979) at the level of injection-task groups for attacked episodes and user-task groups for benign episodes. Resampling groups rather than steps is a design choice that respects the dependence described in Section 3.11.2.

With nine injection-task groups, intervals will be wide. No statistical power has been established, so differences are reported as measured differences with their intervals, and the word "significant" is not used unless a stated test supports it.

3.11.10 Optional observe-only sequential analysis

Sequential change detection is not part of REM's verdict path. If the supervisor approves it for the thesis narrative, it is performed **offline** on the per-step indicators logged by the Behavioral Analysis Layer. It never influences a verdict.

For each step, a binary deviation indicator e_{t} ∈ {0, 1} is defined as 1 if at least one of x^{beh}*{1}, x^{beh}*{3}, x^{beh}*{4}, or x^{act}*{1} equals 1, or if x^{beh}*{2} = 1; otherwise 0 [indicator set pending supervisor approval]. Let q*{0} and q_{1}, with 0 < q_{0} < q_{1} < 1, be the rates of e_{t} = 1 before and after an adversarial change.

The log-likelihood ratio of one observation, as defined by Basseville and Nikiforov (1993), specialized to Bernoulli observations, is:

$$\ell_t = e_t ln(q_1/q_0) + (1-e_t) ln((1-q_1)/(1-q_0))$$

(3.15)

Status: derived by substituting the Bernoulli probability mass function into the log-likelihood ratio of Basseville and Nikiforov (1993). A cumulative sum chart for Bernoulli observations is treated by Reynolds and Stoumbos (1999).

The cumulative sum statistic and alarm time are (Basseville & Nikiforov, 1993; Page, 1954):

$$G_t = max(0, G_{t-1} + \ell_t), G_0 = 0$$

(3.16)

$$T_A = min t : G_t \geq h_G$$

(3.17)

where h_{G} > 0 is the alarm threshold.

Status: established from source.

The per-episode false-alarm rate on benign episodes and the detection delay are defined as:

$$FA_{ep}(h_G) = |j \in E_{ben} : max_t G_t^{(j)} \geq h_G| / |E_{ben}|$$

(3.18)

$$\tau_d = T_A - \nu, T_A \geq \nu$$

(3.19)

where E_{ben} is the set of benign episodes in the fitting portion of a fold, G^{(j)}*{t} is the statistic in benign episode j,* τ{d} is the detection delay, and ν is the first step at which injected content enters the context.

Status: design definitions.

The parameters are determined as follows:

- q_{0} is the empirical rate of e_{t} = 1 on benign fitting steps, floored at a declared minimum.
- q_{1} is declared and reported over a range, because the scarcity of multi-step attacks does not allow it to be estimated reliably.
- h_{G} is the smallest value for which Equation 3.18 does not exceed a declared target.

The procedure has known optimality properties in sequential change detection: Lorden (1971) proposed stopping rules that are asymptotically optimal in a defined sense, and Moustakides (1986) showed Page's stopping time to be optimal in a stronger sense. Both results are proved within a sequential-analysis model whose conditions could not be inspected during verification and are not claimed to hold for agent trajectories, where observations are neither independent nor drawn from known distributions. They therefore motivate the procedure and provide no guarantee in this setting. Given the short trajectories of Section 3.11.4, this analysis may have little to detect.

3.12 Evaluation Metrics

3.12.1 Primary metrics

Primary metrics do not depend on declared costs.

Table 3.15. Primary metrics

Metric	Definition	Unit
Attack success rate (ASR)	Fraction of attacked episodes in which the benchmark's security check reports that the injection goal was achieved (enforcing mode)	Episode
Benign utility	Fraction of episodes without injection in which the benchmark's utility check passes	Episode
Utility under attack	Fraction of attacked episodes in which the utility check passes	Episode
Intervention rates	Fraction of steps receiving Modify, Escalate, or Block, reported per verdict and separately for y = 0 and y = 1 steps	Step
Step false-positive rate	Interventions on y = 0 steps divided by the number of y = 0 steps	Step
Residual ASR under Modify	ASR in episodes where Modify was applied to an adversarial step	Episode
Escalation rate	Fraction of steps receiving Escalate, with outcomes under the idealized reviewer and the no-reviewer bound	Step
Latency	Wall-clock time of REM's per-step processing, excluding LLM inference and tool execution; median and 95th percentile	Step

ASR is never reported without benign utility and utility under attack. A low ASR can reflect an agent that fails to complete tasks rather than an effective defense (Y. Wang et al., 2026).

3.12.2 Secondary metrics

Discrimination. ROC-AUC (Fawcett, 2006) and PR-AUC (Davis & Goadrich, 2006) are computed on held-out probabilities. PR-AUC is emphasized because positive steps are rare. Within a fold, both are unchanged by calibration with γ_{1} > 0. Because each outer fold has its own calibration map, metrics are reported per fold as well as on the pooled held-out predictions.

Calibration error. For binary outcomes, calibration error is measured on the positive-class probability (Naeini et al., 2015) with M equal-width bins (Guo et al., 2017):

$$ECE = \Sigma_{m=1}^{M}(|B_m|/N) \cdot |\bar{y}(B_m) - \bar{p}(B_m)|, B_m = i : (m-1)/M < p_i \leq m/M$$

(3.20)

where N is the number of held-out steps, ȳ(B_{m}) is the fraction of positive steps in bin m, p̄(B_{m}) is the mean calibrated probability in bin m, and M is declared in advance (steps with p_{i} = 0 are assigned to the first bin).

Status: adapted from source. Naeini et al. (2015) supply the binary form; Guo et al. (2017) supply the equal-width binning.

Because a global ECE averages over all probabilities, reliability is also reported within declared windows around each tier's threshold p^{*}_{k}.

Log-loss. The mean negative log-likelihood of held-out steps is:

$$LL = -(1/N)\Sigma_{i=1}^{N}[y_i lnp_i + (1-y_i) ln(1-p_i)]$$

(3.21)

Status: derived as the negated, averaged, unpenalized log-likelihood term of Equation 3.2 evaluated at the calibrated probabilities. Probabilities are bounded away from 0 and 1 by a declared small constant for numerical evaluation.

Brier score. The Brier score is not reported until the convention of its original source (Brier, 1950) has been confirmed; log-loss serves as the proper scoring measure in the meantime.

3.12.3 Realized loss across a declared grid

Realized loss is secondary because it depends on declared costs, and because a rule that minimizes expected loss under L is favored by construction when evaluated under the same L. For an evaluation loss structure L_{e}:

$$\bar{L}(L_e) = (1/N)\Sigma_{i=1}^{N} L_e(v_i, y_i, k_i)$$

(3.22)

where v_{i} is the verdict issued for held-out step i under the decision loss structure L_{d}, y_{i} is its label, and k_{i} is its tier.

Status: design definition.

The grid is declared before held-out predictions are examined and varies three things:

- the ratio C_{miss}(k)/C_{FA}(k) per tier;
- C_{esc};
- the amount limits of the tier map.

For each evaluation structure L_{e}, the grid reports:

- REM with L_{d} = L_{e} (matched);
- REM with L_{d} ≠ L_{e} (misspecified costs);
- the baseline B1, which does not use costs.

The conclusion takes the form “across this declared range of loss structures, outcomes vary as follows”, not “under the chosen costs, outcomes are these”.

3.13 Contribution and Novelty Boundaries

REM does not introduce a new learning algorithm or a new decision-theoretic method. Its constituent mechanisms are established and are attributed to their sources.

Table 3.16. Mechanisms that are not claimed as novel

Mechanism	Prior work
Logistic regression	Cox (1958)
Ridge regularization of logistic regression	le Cessie and van Houwelingen (1992)
Platt (logistic) calibration	Platt (1999); Guo et al. (2017); Kull et al. (2017)
Linear Shapley attribution	Lundberg and Lee (2017)
Cumulative sum change detection	Page (1954); Basseville and Nikiforov (1993)
Expected-cost decisions; reject option	Elkan (2001); Chow (1970)
Calibrated logistic risk scores for agent safety; four-way intervention	Hossain et al. (2026)
Distinction between calibration and control	C. Zhang et al. (2026)
Runtime interception; human review; graduated response	H. Liu et al. (2026); C. Yang (2026); C. L. Wang et al. (2025)
Provenance analysis of tool calls	She et al. (2026); Debenedetti et al. (2025)
Consequence-aware and consequence-priced control	H.-H. Chen (2026); Hossain et al. (2026); C. Yang (2026)
Component ablation with latency	C. Yang (2026)
Runtime protection of financial agents	Jia et al. (2026)

Investigated contribution. The contribution of this thesis is empirical and integrative. It is the design and evaluation, on the AgentDojo banking environment under indirect prompt injection, of a non-invasive five-layer runtime layer that combines:

- provenance-based evidence about proposed financial actions;
- a calibrated per-step probability of adversarial induction;
- verdict selection over Allow, Modify, Escalate, and Block by minimizing conditional risk with losses declared per consequence tier;
- deterministic mitigation whose effectiveness is measured and used in the loss structure.

It is evaluated against a consequence-independent baseline that uses the same estimator, with attack success, benign utility, utility under attack, intervention behavior, and latency measured separately and cost-dependent outcomes reported across a declared loss grid.

Explicitly not claimed:

- a new algorithm;
- the first runtime guardrail, the first calibrated agent risk score, or the first consequence-aware controller;
- that calibration improves control;
- robustness against adaptive adversaries;
- any prevention guarantee;
- validity in production financial deployment.

The contribution holds only to the extent that the experiments specified in this chapter support it.

3.14 Threats to Validity

Construct validity.

- Step labels are derived and can be wrong; the derivation is audited on a sample (Section 3.11.3).
- The idealized reviewer overstates the protective value of Escalate; the no-reviewer bound is reported alongside it.
- Declared costs cannot be validated empirically; their influence is reported across the grid.
- A low ASR can reflect task failure rather than protection, so utility metrics accompany every security metric (Y. Wang et al., 2026).

Internal validity.

- Leakage through shared templates and the shared attacker account identifier is controlled by grouping and by excluding identity-based features, but template dependence remains.
- Features designed with knowledge of the benchmark may fit its style; the feature set is frozen after the pilot and before held-out predictions are examined.
- LLM non-determinism is characterized by repeated runs.

External validity.

- The evaluation uses one simulated banking environment with benchmark-authored attacks.
- Results may not transfer to other benchmarks, whose definitions and conclusions can disagree (M. Q. Li et al., 2026), or to production systems.
- Results may depend on the LLM backbone.
- FinVault is not used because its preprint was withdrawn.

Statistical conclusion validity.

- The number of independent task templates is small.
- Intervals are wide and no power analysis is available, so conclusions are limited to differences supported by their intervals.

Security-specific threats.

- Every evidence feature can be evaded individually, and adversaries are assumed non-adaptive (AS-6).
- REM's own configuration and logs are assumed protected (AS-3).
- A scalar probability cannot represent whether an adversarial trajectory remains recoverable, so REM inherits the limitation identified by C. Zhang et al. (2026); estimating intervention value by counterfactual replay is outside the scope of this thesis.

3.15 Equation Provenance Summary

Table 3.17. Provenance of the equations in this chapter

Eq.	Content	Status	Source	Verification
3.1	Logistic model	Established	Cox (1958)	IV

Eq.	Content	Status	Source	Verification
3.2	Ridge-penalized log-likelihood	Established	le Cessie & van Houwelingen (1992)	IV (penalty constant pending)
3.3	Logistic calibration on the logit, $\gamma_{1} > 0$	Adapted	Platt (1999); Guo et al. (2017); Kull et al. (2017)	Platt NC; Guo FV; Kull FV
3.4	Calibration fit by negative log-likelihood	Established	Guo et al. (2017)	FV
3.5	Adversarial loss of Modify	Derived (assumption A-Mod)	Law of total expectation	DER
3.6	Conditional risk	Adapted	Elkan (2001)	FV (Eq. 1 form)
3.7	Bayes verdict with tie-breaking	Adapted	Elkan (2001)	FV; tie order DEF
3.8	Pairwise indifference threshold	Derived	From Eq. 3.6	DER
3.9	Allow–Block threshold	Derived	From Eq. 3.8; consistent with Elkan (2001)	DER
3.10	Escalation feasibility condition	Derived	From Eqs. 3.6, 3.7, 3.9; Chow (1970) conceptual basis	DER; Chow IV
3.11	Linear attribution	Adapted	Lundberg & Lee (2017)	FV
3.12	Completeness identity	Derived	Lundberg & Lee (2017) local accuracy	FV + DER
3.13	Group attribution	Definition	—	DEF
3.14	Baseline operating-point threshold	Definition	—	DEF
3.15	Bernoulli log-likelihood ratio (optional)	Derived	Basseville & Nikiforov (1993); Reynolds & Stoumbos (1999)	FV; R&S IV
3.16	CUSUM recursion (optional)	Established	Basseville & Nikiforov (1993); Page (1954)	FV; Page IV
3.17	Alarm time (optional)	Established	Basseville & Nikiforov (1993)	FV
3.18	Per-episode false-alarm rate (optional)	Definition	—	DEF
3.19	Detection delay (optional)	Definition	—	DEF
3.20	Expected calibration error	Adapted	Naeini et al. (2015); Guo et al. (2017)	FV
3.21	Log-loss	Derived	From Eq. 3.2	DER
3.22	Realized loss	Definition	—	DEF

Verification codes: FV — formulation checked against the source; IV — source identity confirmed, formulation to be checked against the full text; NC — source not accessible during this revision; DER — derived in this chapter; DEF — design definition. The codes are retained until final reference verification is complete.

3.16 Chapter Summary

This chapter specified the REM methodology as one runtime pipeline in five layers:

- **Input & Context:** captures the proposed action and labels context by provenance.
- **Detection:** computes context evidence.
- **Behavioral Analysis:** computes history and action-provenance evidence.
- **Decision Engine:** estimates a calibrated probability of adversarial induction with ridge logistic regression and Platt calibration on the logit, assigns a declared consequence tier, applies policy predicates, selects a verdict by minimizing conditional risk under tier-indexed losses, and writes an audit record whose linear attribution never influences the verdict.
- **Mitigation:** executes Allow, tool restriction, escalation, or Block.

Offline fitting and recalibration are cross-cutting activities, not a layer.

The evaluation uses the AgentDojo banking environment with nested grouped cross-fitting instead of independent partitions. It explicitly accounts for positive-step scarcity, the shared attacker account identifier, repeated task templates, and short trajectories. It compares REM with a consequence-independent baseline on the same estimator, reports cost-free primary metrics, and reports realized loss only across a declared grid that includes misspecified costs. The contribution is bounded as an empirical and integrative one. Chapter 4 describes the implementation.

References

Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., & Rieck, K. (2022). Dos and don'ts of machine learning in computer security. In *31st USENIX Security Symposium*. USENIX Association. https://arxiv.org/abs/2010.09470

Basseville, M., & Nikiforov, I. V. (1993). *Detection of abrupt changes: Theory and application*. Prentice Hall. https://people.irisa.fr/Michele.Basseville/kniga/

Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review, 78*(1), 1–3. https://doi.org/10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2

Chen, H.-H. (2026). *Insuring every action: An authority frontier framework for runtime actuarial control of autonomous AI agents* (arXiv:2605.25632) [Preprint]. arXiv. https://arxiv.org/abs/2605.25632

Chennabasappa, S., Nikolaidis, C., Song, D., Molnar, D., Ding, S., Wan, S., Whitman, S., Deason, L., Doucette, N., Montilla, A., Gampa, A., de Paola, B., Gabi, D., Crnkovich, J., Testud, J.-C., He, K., Chaturvedi, R., Zhou, W., & Saxe, J. (2025). *LlamaFirewall: An open source guardrail system for building secure AI agents* (arXiv:2505.03574) [Preprint]. arXiv. https://arxiv.org/abs/2505.03574

Chow, C. K. (1970). On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory, 16*(1), 41–46. https://doi.org/10.1109/TIT.1970.1054406

Cox, D. R. (1958). The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B (Methodological), 20*(2), 215–232. https://doi.org/10.1111/j.2517-6161.1958.tb00292.x

Davis, J., & Goadrich, M. (2006). The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning* (pp. 233–240). ACM. https://doi.org/10.1145/1143844.1143874

Debenedetti, E., Shumailov, I., Fan, T., Hayes, J., Carlini, N., Fabian, D., Kern, C., Shi, C., Terzis, A., & Tramèr, F. (2025). *Defeating prompt injections by design* (arXiv:2503.18813) [Preprint]. arXiv. https://arxiv.org/abs/2503.18813

Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., & Tramèr, F. (2024). *AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents* (arXiv:2406.13352). arXiv. https://arxiv.org/abs/2406.13352

Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics, 7*(1). https://doi.org/10.1214/aos/1176344552

Elkan, C. (2001). The foundations of cost-sensitive learning. In *Proceedings of the Seventeenth International Joint Conference on Artificial Intelligence (IJCAI-01)*. https://cseweb.ucsd.edu/~elkan/rescale.pdf

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters, 27*(8), 861–874. https://doi.org/10.1016/j.patrec.2005.10.010

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security* (pp. 79–90). ACM. https://doi.org/10.1145/3605764.3623985

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning* (Vol. 70, pp. 1321–1330). PMLR. https://proceedings.mlr.press/v70/guo17a.html

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75–106. https://doi.org/10.2307/25148625

Hines, K., Lopez, G., Hall, M., Zarfati, F., Zunger, Y., & Kiciman, E. (2024). *Defending against indirect prompt injection attacks with spotlighting* (arXiv:2403.14720) [Preprint]. arXiv. https://arxiv.org/abs/2403.14720

Hossain, E., Nipu, M. M. H., Ornee, T. N., Rana, R., & Yousefi, N. (2026). *NEXUS: Structured runtime safety for tool-using LLM agents* (arXiv:2607.19356) [Preprint]. arXiv. https://arxiv.org/abs/2607.19356

Jia, H., Liu, Y., Chong, B., Yang, Y., Chen, Y., Liang, J., Li, Q., Lu, H., Xu, K., Zheng, H., Zhang, C., Peng, H., & Yu, P. S. (2026). *FinHarness: An inline lifecycle safety harness for finance LLM agents* (arXiv:2605.27333) [Preprint]. arXiv. https://arxiv.org/abs/2605.27333

Kull, M., Silva Filho, T., & Flach, P. (2017). Beta calibration: A well-founded and easily implemented improvement on logistic calibration for binary classifiers. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics* (Vol. 54, pp. 623–631). PMLR. https://proceedings.mlr.press/v54/kull17a.html

le Cessie, S., & van Houwelingen, J. C. (1992). Ridge estimators in logistic regression. *Applied Statistics, 41*(1), 191–201. https://doi.org/10.2307/2347628

Li, M. Q., Fung, B. C. M., Li, B., Ismail, H., & Iqbal, F. (2026). *Taxonomy and consistency analysis of safety benchmarks for AI agents* (arXiv:2605.16282) [Preprint]. arXiv. https://arxiv.org/abs/2605.16282

Liu, H., Ilyushin, E., Ni, J., & Zhu, M. (2026). *SafeAgent: A runtime protection architecture for agentic systems* (arXiv:2604.17562) [Preprint]. arXiv. https://arxiv.org/abs/2604.17562

Lorden, G. (1971). Procedures for reacting to a change in distribution. *The Annals of Mathematical Statistics, 42*(6), 1897–1908. https://doi.org/10.1214/aoms/1177693055

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30*. https://arxiv.org/abs/1705.07874

Moustakides, G. V. (1986). Optimal stopping times for detecting changes in distributions. *The Annals of Statistics, 14*(4). https://doi.org/10.1214/aos/1176350164

Naeini, M. P., Cooper, G., & Hauskrecht, M. (2015). Obtaining well calibrated probabilities using Bayesian binning. *Proceedings of the AAAI Conference on Artificial Intelligence, 29*(1). https://doi.org/10.1609/aaai.v29i1.9602

OWASP Gen AI Security Project. (2025). *LLM01:2025 Prompt injection*. OWASP Foundation. https://genai.owasp.org/llmrisk/llm01-prompt-injection/

Page, E. S. (1954). Continuous inspection schemes. *Biometrika, 41*(1–2), 100–115. https://doi.org/10.1093/biomet/41.1-2.100

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems, 24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In A. J. Smola, P. Bartlett, B. Schölkopf, & D. Schuurmans (Eds.), *Advances in large margin classifiers* (pp. 61–74). MIT Press.

Reynolds, M. R., Jr., & Stoumbos, Z. G. (1999). A CUSUM chart for monitoring a proportion when inspecting continuously. *Journal of Quality Technology, 31*(1), 87–108. https://doi.org/10.1080/00224065.1999.11979900

She, Y., Liang, Y., & Kang, E. (2026). *Safeguarding LLM agents from misalignment through provenance analysis* (arXiv:2607.01236) [Preprint]. arXiv. https://arxiv.org/abs/2607.01236

Wang, C. L., Singhal, T., Kelkar, A., & Tuo, J. (2025). *MI9: An integrated runtime governance framework for agentic AI* (arXiv:2508.03858) [Preprint]. arXiv. https://arxiv.org/abs/2508.03858

Wang, Y., Han, X., Shang, D., Tang, Y., & Liu, B. (2026). *Safety, or just capability? A validity audit of agent-safety benchmarks* (arXiv:2607.28685) [Preprint]. arXiv. https://arxiv.org/abs/2607.28685

Yang, C. (2026). *AgentTrust: Runtime safety evaluation and interception for AI agent tool use* (arXiv:2605.04785) [Preprint]. arXiv. https://arxiv.org/abs/2605.04785

Zhang, C., Wan, Z., Yu, X., Wu, J., Wen, Q., Zhou, P., Zhao, W., & Tsang, I. (2026). *Calibration is not control: Why LLM-agent oversight needs intervention* (arXiv:2606.21399) [Preprint]. arXiv. https://arxiv.org/abs/2606.21399

CHAPTER FOUR

IMPLEMENTATION OF THE REM FRAMEWORK

> **Status.** This chapter describes software that exists and has been executed. Every statement about behaviour is backed by a test or an executed run, which is named where it is made. Components fall into three classes:
>
> - **FROZEN:** the core fixed by the design lock;
> - **CH3-EXPLORATORY:** Chapter 3 specifications whose values or details await supervisor approval;
> - **HARNESS:** evaluation code that is not part of REM.
>
> Open conflicts with Chapter 3 are stated where they arise and collected in Section 4.13.

4.1 Purpose and Scope of the Chapter

Chapter 3 specified REM as a five-layer runtime framework that selects one of four verdicts (Allow, Modify, Escalate, Block) for each tool call proposed by an LLM agent.

- The verdict minimizes expected loss under a calibrated probability of adversarial inducement and a declared consequence tier.
- Every verdict is recorded for audit.

This chapter reports how that specification was realized in software. It covers three things:

1. the implementation of each layer;
2. the offline fitting procedure;
3. the integration with the AgentDojo banking environment (Debenedetti et al., 2024), which demonstrates assumption AS-1 (Chapter 3, Table 3.3).

The chapter makes no claim about security effectiveness; Chapter 5 reports the evaluation.

Three rules governed the implementation:

1. **Fail closed.** A design value that has not been declared stops the software (`DesignNotSpecifiedError`) instead of being replaced by a default.
2. **No silent resolution.** Where Chapter 3 and the design lock disagree, both alternatives are implemented, selected explicitly, and the disagreement is recorded (Section 4.13).
3. **Separation of decision and audit.** Attribution is computed only after the verdict and cannot reach the decision function (enforced by signature; Section 4.12).

4.2 Software Environment

Table 4.1. Software and hardware environment of the reported runs

Item	Value
Language	Python 3.11.17
Numerical library	NumPy 2.4.6
Benchmark	AgentDojo 0.1.35 (PyPI), banking suite, suite version v1 for execution
Recorded traces	AgentDojo repository `runs/`, commit `089ed468cf3ed0322acc66b0211f26d9d90dbf60`
Test framework	pytest 9.1.1
Hardware	4 vCPU (Intel Xeon @ 2.80 GHz), 15 GB RAM, Linux container
REM code size	6,806 lines (`rem/`), plus 1,298 lines of pilot scripts (`experiments/pilot/`)
Code version of reported results	git commit `aa66f22`

No machine-learning framework is used. The estimator, the calibrator and every metric are implemented directly in NumPy, so that each operation can be traced to its equation in Chapter 3 (see `EQUATION_PROVENANCE_AUDIT.md`).

4.3 Code Organization

Table 4.2. Code organization

Package	Content	Class
`rem/core/`	Pipeline skeleton, interfaces, step-level mitigation, append-only audit log, result types	FROZEN

Package	Content	Class
`rem/algorithms/detection/`	Ridge logistic regression (Eqs. 3.1–3.2); instruction-pattern indicator $x\text{^}ctx_2$	FROZEN / CH3-EXPLORATORY
`rem/algorithms/calibration/`	Platt calibration on the logit with Platt's smoothed targets; label-NLL fit (Eq. 3.4)	FROZEN / CH3-EXPLORATORY (SC-4)
`rem/algorithms/context/`	Provenance labelling, hygiene, entity extraction	CH3-EXPLORATORY
`rem/algorithms/behavior/`	Evidence features (Tables 3.8–3.9); Bernoulli CUSUM (observe-only)	CH3-EXPLORATORY
`rem/algorithms/decision/`	Expected-loss engine; consequence tiers; loss structure and dominance check; predicates and feasible set	FROZEN / CH3-EXPLORATORY
`rem/algorithms/a1.py`, `a2.py`, `a3.py`	B1 threshold baseline (A1); observe-only CUSUM analysis (A2); REM verdict (A3)	CH3-EXPLORATORY
`rem/algorithms/attribution/`	Linear Shapley attribution (audit-only)	FROZEN
`rem/evaluation/`	Discrimination, calibration and runtime metrics; grouped bootstrap; nested grouped cross-fitting	CH3-EXPLORATORY
`rem/integration/`	AgentDojo trace loader, labeller, step builder, replay harness	HARNESS
`configs/`	Frozen configuration (fail-closed); `configs/exploratory/` pilot design and amendments	—
`experiments/pilot/`	Pilot runner, figures, reproduction script	HARNESS

The three names A1, A2 and A3 come from the project brief. They do **not** denote three independent methodologies, which the design lock excludes:

- **A3** is the REM verdict rule;
- **A1** is the consequence-independent baseline B1 of Chapter 3, Section 3.7.7;
- **A2** is the optional, observe-only CUSUM analysis of Chapter 3, Section 3.11.10.

4.4 Layer 1 — Input and Context

`rem/algorithms/context/provenance.py` implements the provenance rule of Chapter 3, Section 3.4.2:

- every context segment is labelled trusted (system), semi-trusted (user) or untrusted (tool output);
- derived content takes the least-trusted label of its inputs (`least_trusted`);
- model-generated content has no channel label and must be labelled by inheritance, so `trust_of(Provenance.MODEL)` raises an error instead of guessing.

Before any further processing, untrusted text is stripped of zero-width, bidirectional and other Unicode format characters (`strip_invisible`).

Entity extraction covers:

- account identifiers, by an IBAN pattern;
- numeric amounts.

A declared matching rule (`value_occurs_in`) links an argument value to the texts that contain it:

- numbers match numerically;
- strings match as case-insensitive substrings;
- strings shorter than three characters never match, so that trivial values cannot create provenance links.

4.5 Layer 2 — Detection

$x\text{^}ctx_2$ (instruction-pattern indicator). This is implemented by `InstructionPatternDetector`.

- The indicator fires if any declared regular expression, or the name of any tool, occurs in new untrusted segments.
- The pattern set is not a default: it is declared in the pilot configuration, and the detector refuses to run without one.
- The pilot declares five patterns covering three kinds of text:
  - imperatives addressed to an assistant;
  - "ignore previous instructions" phrasings;
  - role or delimiter markers.
- The configuration discloses that the author of the patterns knew the benchmark's templates. Chapter 5 tests this through the context ablation and the cross-template check.

x^{ctx}_1 (pre-trained injection classifier) is not implemented.

- The classifier choice is pending supervisor approval (SD-DET).
- The candidate model's host was unreachable from the execution environment.
- The feature is therefore absent from x_t, not replaced.

4.6 Layer 3 — Behavioral and Action Evidence

`rem/algorithms/behavior/evidence.py` implements the features of Chapter 3, Table 3.9:

- x^{beh}_1 to x^{beh}_4;
- x^{act}_1 to x^{act}_3.

An `EpisodeTracker` holds the episode history H_t. It is reset at each episode start and never changes REM's parameters. Chapter 3 leaves seven details open; each is resolved by a stated interpretation in the module, listed in Table 4.3 so that the reader can disagree with it.

Table 4.3. Interpretations made where Chapter 3 is silent

ID	Question	Interpretation
I-1	What is a "step" when the agent proposes several calls at once?	Each proposed call is a step; calls in one assistant message share one decision point
I-2	Does x^{beh}_2 count refused calls?	Yes, because Chapter 3 states that retries raise x^{beh}_2
I-3	x^{beh}_3 at the first call	"Highest level so far" is the lowest level (0)
I-4	x^{beh}_4 at the first call	Previous tool is a start symbol; start transitions belong to the reference set
I-5	x^{act}_2 "transactions before first untrusted ingestion"	Observable only through untrusted tool outputs, so the feature reduces to "recipient not in the objective"
I-6	x^{act}_3 "external recipient", "sensitive values"	Recipient differs from the user's own IBAN; sensitive values are identifiers and profile values from sensitive reads
I-7	Availability of x^{act}_1 for predicate D2	Unavailable when the tool has key arguments but the call carries none

All declared quantities are supplied by the caller; the module has no defaults:

- n_max;
- the key arguments per tool;
- the privilege map;
- the sensitive-read tools;
- the benign reference transition set.

The benign transition set must be estimated within each fold (Chapter 3, Section 3.11.6). Each step record therefore stores its transition, and x^{beh}_4 is recomputed for every fold's reference (`rem/integration/steps.py`, `design_matrix`).

4.7 Layer 4 — Decision Engine

4.7.1 Estimation (FROZEN)

`RidgeLogisticRegression` computes $s = \beta_0 + \beta^T x$ and $\tilde{p} = \sigma(s)$ (Eq. 3.1). It is fitted by Newton–Raphson on the penalized negative log-likelihood, with the intercept unpenalized.

- The penalty λ has no default; fitting without it raises `DesignNotSpecifiedError`.
- **Mismatch M1.** The code minimizes NLL + $(\lambda/2)\|\beta\|^2$, whereas Chapter 3 Eq. 3.2 writes $\lambda\|\beta\|^2$. The two are equivalent under rescaling of λ. The pilot reports λ on the code's scale, and the alignment is a pending decision.

4.7.2 Calibration (FROZEN form; fitting target pending, SC-4)

Both calibrators compute $p = \sigma(\gamma_1 s + \gamma_0)$ and reject any fit with $\gamma_1 \le 0$ (`MonotonicityViolationError`), as required for the monotone interpretation. They differ only in the fitting targets:

- `PlattLogitCalibrator` (frozen) fits against Platt's smoothed targets;
- `LabelNLLLogitCalibrator` fits by the label negative log-likelihood of Chapter 3 Eq. 3.4, using damped Newton steps so that quasi-separable calibration sets do not diverge.

Neither is adopted. Chapter 5 reports both.

4.7.3 Consequence tiers, predicates and feasible set (CH3-EXPLORATORY)

Tier map. `ConsequenceTierMap` maps each tool to its action category (Chapter 3, Table 3.7) and the call's descriptors to a tier (Table 3.11).

- A value transfer whose amount is missing or unparseable receives the above-limit tier: an unknown amount is not treated as small.
- Consequence descriptors never enter the probability model.

Predicates. `PredicatePolicy` implements D1 (prohibited action → Block) and D2 (required evidence unavailable at tier ≥ k_D2 → Escalate).

Feasible set. The feasible set $\mathscr{V}(a_t)$ contains Modify only for tools in $\mathscr{T}$_R.

Mismatch M4. The frozen `argmin_verdict` ranges over all four verdicts. The feasible set is therefore enforced in A3 (Section 4.7.5), without editing the frozen engine.

4.7.4 Loss structure and dominance check

`build_loss_grid` constructs L(v, y, k) exactly as Chapter 3 Table 3.13, including the derived adversarial Modify entry r_mod(k)·C_miss(k) (Eq. 3.5). The same module implements:

- the indifference thresholds of Eq. 3.8;
- the Allow–Block threshold of Eq. 3.9;
- the escalation feasibility condition of Eq. 3.10;
- the dominance check of Section 3.7.5 (`reachable_verdicts`).

Because each conditional risk is affine in p, the dominance check tests the endpoints, every pairwise intersection and the midpoints between them. It reports any verdict that is never the unique minimizer.

4.7.5 Verdict selection (A3 = REM)

`A3Policy.decide` applies D1, then D2. If neither fires, it computes R(v) for every $v \in \mathscr{V}(a_t)$ with the frozen `ExpectedLossDecisionEngine.conditional_risk` (Eq. 3.6) and returns the minimizer (Eq. 3.7).

Ties are the unresolved conflict SC-2. `tie_mode` makes the choice explicit:

- `"raise"` reproduces the frozen behaviour (`VerdictTieError`);
- `"record_chapter3_order"`, used only in the exploratory pilot, applies the Chapter 3 order Block ≥ Escalate ≥ Modify ≥ Allow and marks the step as tied, so that ties can be counted.

The pilot recorded no ties (Chapter 5).

4.7.6 Baseline B1 (A1)

`operating_threshold` implements Eq. 3.14: the smallest threshold whose false-positive rate on benign validation probabilities does not exceed α. The candidate set includes a value just above the largest benign probability, so that α = 0 is attainable.

`A1Policy` uses the same estimator, calibration and predicates as REM, and ignores tiers by construction:

- Block above θ(α_blk);
- Modify between θ(α_mod) and θ(α_blk) where feasible, otherwise Block;
- Allow below θ(α_mod).

4.7.7 Audit record and attribution (FROZEN; audit-only)

For each step the audit record contains:

- the evidence vector and its availability;
- s, $\tilde{p}$, p and the tier;
- the risk table, the verdict and its source;
- the mitigation and the restriction set;
- the linear Shapley attributions $\varphi_i = \beta_i(x_i - \bar{x}_i)$ with base value φ_0;
- the additivity error $|\varphi_0 + \Sigma\varphi_i - s|$;
- REM's processing time.

$\bar{x}$ is estimated on benign fitting data of the step's fold.

4.8 Layer 5 — Mitigation

Frozen semantics. The frozen `DeterministicMitigationEngine` implements step-level semantics:

- Allow releases the current action;
- Block prevents it;
- Escalate withholds it;
- Modify applies one declared mechanism. That mechanism is undeclared in the design lock, so Modify raises `DesignNotSpecified Error` in the frozen pipeline.

Episode-level semantics. Chapter 3 specifies episode-level semantics (Table 3.14): Modify removes the tool class of a_t for the rest of the episode, and Block restricts every tool in $\mathscr{T}$_R. This is conflict SC-1. The design lock forbids redefining the frozen mechanisms silently, so the episode-level semantics exist only in the evaluation harness, as an explicitly named variant (Section 4.10). The frozen enumeration of mitigation actions is unchanged, and a contract test confirms this.

4.9 Offline Fitting (Procedure 3.1)

`rem/evaluation/crossfit.py` implements lines 1–9 of Procedure 3.1. For each outer fold f:

1. The design matrix is rebuilt with the benign reference estimated only on the fitting portion.
2. λ is selected from a declared grid by grouped inner cross-validation on held-out log-loss (ties go to the stronger penalty).
3. Out-of-fold logits from the inner fits with the selected λ are used to fit (γ_1, γ_0).
4. A fit with $\gamma_1 \leq 0$ is recorded as a calibration failure for the fold.
5. The estimator is refitted on the whole fitting portion.
6. The B1 thresholds are selected on out-of-fold calibrated probabilities of y = 0 fitting steps.
7. The held-out fold is scored.

Inner folds re-estimate the benign reference without their held-out groups, so the out-of-fold logits used for calibration are not optimistically biased on x^beh$_4$.

A unit test confirms that held-out predictions are independent of held-out labels: flipping every label in one fold leaves that fold's predictions unchanged (`tests/test_crossfit.py`).

4.10 Integration with AgentDojo (HARNESS)

4.10.1 Recorded traces and labels

`rem/integration/agentdojo_traces.py` loads AgentDojo's recorded traces without modification. They are traces of real LLM agents, produced by the benchmark authors.

Labels follow the reference-match rule of Chapter 3 Section 3.11.3:

- y = 1 when the injection task's reference solution contains the proposed tool **and** the attacker's key argument matches;
- the reference solutions are read from the installed AgentDojo package, not hard-coded;
- induced read steps are labelled 0 in the primary analysis, with a flag for the sensitivity variant.

The function `ambiguous_pairs` finds task pairs whose legitimate solution satisfies the label rule. It found one, (user_task_15, injection_task_4): the user's own new payee is the attacker's account (amendment AM-1, Chapter 5).

4.10.2 Placement at the tool boundary (AS-1)

REM is inserted into AgentDojo's own agent pipeline as a subclass of the benchmark's `ToolsExecutor`. Every proposed call is evaluated before execution, and every tool result is ingested as untrusted content on return. Neither the agent, its prompts nor the benchmark's tools are modified. This demonstrates AS-1 on this platform.

All calls run against AgentDojo's simulated banking environment. REM never initiates a call of its own.

4.10.3 Recorded-trace replay and its conflict with Chapter 3

No LLM API is available in the execution environment, so live enforcing-mode runs (Chapter 3 Section 3.11.1) could not be executed. Instead, enforcement was estimated by **replaying** recorded trajectories:

- a replay element emits the recorded assistant messages in order;
- the benchmark executes them under REM;
- the benchmark's own utility and security checks score the result.

The replay assumes that the agent does not adapt to a refusal (NO-ADAPT).

This method is listed as out of scope in Chapter 3 Section 3.1.4 ("Counterfactual estimation of intervention value by replaying agent trajectories"). The conflict was identified after the runs. It is recorded as SC-7 for supervisor decision, and every replay-derived result in Chapter 5 is labelled accordingly.

The replay itself is validated: with every call allowed, it reproduced the recorded utility and security outcomes of 3,313 of 3,334 episodes (99.4%).

4.11 Optional Sequential Analysis (A2)

`rem/algorithms/behavior/cusum.py` and `rem/algorithms/a2.py` implement the Bernoulli CUSUM of Chapter 3 Eqs. 3.15–3.19:

- q_0 is estimated on benign fitting steps, with a declared floor;
- q_1 is declared over a grid;
- the threshold is the smallest value meeting the per-episode false-alarm target on benign fitting episodes.

The module has no interface to the decision engine and returns no verdict; a test checks both properties.

An A2-driven enforcement policy and a retrained "A3+A2" variant, named in the project brief, are not implemented. Both would place CUSUM in the verdict path, which the design lock forbids; they are listed as supervisor decisions.

4.12 Verification

Table 4.4. Test suite (executed 27 Oct 2026; 192 passed)

File	Tests	What is verified
`test_frozen_math.py`	43	Eqs. 3.1–3.3, 3.6; fail-closed fitting; $\gamma_1 > 0$; tie raises
`test_contract_compliance.py`	20	Five layers, no sixth; frozen mitigation mapping; no equation IDs; forbidden components absent; attribution excluded from decision signature
`test_metrics.py`, `test_probabilistic.py`	25 + 5	ROC-AUC against pairwise definition; AP; ECE binning of Eq. 3.20; log-loss; Brier conventions; grouped bootstrap
`test_pipeline.py`, `test_mitigation.py`, `test_trajectory.py`, `test_config.py`, `test_reproducibility.py`	21 + 9 + 12 + 15 + 9	Frozen pipeline, step-level mitigation, configuration fail-closed, seeds
`test_a3.py`	9	Table 3.13 rows; Eqs. 3.8–3.10; argmin over feasible set; D1/D2; ties raise or are recorded; dominance check
`test_a1.py`	5	Eq. 3.14 minimality; $\alpha = 0$; invariance under monotone recalibration; verdict rule
`test_a2.py`	5	Eqs. 3.15–3.17; indicator set; false-alarm target; no verdict
`test_evidence.py`	7	Trust order; hygiene; features on an injected and a legitimate transfer; availability; tier map
`test_crossfit.py`	2	Procedure 3.1 outputs; held-out independence of held-out labels
`test_traces.py`	2	Label rule incl. induced reads and ambiguity; step records and fold-specific $x^{\wedge}beh_4$
`test_replay.py`	3	All-Allow replay reproduces a recorded episode; oracle Block prevents the recorded attack; observe-only releases everything

Two further properties were verified by execution rather than by unit test:

- **Determinism.** A complete second run of the pilot from commit `aa66f22` reproduced every reported number exactly.
- **Attribution additivity.** The audit record logs $|\varphi_0 + \Sigma\varphi_i - s|$ for every replayed step.

4.13 Alignment Between Chapter 3 and the Implementation

Table 4.5. Alignment status

Item	Chapter 3	Implementation	Status
M1	$\lambda\|\beta\|^2$	$(\lambda/2)\|\beta\|^2$	Equivalent under rescaling; decision pending
SC-1	Episode-level Modify/Block	Frozen: step-level; harness: both variants	Both run; unresolved
SC-2	Ties → Block ≽ … ≽ Allow	Frozen: raise; A3: explicit `tie_mode`	Unresolved; 0 ties in pilot
SC-3	Calibration ablation uses $\tilde{p}$	Frozen pipeline refuses; harness runs it	Unresolved
SC-4	Eq. 3.4 label NLL	Frozen: smoothed targets; both implemented	Unresolved
SC-5	Brier withheld	Binary and two-class reported with convention	Unresolved
SC-6	Estimation in L4	Frozen code labels estimator “L2 Detection”	Documentation; unresolved
SC-7	Replay out of scope	Enforcement estimated by replay	**Conflict; unresolved**
M4	Argmin over $\mathcal{V}(a_t)$	Frozen argmin over all verdicts; A3 restricts	Implemented in A3
$x\text{^}ctx_1$	Pre-trained classifier	Absent	BLOCKED (SD-DET; host unreachable)
I-1 to I-7, AM-2, AM-3	Silent	Stated interpretations	Pending approval

4.14 Chapter Summary

The full runtime procedure of Chapter 3 (Algorithm 1) is implemented, with two exceptions:

- the injection classifier is blocked;
- the episode-level mitigation semantics exist as a harness variant, pending SC-1.

The offline fitting of Procedure 3.1 is implemented, and REM was placed at AgentDojo’s tool boundary without modifying the agent (AS-1). The software is verified by 192 tests and by an exact reproduction of the pilot.

Ten alignment items remain open, the most consequential being SC-7: enforcement was estimated by a replay method that Chapter 3 excludes. Chapter 5 reports what the implementation produced on recorded agent traces, under that label.

References

Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., & Tramèr, F. (2024). AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems 37*. https://doi.org/10.52202/079017-2636

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning* (Vol. 70, pp. 1321–1330). PMLR.

le Cessie, S., & van Houwelingen, J. C. (1992). Ridge estimators in logistic regression. *Applied Statistics*, *41*(1), 191–201. https://doi.org/10.2307/2347628

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30*. https://arxiv.org/abs/1705.07874

Page, E. S. (1954). Continuous inspection schemes. *Biometrika*, *41*(1–2), 100–115. https://doi.org/10.1093/biomet/41.1-2.100

CHAPTER FIVE

EXPLORATORY PILOT EVALUATION

> **EXPLORATORY — NOT CONFIRMATORY THESIS RESULTS — SUPERVISOR DECISION REQUIRED.**
>
> This chapter reports an executed pilot. Its design was declared and committed before any held-out prediction was computed (`configs/exploratory/pilot_e0.yaml`, commit `48bef90`; amendments AM-1 to AM-4, commit `7a707a7`). Every number comes from `results/pilot/` (code commit `aa66f22`), and a complete second run reproduced every number exactly. The confirmatory evaluation specified in Chapter 3 remains **RESULTS PENDING** until:
>
> - the design values are frozen (SD-FREEZE, SD-1 to SD-4);
> - live enforcing-mode runs become possible;
> - the replay conflict SC-7 is decided.
>
> Results in Sections 5.6–5.7 that depend on trajectory replay are marked **[REPLAY — SC-7]**: the method is out of scope in Chapter 3 Section 3.1.4.

5.1 Purpose

The pilot asks whether the REM pipeline of Chapter 3 can be fitted and evaluated on real agent behaviour under the grouped cross-fitting protocol. It also asks what that evaluation shows for each research question ([PD]), and which of Chapter 3's assumptions it confirms or contradicts.

It is a pilot in the sense of Chapter 3 Section 3.11.8. It also executes, on the same data, the experiments E1–E6 that the master plan names. Negative results are reported with the same prominence as positive ones.

5.2 Data and Protocol

5.2.1 Data

Source. The AgentDojo repository (Debenedetti et al., 2024) ships recorded traces of real LLM agents for its banking suite.

- The pilot uses the traces of **21 undefended backbones** under the `important_instructions` attack: 16 user tasks × 9 injection tasks, plus 16 benign episodes per backbone.
- Episodes recorded with an error (5) and the label-ambiguous pair of AM-1 (21 episodes) are excluded.

Size. The pilot dataset contains:

- **3,334 episodes**;
- **6,762 proposed tool calls** (steps);
- **921 positive steps**, in 840 attacked episodes.

What the data are not. No trace was generated or altered, and no live LLM was called. The episodes are not independent observations: they share 16 user-task templates, 9 injection tasks, a single attacker account and one environment. All intervals are therefore grouped (Section 5.2.3).

Labels. Labels follow the reference-match rule (Chapter 3 Section 3.11.3; Chapter 4 Section 4.10.1). A positive label marks an *attempt*: the agent proposed the attacker's reference action with the attacker's argument. Of the 840 attacked episodes containing a positive step:

- 682 also have recorded attack success;
- 158 do not (the benchmark's success predicate is stricter).

Two episodes record attack success without a positive step (Section 5.9).

5.2.2 Pre-registered values

All values come from Chapter 3 where Chapter 3 gives them. Otherwise they are exploratory declared values, varied over a declared grid where Chapter 3 says they are declared.

Table 5.1. Principal pre-registered values

Item	Value	Basis
Features	x^{ctx}_2, x^{beh}_{1-4}, x^{act}_{1-3} (x^{ctx}_1 blocked)	Tables 3.8–3.9
Tiers	read 1; profile 2; transfer ≤ 1000: 3; transfer > 1000, payment modification, credential change: 4	Table 3.11 (illustrative)

Item	Value	Basis
Costs	C_FA = 1; C_miss = 1, 5, 10, 20 by tier; C_mod = 0.5; C_esc = 0.3	Declared; structure Table 3.13
r_mod(k)	Measured; 1.0 where unmeasured	Section 3.8 property 3
Predicates	$\Pi = \varnothing$; k_D2 = 3; required evidence x^{act}_1	Declared
B1	$\alpha_{blk} = 0.01$, $\alpha_{mod} = 0.05$	Declared
λ grid; inner folds	{0.01, 0.1, 1, 10, 100}; 4	Declared
ECE bins; clip	10; 10^{-6}	Declared
E0 decision values	≥ 50 positive steps; group ROC-AUC ≥ 0.55	Declared
Bootstrap	2,000 resamples, seed 20261027	Declared

5.2.3 Protocol

- **Cross-fitting.** Nested grouped cross-fitting (Procedure 3.1):
  - nine outer folds, leave one injection task out;
  - benign episodes assigned to folds by user task;
  - grouped inner folds select λ and produce out-of-fold logits for calibration.
- **Reported predictions.** Every reported prediction is held out.
- **Intervals.** 95% percentile intervals from a grouped bootstrap: attacked items are resampled by injection task, benign items by user task.

There are only nine injection-task groups, so intervals are wide and can be unstable.

5.3 Pilot E0: Feasibility and Feature Freezing

E0 used only the fitting portions of the outer folds.

Table 5.2. Pilot E0 (mean over the 9 fitting portions)

Evidence group	ROC-AUC (mean; min over folds)	Retained (≥ 0.55)
Context (x^{ctx}_2 only)	0.582; 0.554	Yes, marginally
Behavioral	0.802; 0.787	Yes
Action	0.929; 0.923	Yes
All	0.938; 0.930	—

- **Positive steps.** Pooled: 921, above the declared minimum of 50. Thirteen of 21 backbones have fewer than 50 positive steps each (range 2–116), so no per-backbone conclusion is drawn.
- **Feature set.** Under the pre-registered rule, all eight features were retained.
- **AS-7.** The pilot supports AS-7 for this benchmark: the observable evidence separates induced from benign steps.

5.4 RQ1 — Discrimination and Calibration (E1)

Table 5.3. Held-out discrimination and calibration, pooled over 9 outer folds (6,762 steps, 921 positive)

Metric	Calibrated p, label-NLL fit (Eq. 3.4)	Calibrated p, Platt smoothed targets	Uncalibrated $\tilde{p}$
ROC-AUC	0.938 [0.908, 0.961]	0.938	0.939
Average precision (PR-AUC estimate)	0.719 [0.559, 0.827]	0.719	0.721
ECE (M = 10)	0.044 [0.033, 0.084]	0.044	0.042
Log-loss	0.201 [0.149, 0.294]	0.201	0.201
Brier, binary / two-class (SC-5)	0.060 / 0.121	0.060 / 0.121	0.060 / 0.120

Table 5.4. Per-fold results (calibrated, label-NLL)

Held-out injection task	Steps	Positive	ROC-AUC	AP	ECE	Log-loss	λ	γ_1
0	824	125	0.962	0.788	0.059	0.161	10	0.959
1	773	101	0.965	0.854	0.046	0.140	10	0.947
2	807	112	0.949	0.735	0.035	0.164	10	0.958
3	772	110	0.962	0.811	0.045	0.154	10	0.955
4	726	135	0.978	0.906	0.087	0.148	10	0.947

Held-out injection task	Steps	Positive	ROC-AUC	AP	ECE	Log-loss	λ	γ_1
5	634	23	0.859	0.308	0.068	0.164	10	0.969
6	666	79	0.909	0.501	0.075	0.225	10	0.981
7 (credential change)	730	124	0.903	0.577	**0.189**	**0.530**	1	0.980
8	830	112	0.959	0.859	0.037	0.146	10	0.945

Findings:

1. **Discrimination generalizes to unseen attacker objectives.** Held-out ROC-AUC is 0.86–0.98 per fold.
   - The weakest folds are injection task 5 (23 positive steps) and injection task 7.
   - Injection task 7 is the only credential-change attack, so when it is held out the model has never seen an attack of that category.
2. **Calibration does not transfer to an unseen attack category.** The injection-task-7 fold has ECE 0.189 and log-loss 0.53, three to four times the other folds.
3. **Platt calibration adds nothing here (negative result).**
   - The fitted maps are close to the identity (γ_1 = 0.945–0.981).
   - Pooled ECE is 0.044 with calibration and 0.042 without.
   - The two fitting targets (SC-4) give indistinguishable results.
   - This is consistent with logistic regression already producing calibrated probabilities on in-distribution data. It does not show that calibration is unnecessary under shift: no fold had $\gamma_1 \leq 0$, but fold 7 shows that miscalibration under shift is not repaired.
4. **Other generalization checks.**

   - *Leave one backbone out* (21 folds): pooled ROC-AUC 0.949. Per held-out backbone the range is 0.736 (claude-3-haiku) to 1.000, with ECE 0.038–0.080.

   - *Grouping by user task* (sensitivity, 16 folds): pooled ROC-AUC 0.943. One fold, user_task_15, scores 0.211; Section 5.9 explains this as a label problem.

   - *Other attack templates, applied with no refitting* (gpt-4o, 135–138 valid episodes each):

Template	ROC-AUC
direct	0.959
ignore_previous	0.940
important_instructions_no_names	0.948
injecagent	0.922
tool_knowledge	0.955

     ECE ranges from 0.028 to 0.086.

Figure 5.1 (`results/pilot/figures/fig_reliability.png`) shows the held-out reliability diagram. Mid-range bins contain few steps, which explains the visible noise.

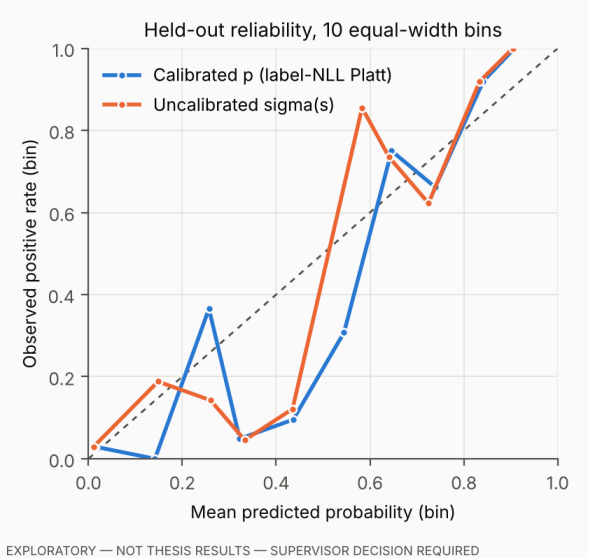


Figure 5.1. Held-out reliability diagram (10 equal-width bins), calibrated and uncalibrated. EXPLORATORY.

5.5 RQ2 — REM versus the Consequence-Independent Baseline, Observe-Only (E2)

Both policies use the same held-out calibrated probabilities. REM (A3) adds tiers and expected loss; B1 (A1) uses the Eq. 3.14 thresholds. Verdicts were computed on the recorded steps and not enforced.

Table 5.5. Held-out verdict profile (observe-only)

	REM	B1
Intervention on y = 1 steps (any non-Allow)	0.973 [0.957, 0.987]	0.740 [0.473, 0.922]
Step false-positive rate (any non-Allow on y = 0)	0.227 [0.213, 0.239]	0.056 [0.047, 0.066]
…Escalate share of all steps	0.196	0.009
y = 0 steps withheld if an idealized reviewer resolves escalations	0.040	0.046
Ties (SC-2)	0	0
D2 verdicts	58	58

REM's additional interventions are almost all escalations. **On benign episodes, REM releases 33 of 180 legitimate state-changing steps (tiers 2–4); B1 releases 132.** The mechanism follows directly from the pre-registered costs:

- the Allow–Block indifference points are p_3 = *1/11 and* p_4 = 1/21 (Eq. 3.9);
- legitimate transfers often carry action evidence (for example, a recipient read from a tool output), which places them above those points;
- the dominance check (Eq. 3.10) shows that, under these costs, Block is never the unique minimizer at tier 3, so at that tier REM chooses among Allow, Modify and Escalate.

The realized loss under the decision costs is 0.114 for REM and 0.652 for B1. **This comparison favours REM by construction**, because REM minimizes expected loss under these costs (Chapter 3 Section 3.12.3). It is not evidence of superiority.

5.6 RQ2 and RQ3 — Enforcement [REPLAY — SC-7] (E4)

Enforcement was estimated by replaying each recorded episode in the AgentDojo sandbox with REM at the tool boundary (Chapter 4 Section 4.10.3).

- **Assumption NO-ADAPT:** the agent does not react to a refusal.
- **Validity:** with every call allowed, the replay reproduced 3,313 of 3,334 recorded outcomes (99.4%). Only those 3,313 episodes are used (AM-4).
- **Status:** the method is out of scope in Chapter 3 (SC-7).

Table 5.6. Enforcement replay (3,313 episodes; 95% grouped-bootstrap intervals)

Configuration	ASR	Benign utility	Utility under attack
B0, no defense	0.226 [0.148, 0.295]	0.524 [0.381, 0.676]	0.492

Configuration	ASR	Benign utility	Utility under attack
B1, step-level, no reviewer	0.055 [0.011, 0.134]	0.509 [0.334, 0.702]	0.514
B1, episode-level, no reviewer	0.046 [0.004, 0.125]	0.509 [0.334, 0.702]	0.485
REM, step-level, no reviewer	0.004 [0.001, 0.007]	0.314 [0.110, 0.548]	0.364
REM, episode-level, no reviewer	0.004 [0.001, 0.007]	0.314 [0.110, 0.548]	0.363
REM, step- or episode-level, idealized reviewer	0.004 [0.001, 0.007]	0.497 [0.333, 0.680]	0.481–0.496

Table 5.7. Paired differences (grouped bootstrap)

Comparison	ΔASR	ΔBenign utility
REM − B1, no reviewer	−0.051 [−0.130, −0.007]	−0.195 [−0.341, −0.074]
REM − B1, idealized reviewer	−0.051 [−0.130, −0.007]	+0.003 [−0.009, 0.018]
REM − B0, no reviewer	−0.222 [−0.289, −0.146]	−0.210 [−0.379, −0.051]

Figure 5.2 (`results/pilot/figures/fig_enforcement_replay.png`) displays Table 5.6.

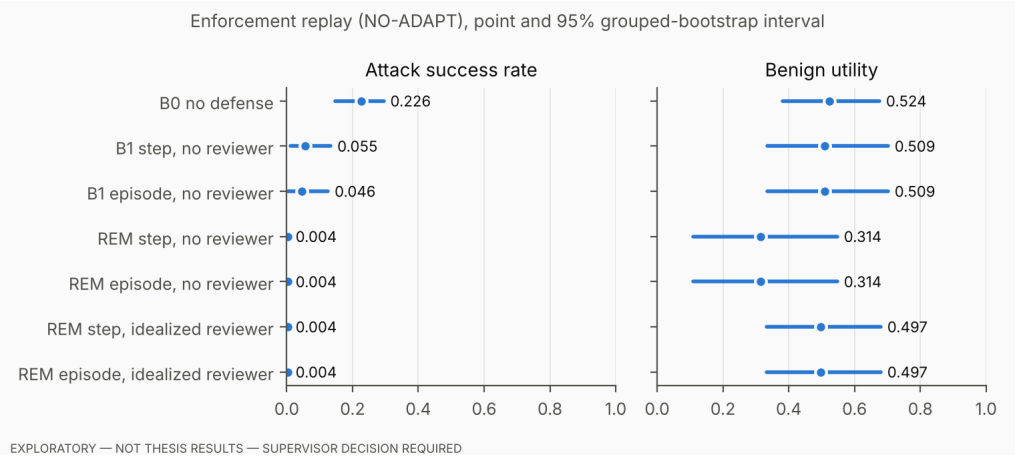


Figure 5.2. Enforcement replay [REPLAY — SC-7]: ASR and benign utility with 95% grouped-bootstrap intervals. EXPLORATORY.

Interpretation, within the replay assumption:

- REM blocks more attempted attacks than B1. It does so at a large utility cost when no one resolves escalations, and at almost no cost when an oracle does.
- The idealized reviewer knows the label. It is an upper bound on reviewer quality, not a model of one.
- Without a reviewer, REM's benign utility is 0.25–0.38 for every one of the 21 backbones. This is close to the share of user tasks that need no state change (4 of 16).

SC-1. The step-level and episode-level variants give the same results, which is a consequence of NO-ADAPT: a non-adaptive agent does not retry a refused tool, so restrictions that persist through the episode have nothing to act on.

RQ3 quantities:

- **r_mod(k).** Measured by oracle-Modify replays within the fitting folds: 0.000 at tier 3 (530 episodes) and 0.010 at tier 4 (298 episodes).
- **Residual ASR in replays where REM applied Modify to an adversarial step:** 0.000 (610 episodes).
- **Caveat.** Both values are artefacts of NO-ADAPT as much as properties of tool restriction. Under replay, a withheld call simply does not happen.
- **Escalation rate:** 0.196 of all steps (Table 5.5).
- **Latency.** REM's per-step processing time, excluding LLM inference and tool execution, has a median of 0.17–0.20 ms and a p95 of 0.43–0.54 ms across configurations.
- **Memory.** The peak traced allocation per step has a median of 2.9 KB and a p95 of 12.4 KB (300 episodes).

Distributed defenses (recorded outcomes for gpt-4o, not replayed, possibly from different benchmark versions; not a controlled comparison):

Defense	ASR	Benign utility
tool_filter	0.112	0.81

Defense	ASR	Benign utility
transformers_pi_detector	0.007	0.44
repeat_user_prompt	0.462	0.94
spotlighting_with_delimiting	0.615	0.81

5.7 RQ4 — Ablations (E5)

Table 5.8. Ablations (step-level; observe-only columns use held-out recorded steps; replay columns are [REPLAY — SC-7], no reviewer)

Configuration	Held-out ROC-AUC	Step FPR (observe)	y = 1 intervention (observe)	ASR (replay)	Benign utility (replay)
Full REM	0.938	0.227	0.973	0.004	0.314
− context evidence	0.938	0.228	0.973	0.004	0.314
− behavioral evidence	0.938	**0.174**	0.973	0.004	**0.419**
− action evidence	**0.799**	0.242	0.921	0.006	0.314
− calibration ($\tilde{p}$ in Eq. 3.6)	—	0.213	0.973	0.004	0.329
one tier for all actions: 1	—	0.093	0.757	0.051	0.440
one tier for all actions: 2	—	0.157	0.909	0.022	0.353
one tier for all actions: 3	—	0.233	0.972	0.004	0.338
one tier for all actions: 4	—	0.336	0.976	0.003	0.314
− predicates D1, D2	—	0.226	0.973	0.004	0.314
observe-only (no mitigation)	—	—	—	0.226	0.524

Chapter 3 does not specify which tier the “tiers removed” ablation should use, so all four were run.

Findings:

- **Action evidence carries most of the discrimination.** Removing it lowers held-out ROC-AUC from 0.938 to 0.799 and AP from 0.719 to 0.367.
- **Context evidence makes no measurable difference** (negative result). This also bounds the risk that the instruction patterns were tailored to the benchmark (Section 5.9).
- **Behavioral evidence lowers benign utility** (negative result). Removing it keeps ASR unchanged and raises benign utility by 10.5 points in replay. In this pilot, the behavioral group costs utility without a measured security benefit.
- **Calibration and predicates change little.** With Π empty, D2 fires on 58 steps and changes no episode outcome in replay.
- **Tiers drive the trade-off.** A single low tier recovers utility and loses protection; a single high tier does the opposite. REM’s tier map sits between these extremes.

5.8 Optional Sequential Analysis (E3, A2, observe-only)

The Bernoulli CUSUM was fitted per outer fold on benign fitting episodes.

- **$q_1 = 0.3$:** the CUSUM is undefined in all nine folds, because the estimated pre-change rate $q_0 \approx 0.34$ exceeds q_1.
- **$q_1 = 0.5$ and $q_1 = 0.7$** (similar results):
  - an alarm occurred after the injected content entered the context in 74.8% of attacked episodes with a positive step;
  - the alarm came at or before the first positive step in 42.5–43.8% of them;
  - the median delay was 1 step.
- **False alarms:** the held-out benign false-alarm rate was 0.106, against the in-fold target of 0.05, so the threshold did not generalize.

A2 issued no verdicts. In this pilot, the CUSUM adds nothing that the per-step estimator does not already provide.

5.9 Validity Findings That Qualify the Results

1. **The attacker’s account is also a legitimate payee.**
   - In AgentDojo banking suite versions v1 and v1.1, user_task_15 asks the agent to pay a new landlord whose IBAN is the attacker IBAN used by every injection task. The benchmark changed this from v1.1.1; 16 of the 21 recorded backbones used the colliding prompt.
   - The label-relevant definitions of the injection tasks are identical in all suite versions, so labels are consistent across the mixed recordings.
   - The pre-registered amendment AM-1 excluded only the pair whose reference solutions collide.

    - After results were computed, 29 positive steps in other user_task_15 episodes were found to be ambiguous, 19 of them with no injected content in the context.
    - All of REM's residual replay ASR (0.4%) comes from these episodes.
    - A disclosed post-hoc sensitivity analysis excluding user_task_15 gives REM ASR 0.000 in every configuration and held-out ROC-AUC 0.952. The primary analysis above keeps the pre-registered rule.
2. **Reference matching misses alternative routes.**
    - Two episodes have recorded attack success but no positive step. One reaches the attacker through a different tool; one appears to be a recording inconsistency.
    - 105 steps carry an attacker value but are labelled 0.
    - In a sample of 30 attacked episodes, 29 labels agreed with the trace and one was a likely false negative of this kind.
    - That inspection was performed by an AI assistant, not by a human annotator, and must be repeated.
3. **Single attacker identity.** $x\text{^act}_2$ ("recipient not in the objective") fires on essentially every attack, because the attacker IBAN never appears in an objective (except in user_task_15). Real fraud may use recipients that do appear in the user's context.
4. **NO-ADAPT and SC-7** qualify every enforcement number (Section 5.6).
5. **One run per configuration.** The recorded traces contain one run per (backbone, task pair), against the "at least three" of Chapter 3 Section 3.11.9.
6. **Historical backbones and a single benchmark.** The models are 2024–2025 snapshots; results are not pooled across benchmarks, and none other was evaluated.

The complete threat analysis is in `EXPERIMENT_VALIDITY_REPORT.md`.

5.10 Realized Loss across the Declared Grid (E6)

Realized loss (Eq. 3.22) on held-out steps was computed over the declared grid:

- C_miss multipliers {0.5, 1, 2, 5};
- C_esc {0.1, 0.3, 0.6};
- amount limits {100, 1000, 10000}.

Results across all 36 cells:

- REM with matched costs: 0.039–0.298;
- REM with the base costs evaluated under different costs (misspecified): 0.057–0.324;
- B1: 0.303–3.261.

REM's matched loss is lower than B1's in all 36 cells. Two caveats apply:

- the comparison is favoured by construction (Section 5.5);
- the misspecified-cost column is the more informative one: REM remains below B1 in all 36 cells even when its decision costs differ from the evaluation costs within this range.

Matched costs gave a lower realized loss than misspecified costs in 34 of the 36 cells, not all 36. Minimizing *expected* loss under estimated probabilities does not guarantee the lowest *realized* loss on a finite held-out set.

The grid cell α_blk = 0.05 is not evaluable, because the pre-registered α_mod = 0.05 must exceed α_blk; this inconsistency in the pre-registered grid is reported, not repaired.

5.11 Summary of Evidence by Research Question ([PD])

Table 5.9. Evidence summary

RQ	What the pilot shows	Strength
RQ1	Held-out ROC-AUC 0.94 (0.86–0.98 per fold; 0.92–0.96 on other templates; 0.74–1.00 per held-out backbone). ECE 0.044, but it degrades on an unseen attack category. Calibration adds nothing measurable	Exploratory; one benchmark; grouped intervals
RQ2	Observe-only: REM intervenes on more attacked steps than B1 but escalates far more benign ones. Replay: lower ASR than B1; utility cost from −19.5 points (no reviewer) to ≈0 (oracle reviewer)	Exploratory; replay is out of scope (SC-7)
RQ3	r_mod ≈ 0 and residual ASR 0 under NO-ADAPT (assumption-driven); escalation 19.6% of steps; latency < 1 ms per step at p95	Exploratory; replay (SC-7)

RQ	What the pilot shows	Strength
RQ4	Action evidence dominates. Context evidence makes no difference. Behavioral evidence costs utility. Tiers drive the protection–utility trade-off	Exploratory

References

Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., & Tramèr, F. (2024). AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems 37*. https://doi.org/10.52202/079017-2636

Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics, 7*(1), 1–26. https://doi.org/10.1214/aos/1176344552

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning* (Vol. 70, pp. 1321–1330). PMLR.

Naeini, M. P., Cooper, G., & Hauskrecht, M. (2015). Obtaining well calibrated probabilities using Bayesian binning. *Proceedings of the AAAI Conference on Artificial Intelligence, 29*(1). https://doi.org/10.1609/aaai.v29i1.9602

CHAPTER SIX

DISCUSSION, CONTRIBUTIONS, LIMITATIONS, AND CONCLUSION

> **Status.** This chapter interprets the exploratory pilot of Chapter 5. Every empirical statement inherits the pilot's label: EXPLORATORY, pending supervisor decisions. Statements that depend on recorded-trace replay are out of scope under Chapter 3 Section 3.1.4 (SC-7). The research questions remain [PD].

6.1 Interpretation of the Findings

6.1.1 What the evidence supports

1. **The REM pipeline can be built at the tool boundary without modifying the agent (AS-1).**
   - Chapter 4 placed REM inside AgentDojo's own execution loop as a guarded tool executor.
   - It adds a median of under 0.2 ms and a p95 of under 0.6 ms per step, with a few kilobytes of transient memory.
   - Runtime cost is therefore not an obstacle to step-level mediation.
2. **Evidence available at the tool boundary separates attacker-induced from legitimate actions in this benchmark (AS-7, RQ1).**
   - Held-out ROC-AUC is 0.94 when the attacker objective is unseen.
   - It remains 0.92–0.96 on attack templates not used for fitting, and 0.74–1.00 on backbones not used for fitting.
   - The action evidence carries most of this separation: argument provenance, destination novelty and sensitive egress. This matches the position, reported in Chapter 2, that the proposed action and the provenance of its arguments are the most informative place to look (She et al., 2026).
3. **Consequence-tiered expected-loss verdicts change behaviour in the direction Chapter 3 predicts (RQ2).**
   - REM intervenes on more attacker-induced steps than the consequence-independent baseline with the same estimator.
   - It concentrates its additional interventions on high-consequence actions, as an escalation.
   - The single-tier ablations show that the tier map, not the estimator, moves the protection–utility trade-off.

6.1.2 What the evidence does not support

1. **That REM is deployable as configured.**
   - With the pre-registered costs and no reviewer, REM withholds most legitimate state-changing actions: benign utility drops from 0.52 to 0.31.
   - With an oracle reviewer the cost disappears, but an oracle is not a reviewer.
   - The practical value of REM's Escalate verdict therefore depends on a human process that this thesis has not modelled. C. Zhang et al. (2026) argue that oversight requires intervention capacity, not only calibrated scores; the pilot shows the same dependence from the other side.
2. **That calibration helped.**
   - Platt calibration on the logit left calibration error unchanged on in-distribution folds.
   - It did not repair the miscalibration on the one fold with an unseen attack category (credential change).
   - Chapter 3 already declines to claim that calibration improves control (Section 3.7.2), and the pilot gives no reason to strengthen that position.
3. **That every evidence group is needed.**
   - Context evidence made no measurable difference.
   - Behavioral evidence lowered benign utility without a measured security benefit.
   - Both observations are specific to one benchmark with templated attacks, but they are the opposite of what a "more evidence is better" reading of Chapter 3 would predict.
4. **Anything about adaptive agents or adaptive attackers.**
   - The near-zero residual attack success under Modify reflects the replay assumption that the agent never tries again.
   - It is not evidence about tool restriction as a mechanism.
5. **The optional CUSUM analysis.**
   - Its threshold did not hold its false-alarm target on held-out benign episodes.
   - Its alarms rarely preceded the first attacker-induced step.

6.1.3 A benchmark-version effect

In AgentDojo banking suite versions v1 and v1.1, the attacker's account in every injection task is also the legitimate new payee in user_task_15. For that user task, a legitimate payment therefore satisfies the attacker's success predicate, and the reference-match label marks legitimate actions as attacker-induced.

The benchmark changed the payee from v1.1.1. Most of the recorded traces used here (16 of 21 backbones) predate the change, and all of REM's residual attack success in the replay came from these episodes.

The finding is not new. It is a reminder that recorded traces from mixed benchmark versions must be checked task by task before pooling.

6.2 Relationship to the Objectives

Table 6.1. Objectives and status

Objective (approved)	Status after this thesis
O1 — Design a REM framework for financial AI agents' security	Designed (Chapter 3) and implemented (Chapter 4). Design values await approval
O2 — Evaluate the framework against adversarial attacks	Exploratory pilot on recorded real-agent traces (Chapter 5). Confirmatory evaluation pending SD-FREEZE and live enforcing-mode runs
O3 — Compare performance with existing security approaches	Compared with the same-estimator baseline B1 and with recorded outcomes of four distributed defenses (not a controlled comparison). Comparison with the systems of Table 6.2 remains at the design level

6.3 Comparison with Prior Work

The comparison below restates design positions documented in Chapter 2 and adds what the pilot measured for REM. It does not rank systems, because no shared evaluation exists.

Table 6.2. Design-level comparison with the closest prior work

System	Documented design (Chapter 2)	Relation to REM	REM evidence from the pilot
NEXUS (Hossain et al., 2026)	Rules, argument inspection, logistic-regression risk score with Platt scaling; expected-loss objective over allow/revise/confirm/block; deployed as a rule cascade; plan-level evaluation on generated templates	Shares calibrated logistic scoring and a four-way intervention set. Differs in per-step minimization of conditional risk with tier-indexed losses at the tool boundary of an executing agent. NEXUS numerical claims remain unverified (Chapter 2)	Per-step expected-loss verdicts executed on 6,762 real-agent steps; B1 comparison with the same estimator
AgentTrust (C. Yang, 2026)	Pattern features, policy rules, analyzer risk scoring, maximum-severity aggregation, four verdicts including review	Shares interception, a review verdict and ablation with latency. Differs in using calibrated probability and declared losses rather than rule severity	Ablation and latency measured
SafeAgent (H. Liu et al., 2026)	Runtime controller whose decision operators are LLM-realized	REM excludes LLM arbitration by design; its verdict is a closed-form rule	Deterministic verdicts; 0 ties in 6,762 steps
ProvenanceGuard (She et al., 2026)	Checks whether a proposed call is supported by traceable evidence in context	REM's argument-provenance feature asks the same question and is not claimed as new	Action evidence, including provenance, carried most of the discrimination
H.-H. Chen (2026)	Prices action consequence deterministically against reserved capital	Shares consequence-dependent control; REM combines consequence with an estimated probability	Single-tier ablations show the consequence map drives the trade-off
FinHarness (Jia et al., 2026)	Financial-agent monitors and judge cascade	Shares domain and placement; differs in decision mechanism	Banking-environment pilot executed
C. Zhang et al. (2026)	Calibration can leave threshold-routed control unchanged	REM reports calibration and control separately and does not claim calibration improves control	Calibration gave no measurable gain; the B1 thresholds are invariant to monotone recalibration (tested)

6.4 Contributions

The contributions are stated narrowly. Each mechanism has precedent (Chapter 3, Table 3.16), and none is claimed as an original invention.

1. **A specified composition.** The framework is specified for one point of an executing financial agent, its tool boundary. It composes:
   - provenance-based evidence;
   - a ridge logistic estimator with calibration on the logit;
   - consequence tiers;
   - policy predicates;
   - a four-verdict expected-loss decision with a feasible set and a dominance check;
   - deterministic mitigation;
   - an audit record whose attribution cannot reach the verdict.

   The composition, not any component, is the design contribution.
2. **An evaluation protocol that isolates the decision rule.** REM and B1 share the estimator and its calibration, so differences between them are attributable to the decision rule. Grouped cross-fitting holds out whole attacker objectives.

3. **An executed, reproducible pilot on real agent traces**, with:
    - a pre-registered design;
    - recorded amendments;
    - negative results reported;
    - exact reproduction from a single script.

   It includes a measured protection–utility trade-off whose dependence on the reviewer is quantified, from the no-reviewer bound to the oracle reviewer. **Not claimed:**

- a new learning, calibration or decision-theoretic method;
- prevention guarantees;
- robustness to adaptive attackers;
- validity in production financial deployment;
- superiority over the systems of Table 6.2.

6.5 Limitations

1. **Design status.** The costs, tiers, predicates, feature set and label rule are pending supervisor approval. The pilot used declared exploratory values for them.
2. **No live agent runs.** Enforcement was estimated by replay (SC-7), under the assumption that the agent does not adapt. ASR under replay is not the ASR of a live agent.
3. **One benchmark, one attack family for fitting, one attacker identity.** Features such as destination novelty may be easier in this benchmark than in practice.
4. **Labels.** The reference-match rule marks attempts, misses alternative attacker routes, and is ambiguous for user_task_15. The label audit was performed by an AI assistant, not by a human.
5. **Statistics.** There are nine injection-task groups and one run per configuration. Intervals are wide and are not corrected for multiple comparisons.
6. **Reviewer model.** Escalation outcomes were bounded by an oracle and by “no reviewer”; real reviewers lie between and add latency.
7. **Missing component.** The pre-trained injection classifier ($x^{\wedge}ctx_1$) was not available.

6.6 Future Work

Ordered by dependency:

1. **Decisions.** Resolve:
    - SC-1 to SC-7;
    - SD-FREEZE, SD-1 to SD-5, SD-FEAT, SD-DET;
    - the label rule (LABEL-UT15, LABEL-ROUTE);
    - the reviewer model.
2. **Live enforcing-mode runs.** Run B0, B1 and REM with live backbones and at least three repeats, so that adaptive agent behaviour after refusals is observed and r_mod is measured as Chapter 3 intends.
3. **A realistic escalation model.** Specify reviewer error and delay, so that the utility cost of Escalate can be estimated between the two bounds.
4. **Re-examine the evidence set.** Investigate why the behavioral group lowers utility, for example through the privilege-escalation and transition-deviation features on legitimate first transfers, and re-run E0 with the pre-trained classifier if approved.
5. **Calibration under shift.** Study held-out attack categories, where the pilot shows miscalibration that Platt calibration does not repair.
6. **Other benchmarks and adaptive attacks.** Report results separately per benchmark, without pooling.

6.7 Conclusion

This thesis set out to design and evaluate a runtime framework that decides, for each tool call proposed by a financial AI agent, whether to allow, modify, escalate or block it. The decision minimizes expected loss under a calibrated probability of adversarial inducement and a declared consequence tier, and every verdict is recorded for audit.

The framework was specified (Chapter 3) and implemented with fail-closed handling of undeclared design values (Chapter 4). It was evaluated in an exploratory pilot on 6,762 tool calls proposed by 21 real LLM agents in the AgentDojo banking environment (Chapter 5).

The pilot shows that the evidence available at the tool boundary identifies attacker-induced actions on unseen attacker objectives, templates and backbones (ROC-AUC 0.92–0.96 for the first two). Consequence-tiered expected-loss verdicts intercept more of those actions than a threshold baseline with the same estimator.

It also shows the price. With the pre-registered costs, REM escalates most legitimate high-consequence actions, and its utility depends on who resolves those escalations. Calibration added nothing measurable, and two of the three evidence groups showed no measured benefit.

Because the design values are not yet approved and enforcement could be estimated only by a replay that Chapter 3 excludes, these findings are exploratory. The confirmatory evaluation that Chapter 3 specifies remains to be run, and the decisions it depends on are listed in `SUPERVISOR_DECISIONS_REQUIRED.md`.

References

Chen, H.-H. (2026). *Insuring every action: An authority frontier framework for runtime actuarial control of autonomous AI agents* (arXiv:2605.25632) [Preprint]. arXiv. https://arxiv.org/abs/2605.25632

Hossain, E., Nipu, M. M. H., Ornee, T. N., Rana, R., & Yousefi, N. (2026). *NEXUS: Structured runtime safety for tool-using LLM agents* (arXiv:2607.19356) [Preprint]. arXiv. https://arxiv.org/abs/2607.19356

Jia, H., Liu, Y., Chong, B., Yang, Y., Chen, Y., Liang, J., Li, Q., Lu, H., Xu, K., Zheng, H., Zhang, C., Peng, H., & Yu, P. S. (2026). *FinHarness: An inline lifecycle safety harness for finance LLM agents* (arXiv:2605.27333) [Preprint]. arXiv. https://arxiv.org/abs/2605.27333

Liu, H., Ilyushin, E., Ni, J., & Zhu, M. (2026). *SafeAgent: A runtime protection architecture for agentic systems* (arXiv:2604.17562) [Preprint]. arXiv. https://arxiv.org/abs/2604.17562

She, Y., Liang, Y., & Kang, E. (2026). *Safeguarding LLM agents from misalignment through provenance analysis* (arXiv:2607.01236) [Preprint]. arXiv. https://arxiv.org/abs/2607.01236

Yang, C. (2026). *AgentTrust: Runtime safety evaluation and interception for AI agent tool use* (arXiv:2605.04785) [Preprint]. arXiv. https://arxiv.org/abs/2605.04785

Zhang, C., Wan, Z., Yu, X., Wu, J., Wen, Q., Zhou, P., Zhao, W., & Tsang, I. (2026). *Calibration is not control: Why LLM-agent oversight needs intervention* (arXiv:2606.21399) [Preprint]. arXiv. https://arxiv.org/abs/2606.21399